Università Ca'Foscari Venezia

Deep Learning

FOUNDATIONS OF ARTIFICIAL INTELLIGENCE

Andrea Torsello

Dip. Sc. Ambientali, Informatica, Statistica
Università Ca'Foscari Venezia

What is Deep Learning

Good old Neural Networks (with more layers/modules)

Non-linear, hierarchical, abstract representations of data

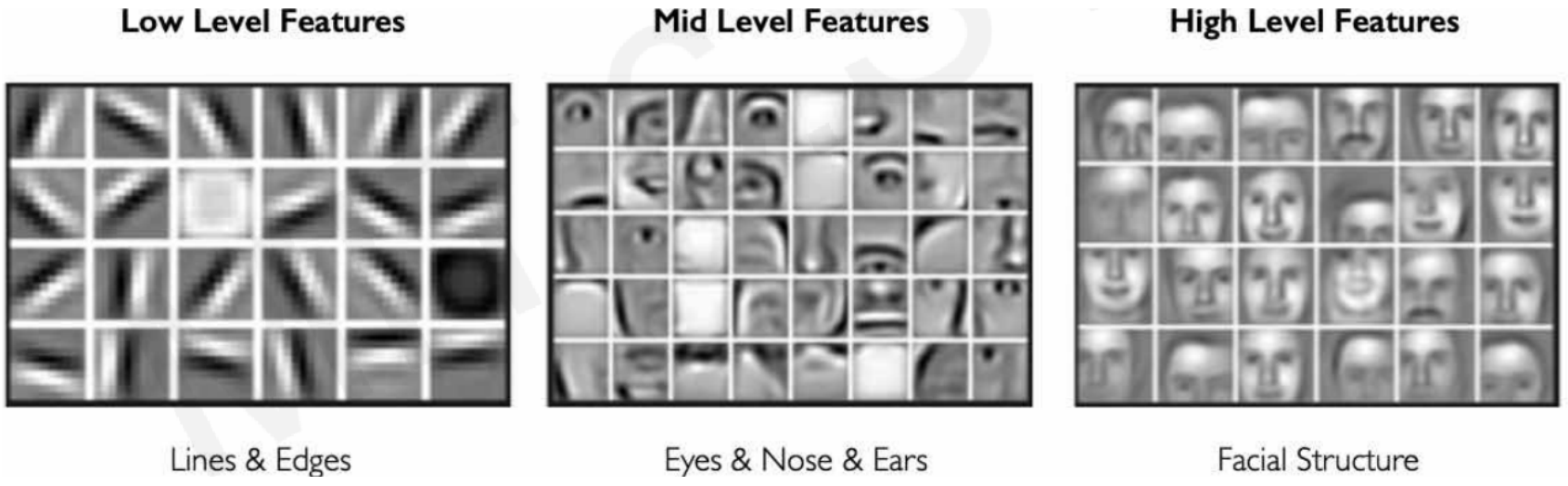
Flexible models with any input/output type and size

Differentiable Functional Programming

Why Deep Learning?

Hand Engineered features are time-consuming, brittle, and not scalable

Learn the **underlying features** from data



Università
Ca'Foscari
Venezia

Why Now?

Better algorithms & understanding

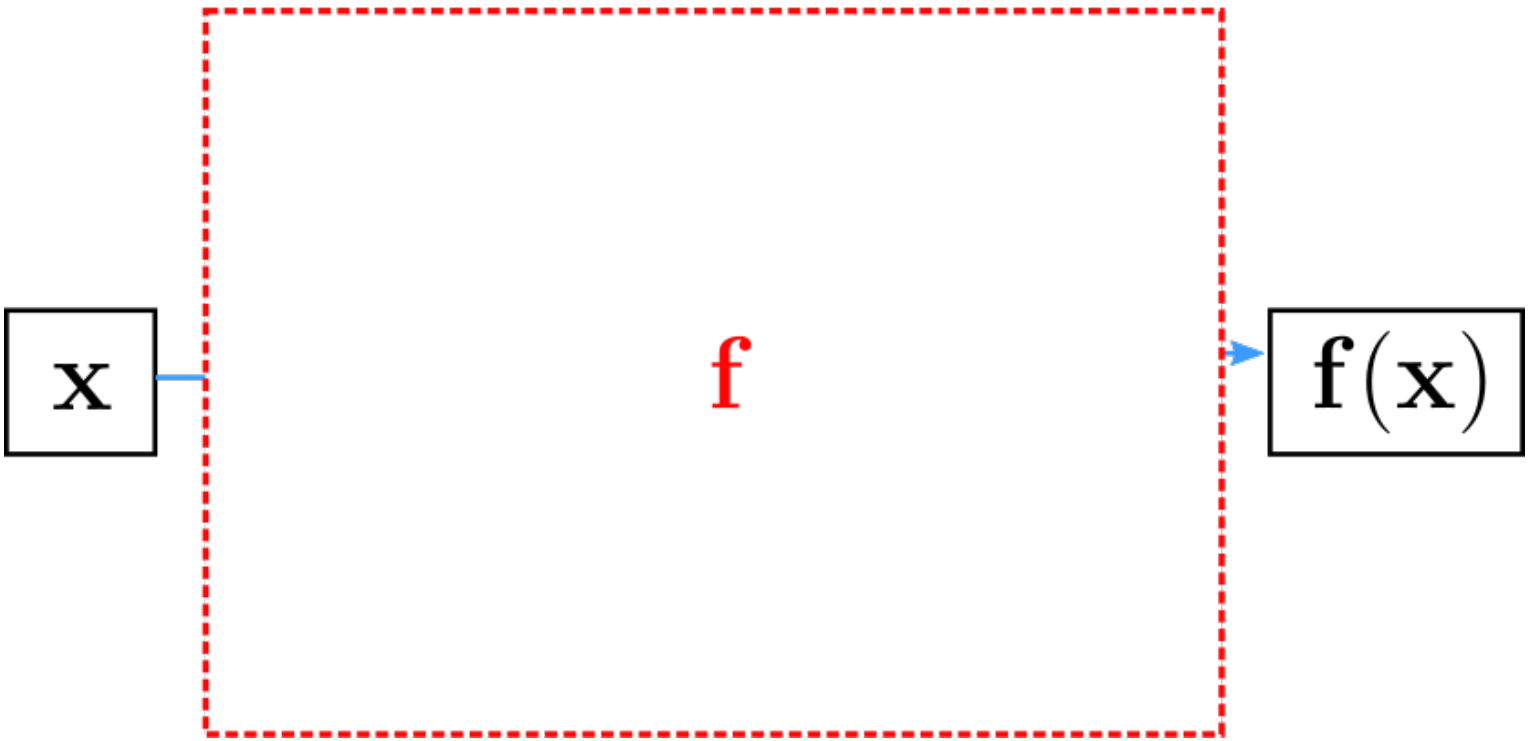
Computing power (GPUs, TPUs, ...)

More data with labels

Open source tools and models

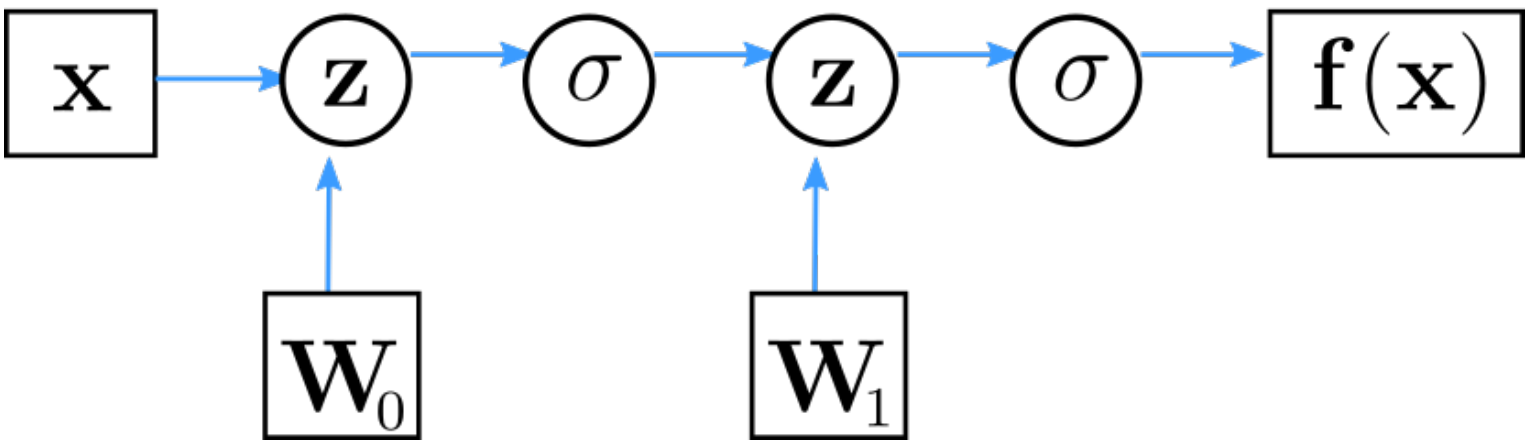
Università
Ca'Foscari
Venezia

Computation Graph



Neural network = parametrized, non-linear function

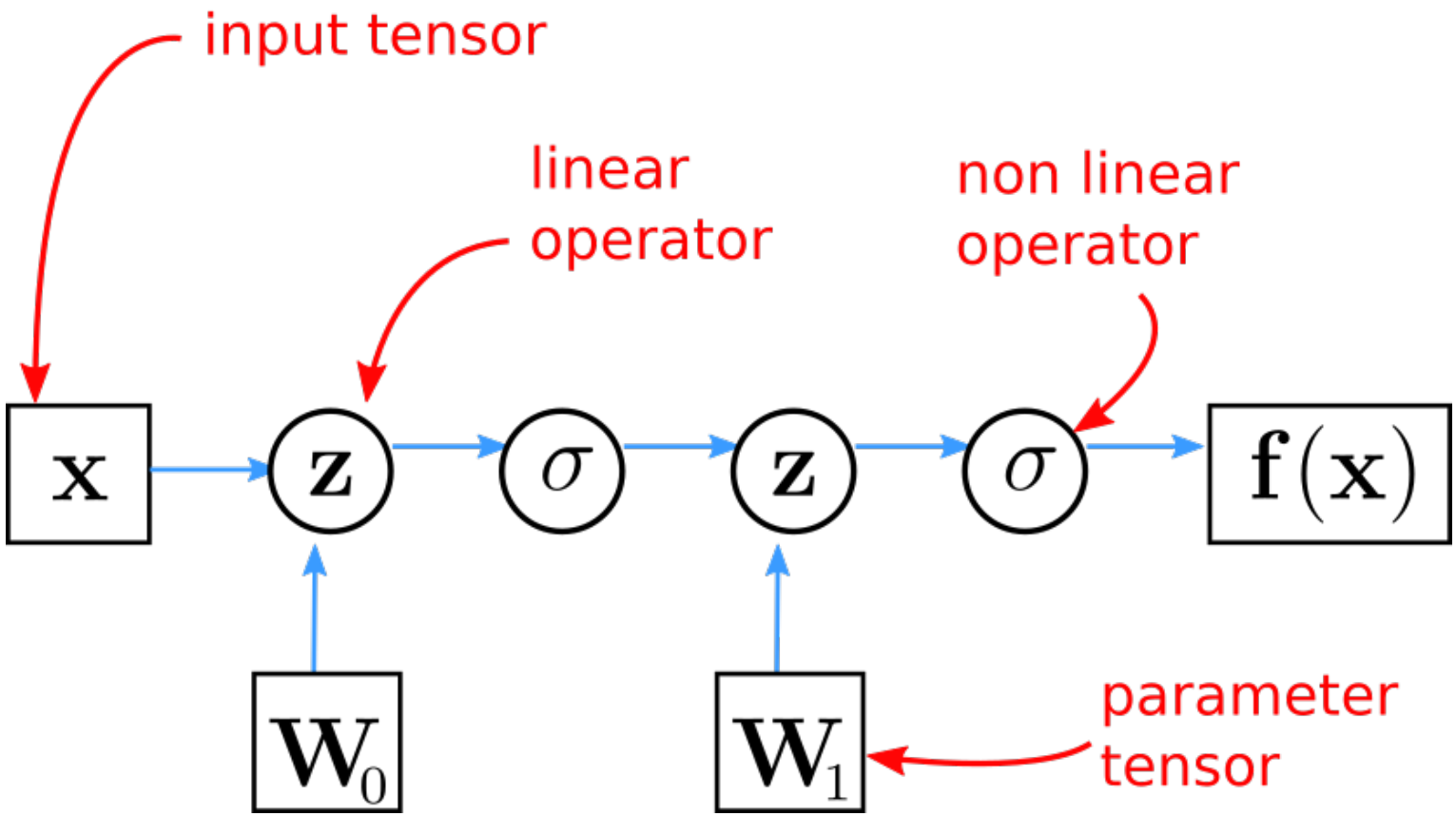
Computation Graph



Computation graph: Directed graph of functions, depending on parameters (neuron weights)

Università
Ca'Foscari
Venezia

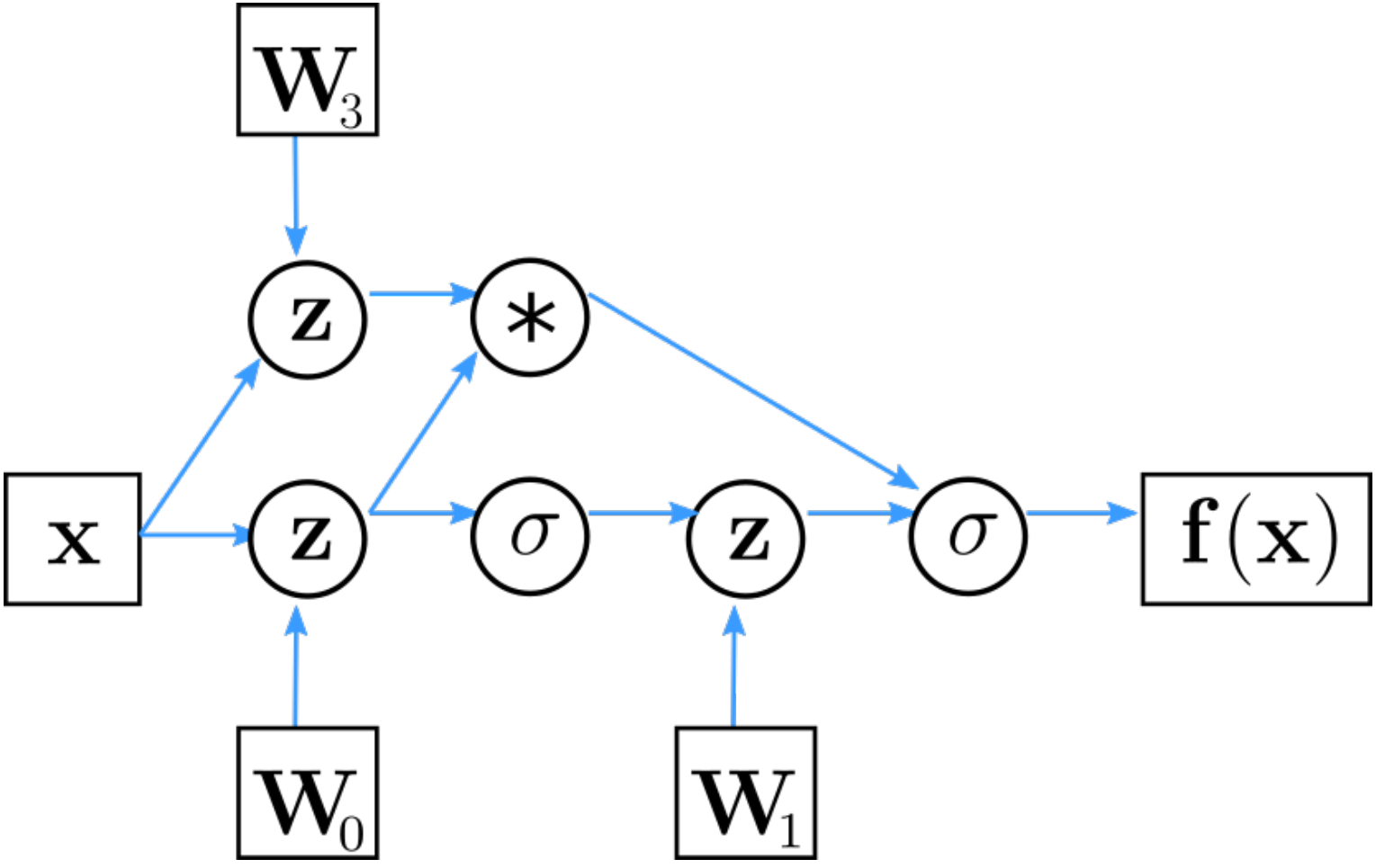
Computation Graph



Combination of linear (parametrized) and non-linear functions

Università
Ca'Foscari
Venezia

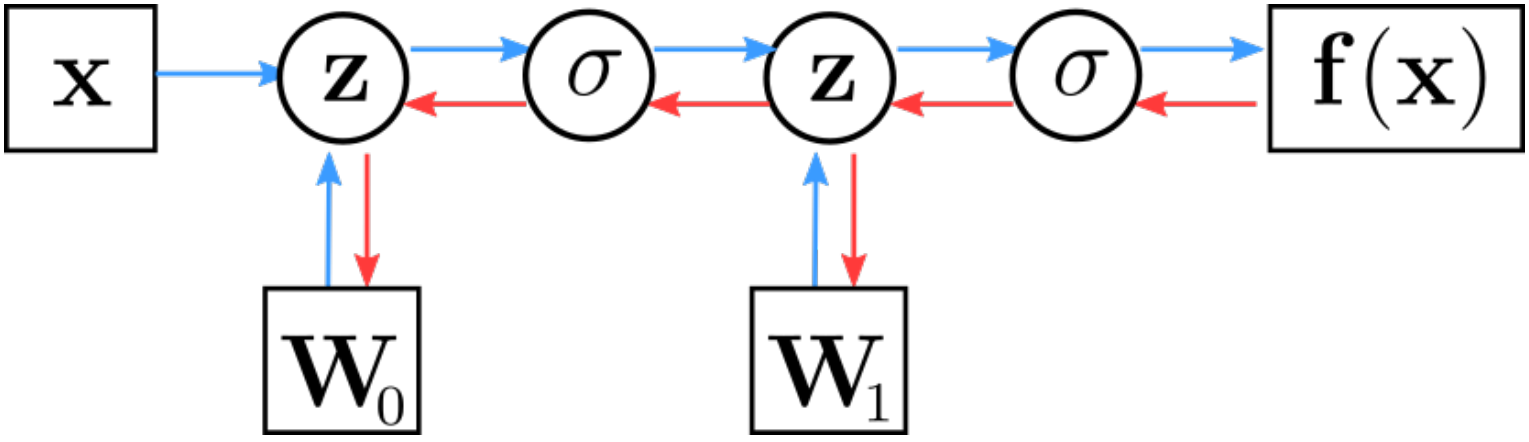
Computation Graph



Not only sequential application of functions

Università
Ca'Foscari
Venezia

Computation Graph



Automatic computation of gradients: all modules are differentiable!

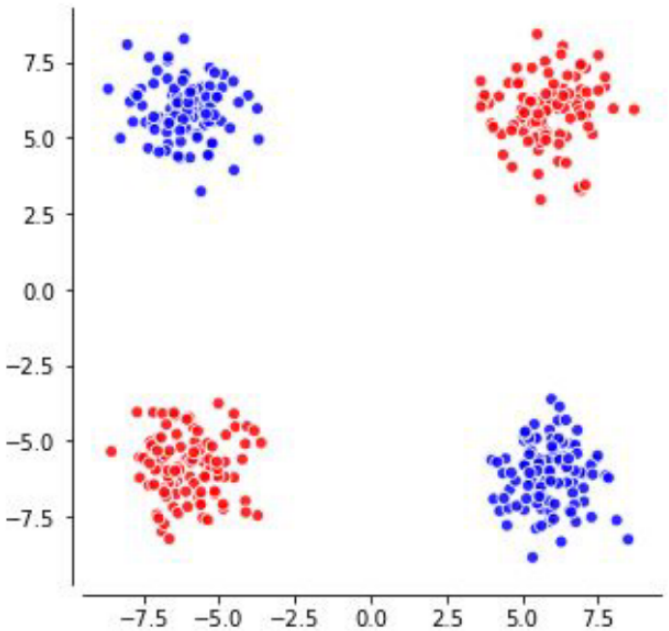
Università
Ca'Foscari
Venezia

Why Deep? (Limitations of the Perceptron)

The Perceptron was developed as a model of the natural Neurons

Can only classify linearly separable data

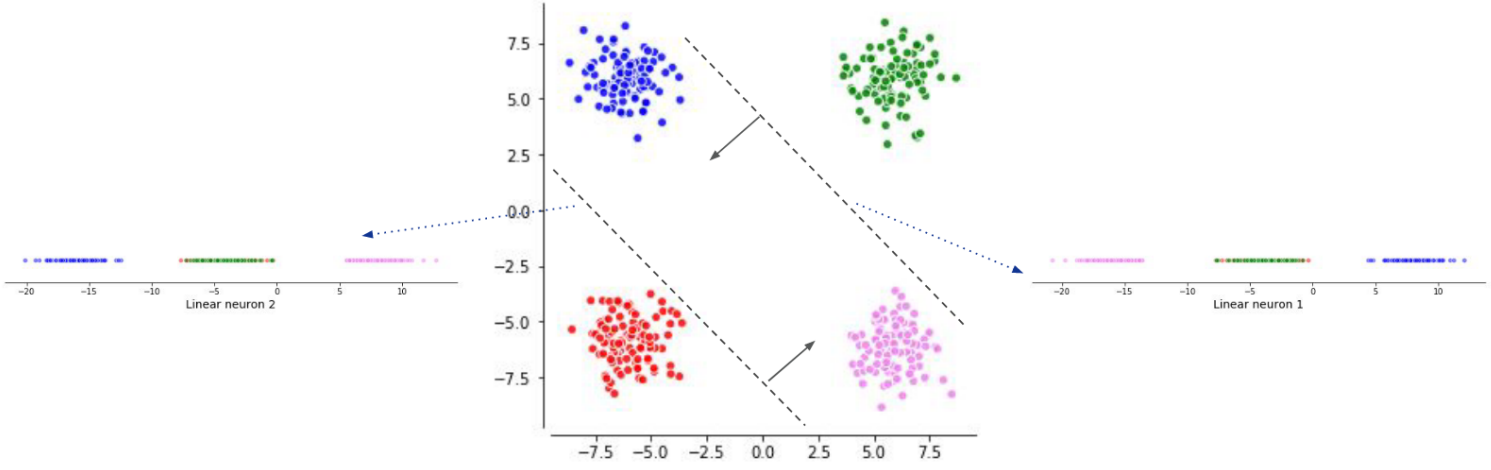
- altough we could use the kernel trick to "bend" the boundary a bit



Cannot learn the "xor" function

Università
Ca'Foscari
Venezia

2 Layer Network

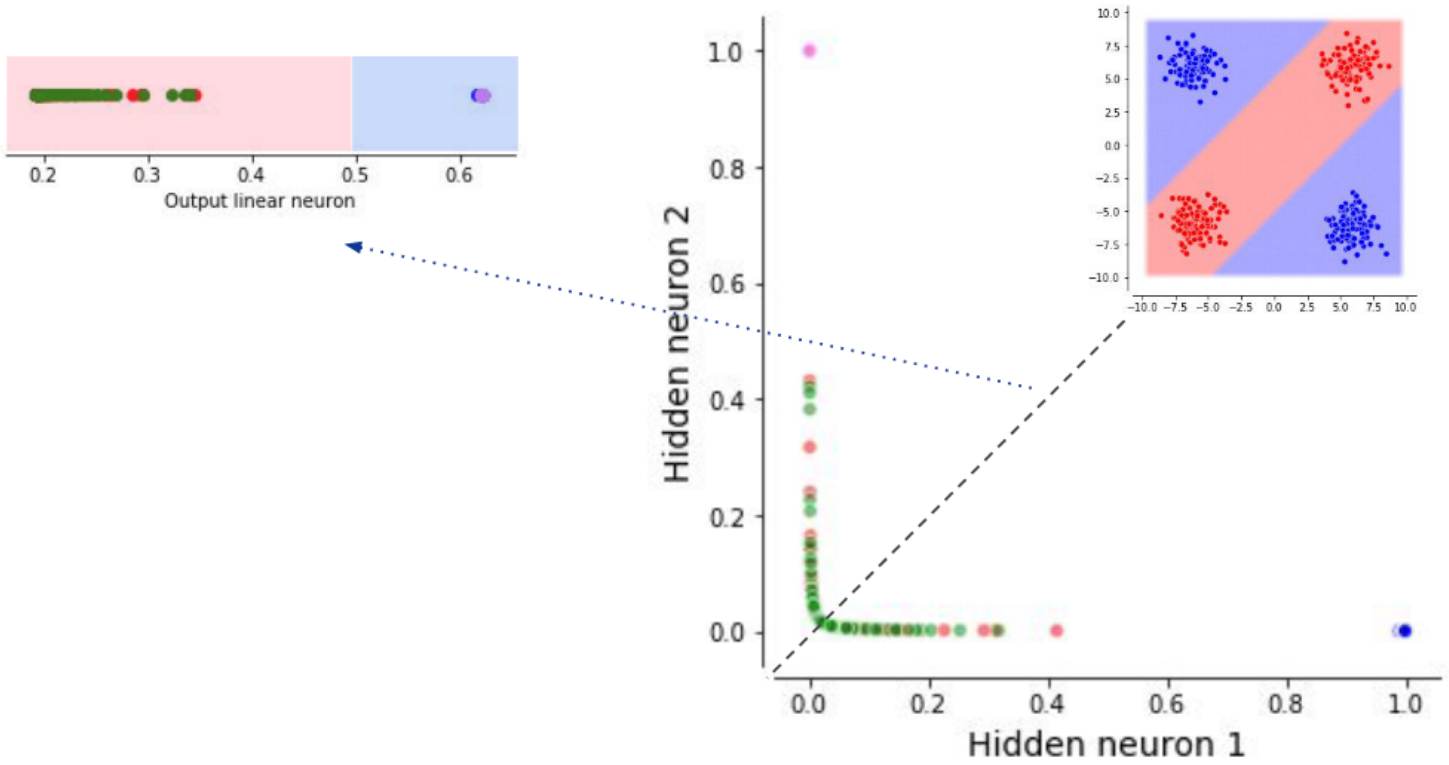


$$\mathbf{W} = \begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix} \quad \mathbf{b} = \begin{bmatrix} -4 \\ -4 \end{bmatrix}$$

Different perceptrons can provide differtn (linear) views of the data

Università
Ca'Foscari
Venezia

2 Layer Network



$$\mathbf{W} = \begin{bmatrix} -1 & 1 \\ 1 & -1 \end{bmatrix} \quad \mathbf{b} = \begin{bmatrix} -4 \\ -4 \end{bmatrix}$$

Combine them (non-linearly) to obtain a new **separable** representation

Universal Function Approximation

Theorem

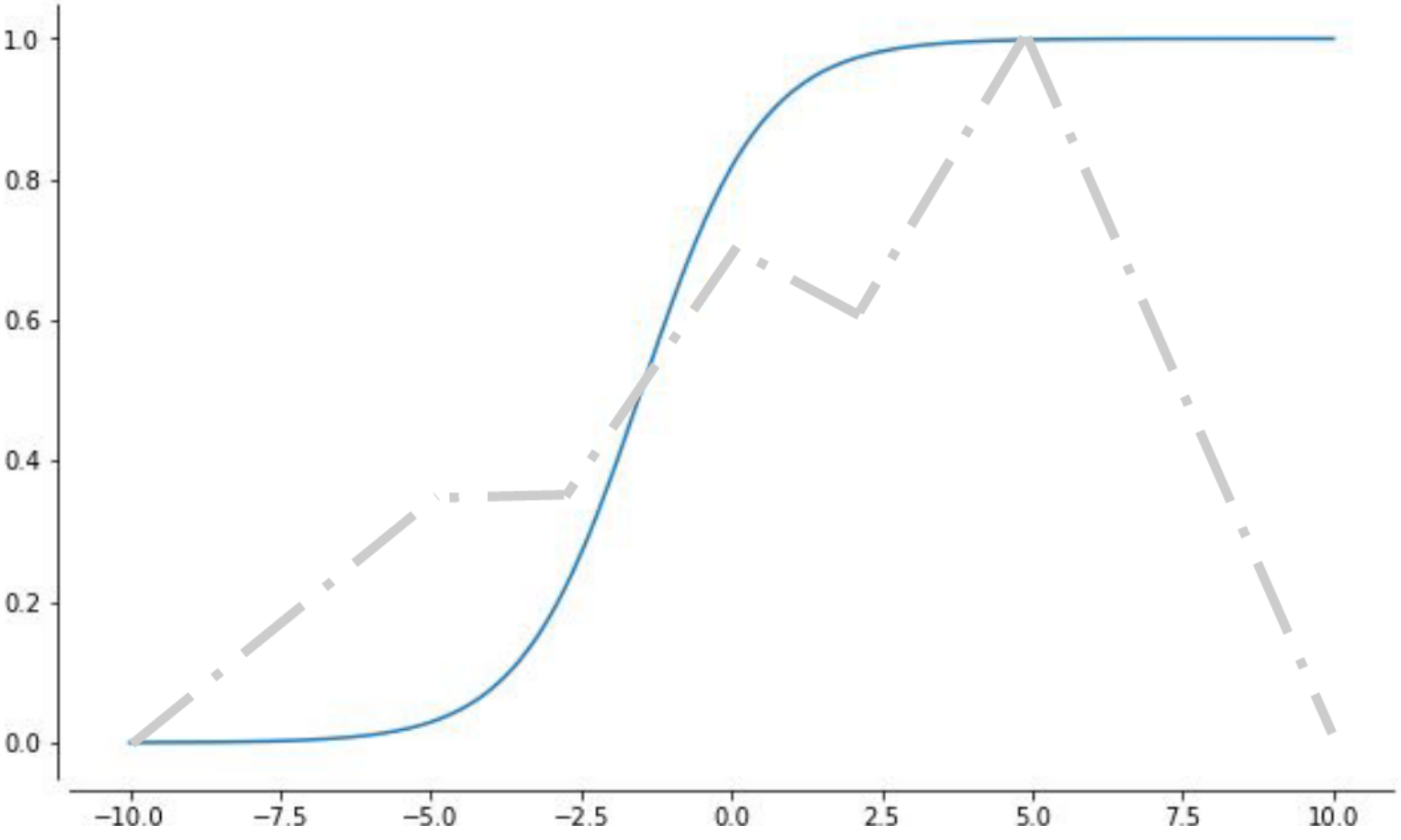
Let σ be a nonconstant, bounded, and monotonically-increasing continuous function. For any $f \in C([0,1]^d)$ and $\epsilon > 0$, there exists $h \in \mathbb{N}$ real constants $v_i, b_i \in \mathbb{R}$ and real vectors $w_i \in \mathbb{R}^d$ such that:

$$\left| \sum_{i=1}^{h} v_i \sigma(w_i^T x + b_i) - f(x) \right| < \epsilon$$

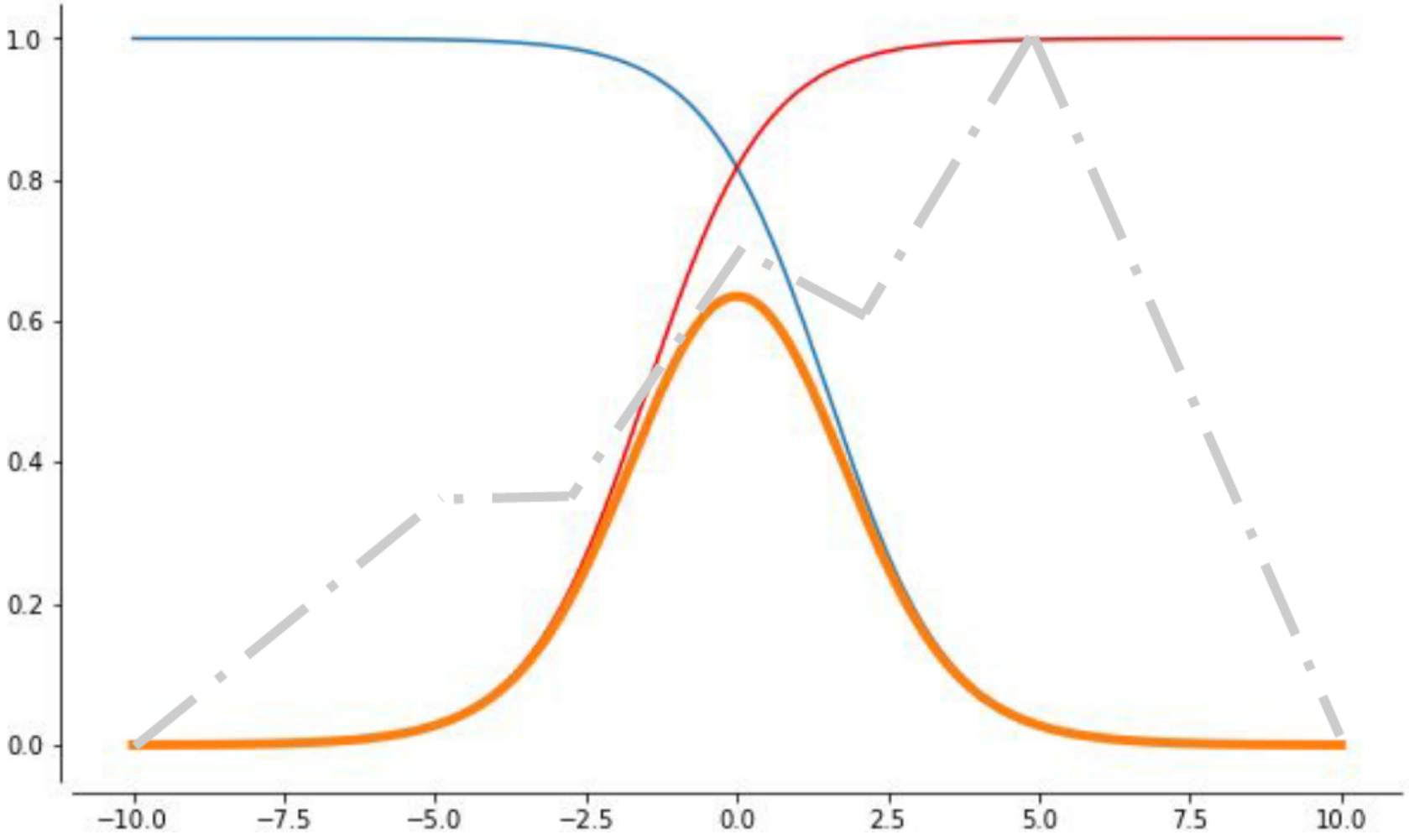
i.e., neural nets are dense in $C([0,1]^d)$

This still holds for any compact subset of $\mathbb{R}^d$ and if σ is the ReLU function

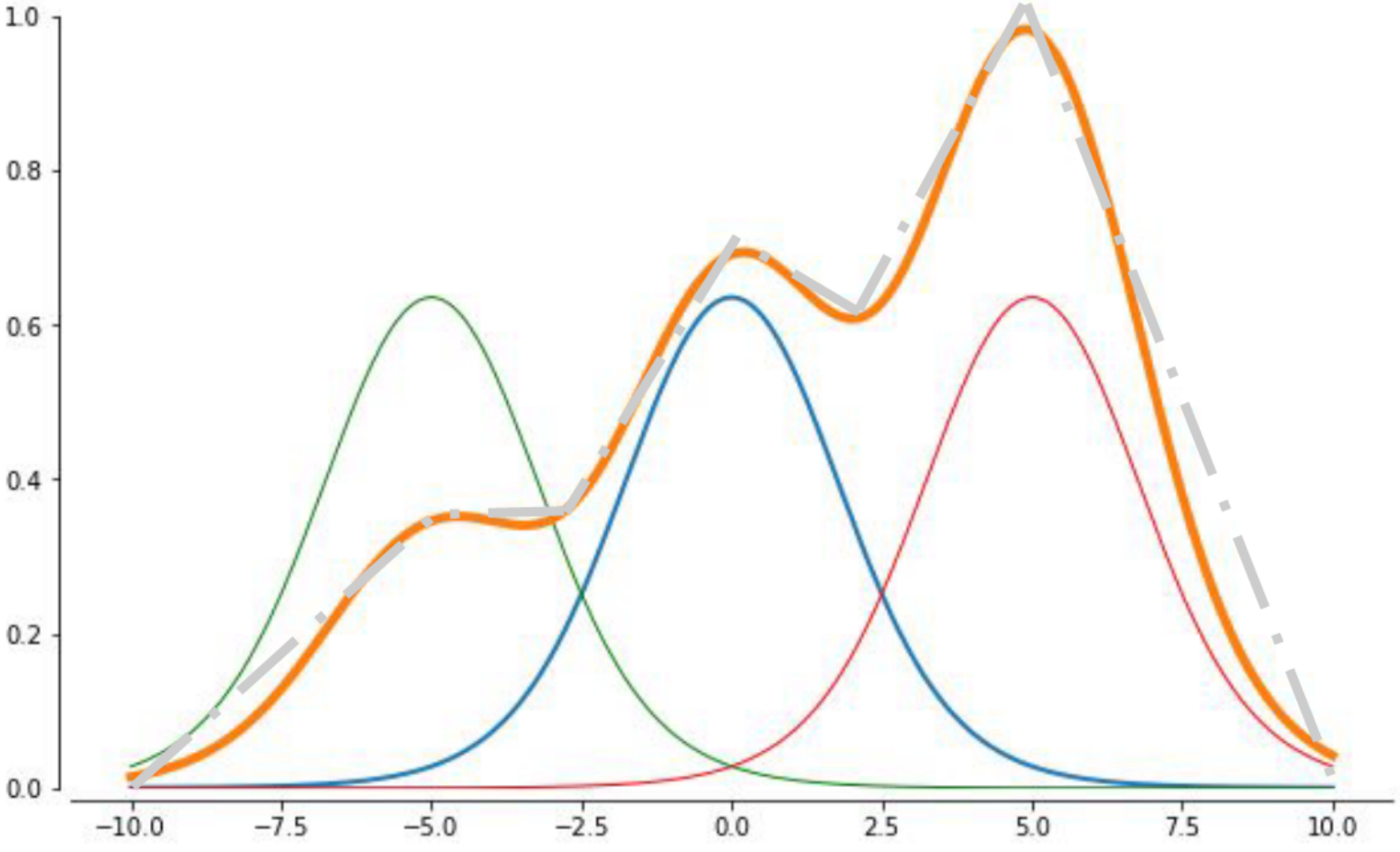
Intuition



Intuition



Intuition



What UFA Does NOT Tell Us

The size of h

- The number h of hidden units might be too big to fit network in RAM.

The optimal function parameters

- the optimization landscape can have several local minima

Neural Network for Classification

Vector function with tunable parameters θ

$$f(\cdot;\theta) : \mathbb{R}^N \to (0,1)^K$$

Sample s in dataset S:

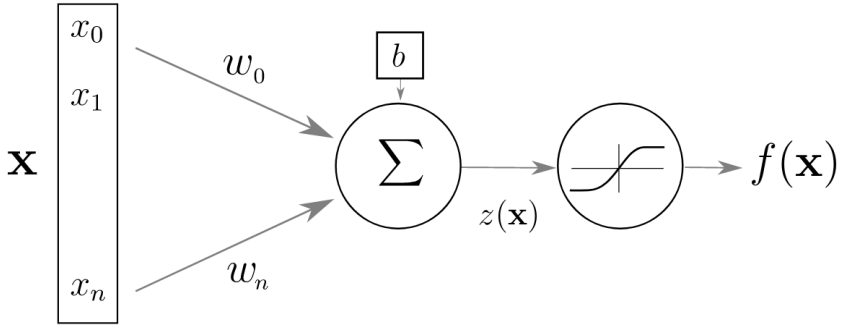
- input: $x^s \in \mathbb{R}^N$
- expected output: $y^s \in [0, K-1]$

Output is a conditional probability distribution:

$$f(x^s;\theta)_c = P(Y = c|X = x^s)$$

Università
Ca'Foscari
Venezia

Artificial Neuron



$$z(x) = w^T x + b$$
$$f(x) = g(w^T x + b)$$

$x, f(x)$ input and output

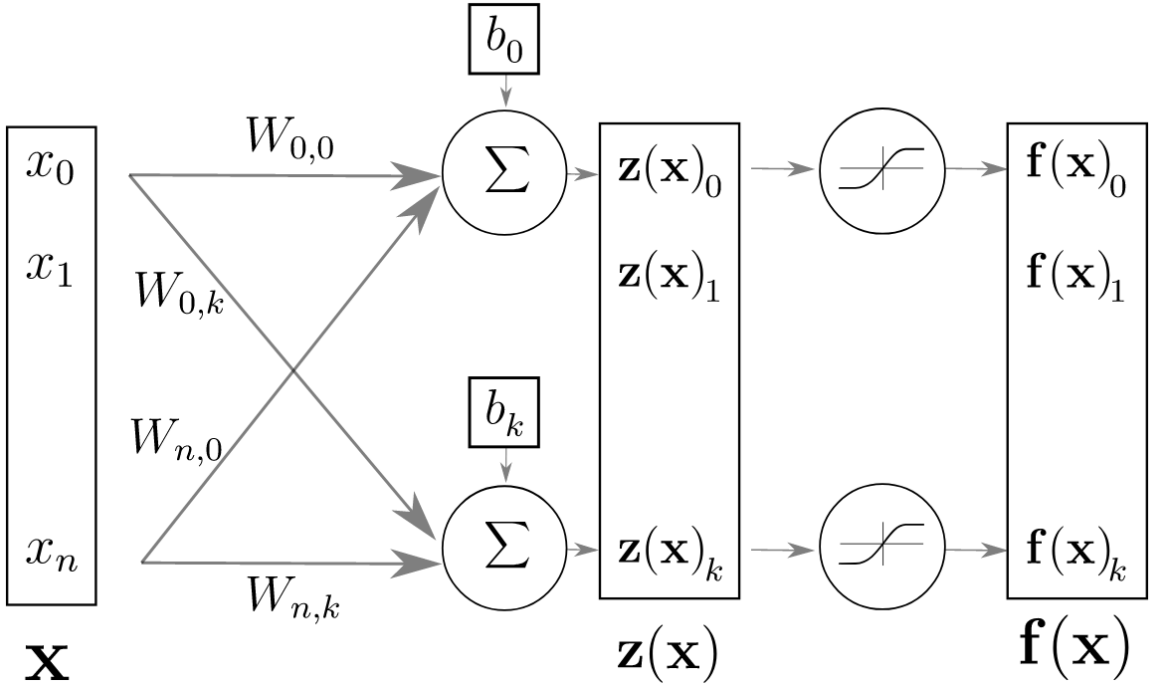
$z(x)$ pre-activation

w, b weights and bias

g activation function

Università
Ca'Foscari
Venezia

Layer of Neurons

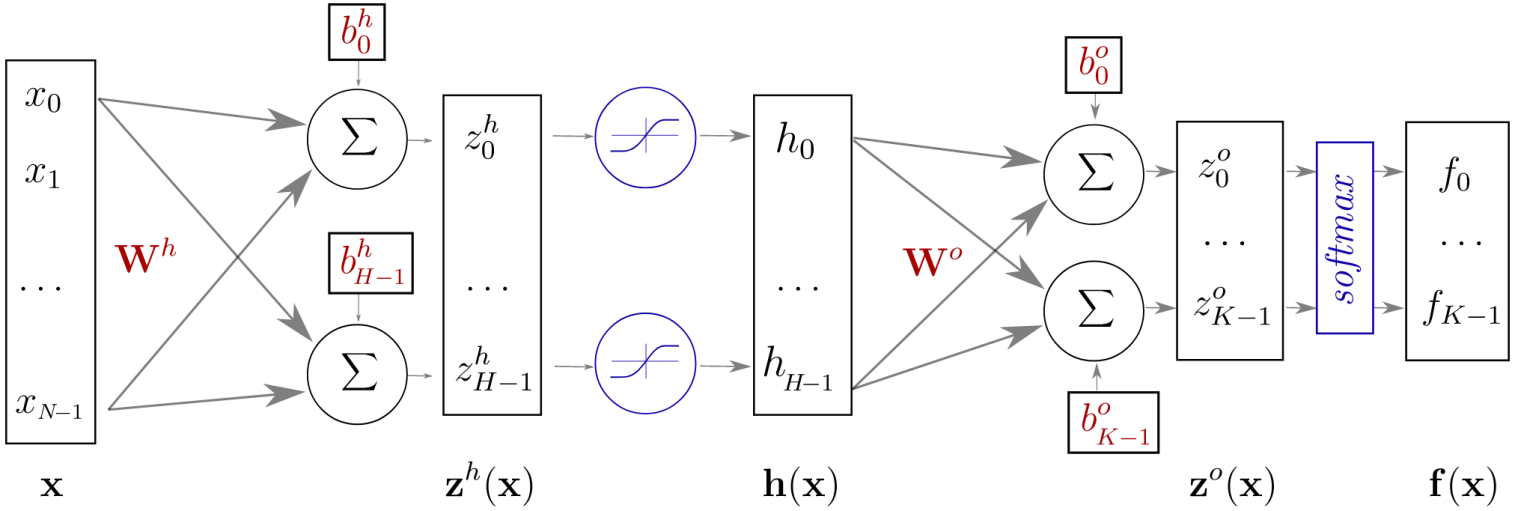


$$f(x) = g(z(x)) = g(Wx + b)$$

W,b now matrix and vector

Università
Ca'Foscari
Venezia

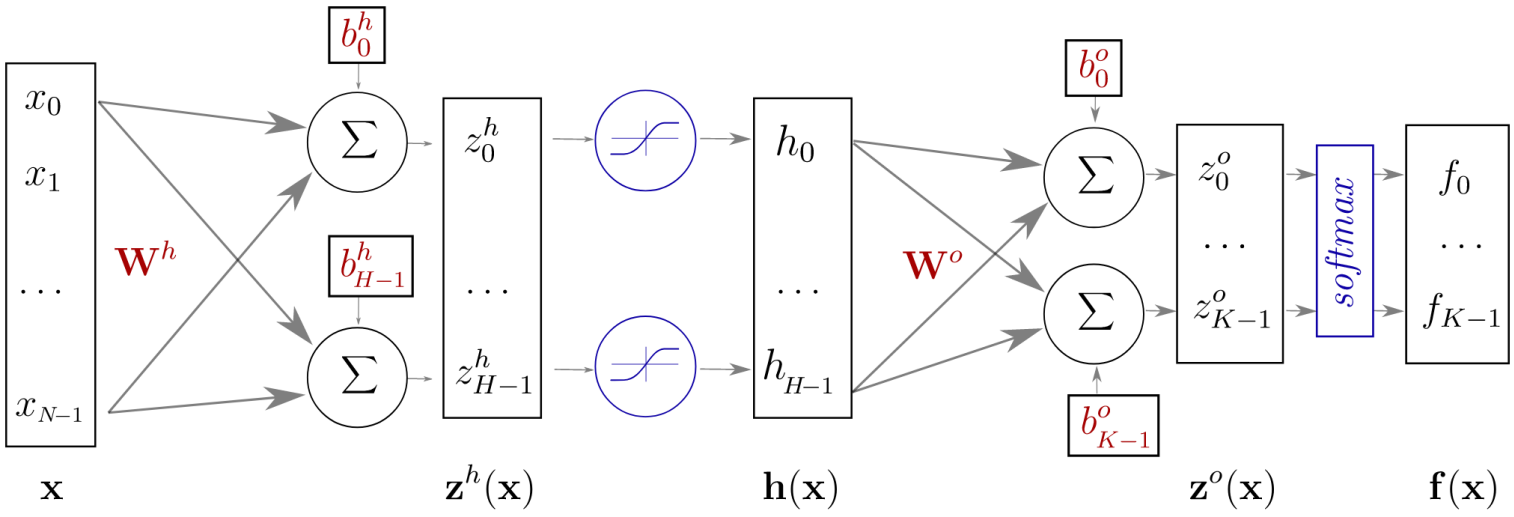
One Hidden Layer Network



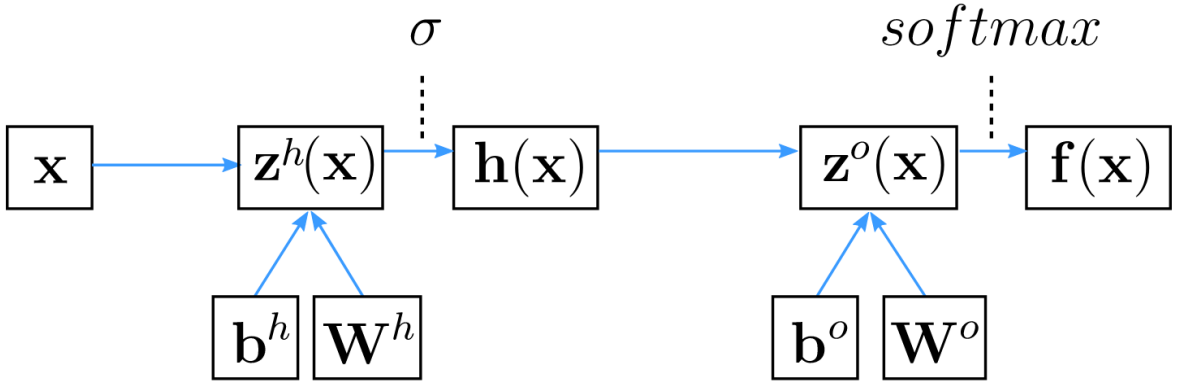
$$
\begin{aligned}
z^h(x) &= W^h x + b^h \\
h(x) &= g(z^h(x)) = g(W^h x + b^h) \\
z^o(x) &= W^o h(x) + b^o \\
f(x) &= \text{softmax}(zo) = \text{softmax}(W^o h(x) + b^o)
\end{aligned}
$$

Università
Ca'Foscari
Venezia

One Hidden Layer Network

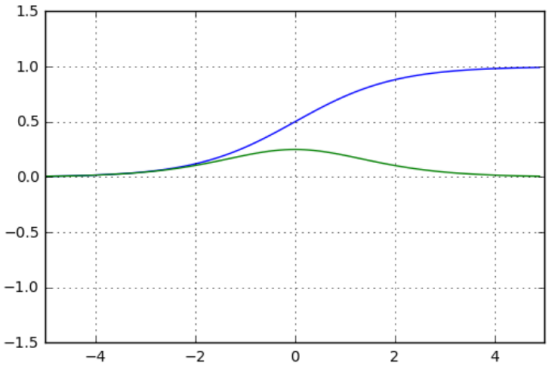


Alternate representation



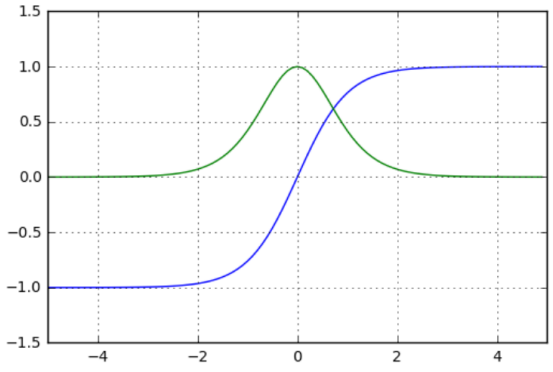
Università
Ca'Foscari
Venezia

Activation Functions



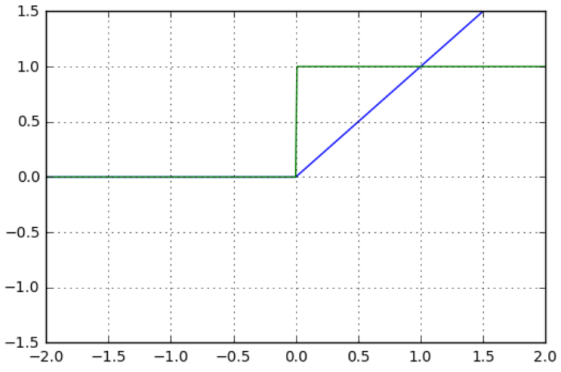
$$\text{sigm}(x) = \frac{1}{1+e^{-x}}$$

$$\text{sigm}'(x) = \text{sigm}(x)(1-\text{sigm}(x))$$



$$\tanh(x) = \frac{e^{2x}-1}{e^{2x}+1}$$

$$\tanh'(x) = 1-\tanh(x)^2$$



$$\text{relu}(x) = \max(0,x)$$

$$\text{relu}'(x) = 1_{x>0}$$

Sofmax

$$softmax(\mathbf{x}) = \frac{1}{\sum_{i=1}^{n} e^{x_i}} \cdot \begin{bmatrix} e^{x_1} \\ e^{x_2} \\ \vdots \\ e^{x_n} \end{bmatrix}$$

$$\frac{\partial softmax(\mathbf{x})_i}{\partial x_j} = \begin{cases} softmax(\mathbf{x})_i \cdot (1 - softmax(\mathbf{x})_i) & i = j \\ -softmax(\mathbf{x})_i \cdot softmax(\mathbf{x})_j & i \neq j \end{cases}$$

Vector of values in $(0, 1)$ that add up to 1

$P(Y = c|X = x) = \text{softmax}(z(x))_c$

The pre-activation vector $z(x)$ is often called the **logits**

Training the network

Find parameters $\theta = (W^h; b^h; W^o; b^o)$ that minimize the **negative log likelihood** (or **cross entropy**)

The loss function for a given sample $s \in S$:

$$l(f(x^s; \theta), y^s) = -\log f(x^s; \theta)_{y^s}$$

The cost function is the negative likelihood of the model computed on the full training set (for i.i.d. samples):

$$L_S(\theta) = -\frac{1}{|S|} \sum_{s \in S} \log f(x^s; \theta)_{y^s} + \lambda \Omega(\theta)$$

$\lambda \Omega(\theta) = \lambda(\|W^h\|^2 + \|W^o\|^2)$ is an optional **regularization term**

Stochastic Gradient Descent

Initialize θ randomly

For E epochs perform:

- Randomly select a small batch of samples ($B \subset S$)
- Compute gradients: $\Delta = \nabla_\theta L_B(\theta)$
- Update parameters: $\theta \leftarrow \theta - \eta\Delta$
  $\eta > 0$ is called the **learning rate**

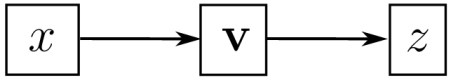
Repeat until the epoch is completed (all of S is covered)

Stop when reaching criterion: cross entropy stops decreasing when computed on validation set

Computing Gradients

The network is a composition of differentiable modules

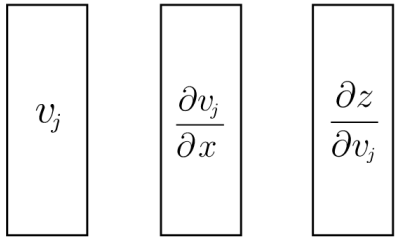
- We can apply the **chain rule**



$$z = u(\mathbf{v}(x))$$

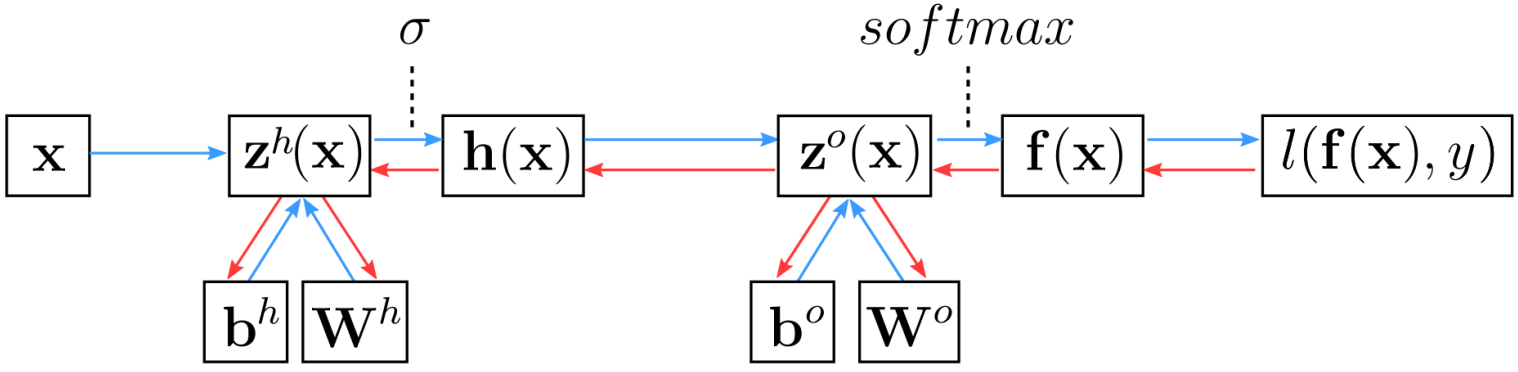
chain-rule

$$\frac{\partial z}{\partial x} = \sum_j \frac{\partial z}{\partial v_j}\frac{\partial v_j}{\partial x} = \nabla u . \frac{\partial \mathbf{v}}{\partial x}$$



Università
Ca'Foscari
Venezia

Backpropagation



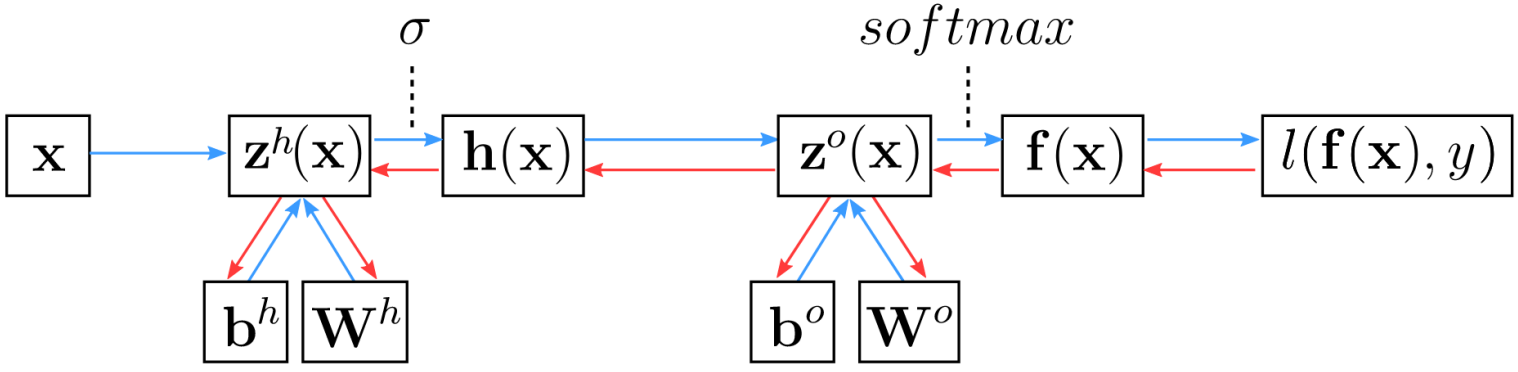
Compute partial derivatives of the loss

$$\frac{\partial l}{\partial f(x)_i} = \frac{\partial l(f(x), y)}{\partial f(x)_i} = \frac{\partial(-\log f(x)_y)}{\partial f(x)_i} = \frac{-1_{y=i}}{f(x)_i}$$

$$\frac{\partial l}{\partial z^o(x)_i} = ?$$

Università
Ca'Foscari
Venezia

Backpropagation



Compute partial derivatives of the loss

$$\frac{\partial l}{\partial f(x)_i} = \frac{\partial l(f(x), y)}{\partial f(x)_i} = \frac{\partial(-\log f(x)_y)}{\partial f(x)_i} = \frac{-1_{y=i}}{f(x)_i}$$

$$\frac{\partial l}{\partial z^o(x)_i} = \sum_j \frac{\partial l}{\partial f(x)_j} \frac{\partial f(x)_j}{\partial z^o(x)_i}$$

Chain rule!

Backpropagation

$$
\begin{aligned}
\frac{\partial l}{\partial z^o(x)_i} &= \sum_j \frac{\partial l}{\partial f(x)_j} \frac{\partial f(x)_j}{\partial z^o(x)_i} \\
&= \sum_j \frac{-1_{y=j}}{f(x)_j} \frac{\partial \operatorname{softmax}(z^o(x))_j}{\partial z^o(x)_i} \\
&= -\frac{1}{f(x)_y} \frac{\partial \operatorname{softmax}(z^o(x))_y}{\partial z^o(x)_i} \\
&= \begin{cases} -1 + f(x)_y & \text{if } y = i \\ f(x)_i & \text{if } y \neq i \end{cases}
\end{aligned}
$$

Hence:

$$\nabla_{z^o(x)} l(f(x), y) = f(x) - e(y)$$

$e(y)$ one-hot encoding of y

Backpropagation

$$\nabla_{b^o} l = f(x) - e(y)$$

because $z^o(x) = W^o h(x) + b^o$ and then $\frac{\partial z^o(x)_i}{\partial b^o_j} = 1_{i=j}$

Partial derivatives w.r.t W^o

$$\begin{aligned} \frac{\partial l}{\partial W^o_{ij}} &= \sum_k \frac{\partial l}{\partial z^o(x)_k} \frac{\partial z^o(x)_k}{\partial W^o_{ij}} \\ \nabla_{W^o} l &= \big(f(x) - e(y)\big) \cdot h(x)^T \end{aligned}$$

Backpropagation

1. Compute activation gradients

$$\nabla_{z^o(x)} l = f(x) - e(y)$$

2. Compute layer params gradients

$$\begin{aligned} \nabla_{W^o} l &= \big(f(x) - e(y)\big) \cdot h(x)^T \\ \nabla_{b^o} l &= f(x) - e(y) \end{aligned}$$

3. Compute prev layer activation gradients

$$\begin{aligned} \nabla_{h(x)} l &= W^{o\,T} \nabla_{z^o(x)} l \\ \nabla_{z^h(x)} l &= \nabla_{h(x)} l \odot \sigma'(z^h(x)) \end{aligned}$$

Momentum

Accumulate gradients across successive updates

Nesterov accelerated gradient

$$m_y = \gamma m_{t-1} + \eta \nabla_\theta L_{B_t}(\theta_{t-1} - \gamma m_{t-1}) \quad (1)$$

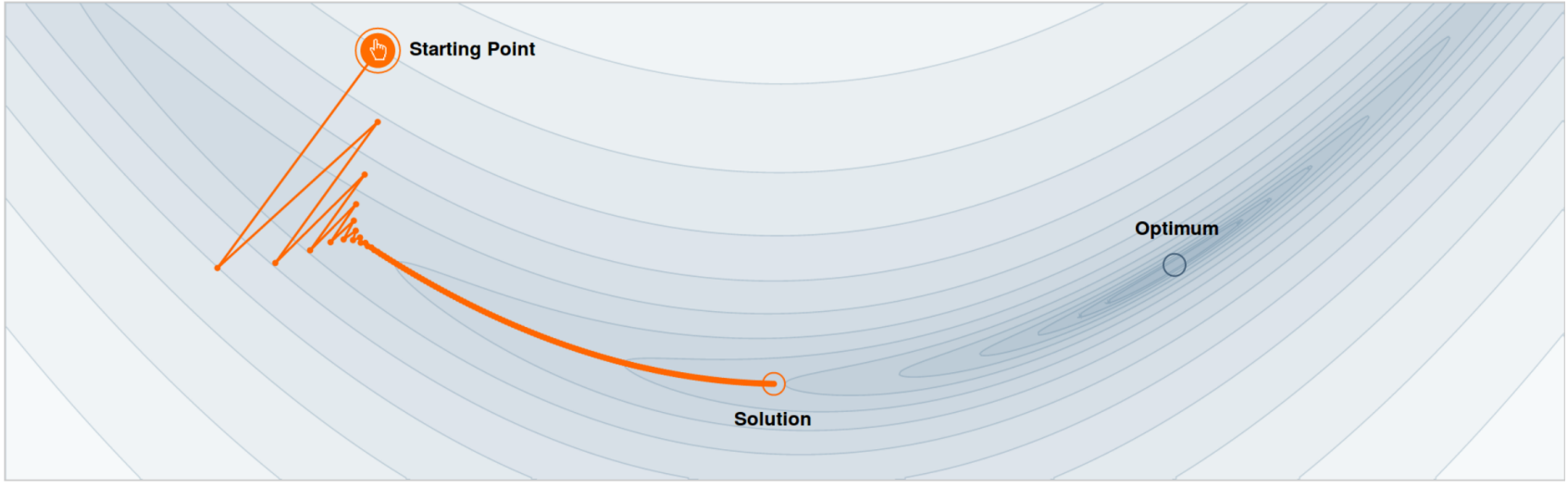
$$\theta_t = \theta_{t-1} - m_t \quad (2)$$

γ is typically set to 0.9

Larger updates in directions where the gradient sign is constant to accelerate in low curvature areas

Università
Ca'Foscari
Venezia

Why Momentum Works



Step-size α = 0.0030

0 0.003 0.006

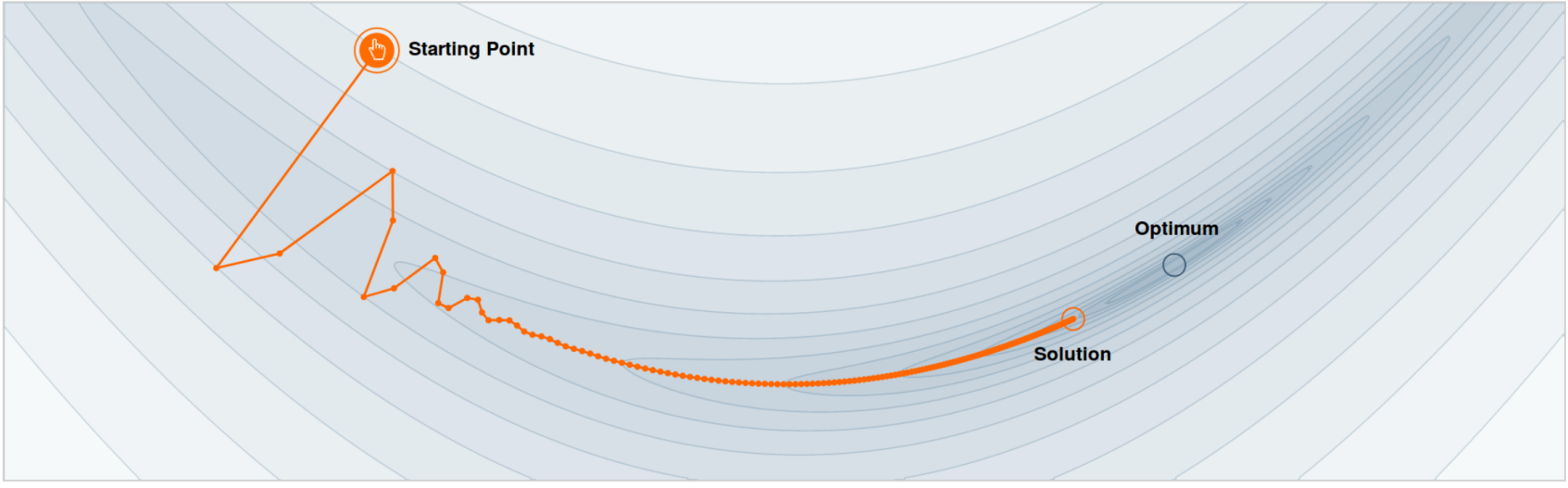
Momentum β = 0.0

0.00 0.500 0.990

We often think of Momentum as a means of dampening oscillations and speeding up the iterations, leading to faster convergence. But it has other interesting behavior. It allows a larger range of step-sizes to be used, and creates its own oscillations. What is going on?

Università
Ca'Foscari
Venezia

Why Momentum Works



Step-size α = 0.0030

0 0.003 0.006

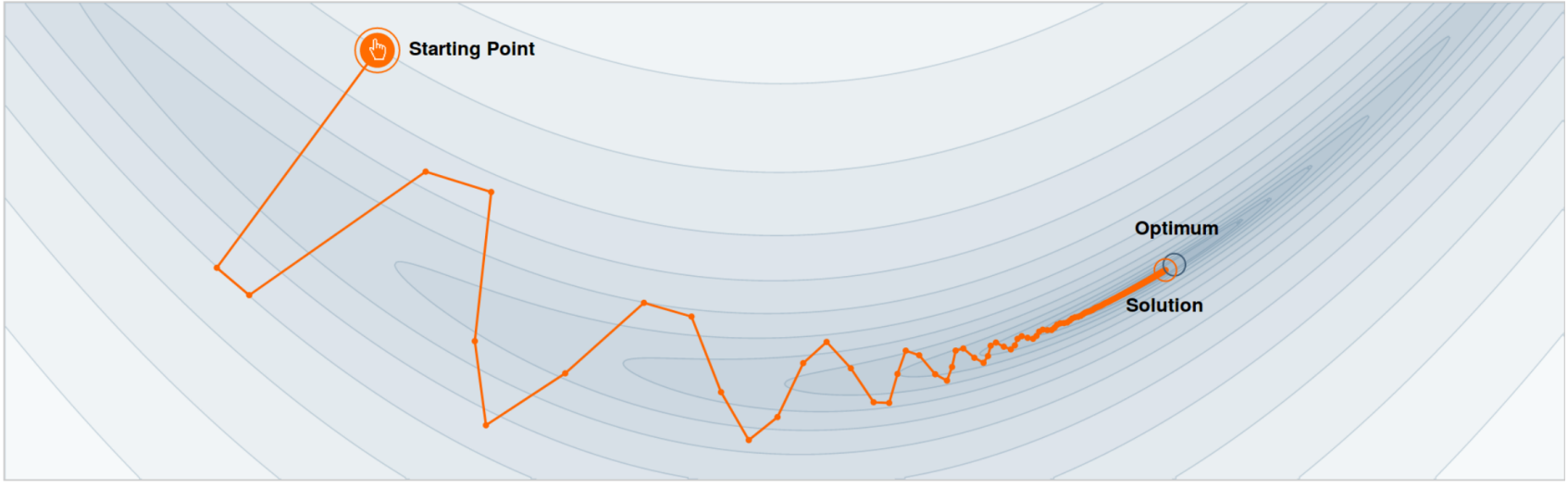
Momentum β = 0.60

0.00 0.500 0.990

We often think of Momentum as a means of dampening oscillations and speeding up the iterations, leading to faster convergence. But it has other interesting behavior. It allows a larger range of step-sizes to be used, and creates its own oscillations. What is going on?

Università
Ca'Foscari
Venezia

Why Momentum Works



Step-size α = 0.0030

0 0.003 0.006

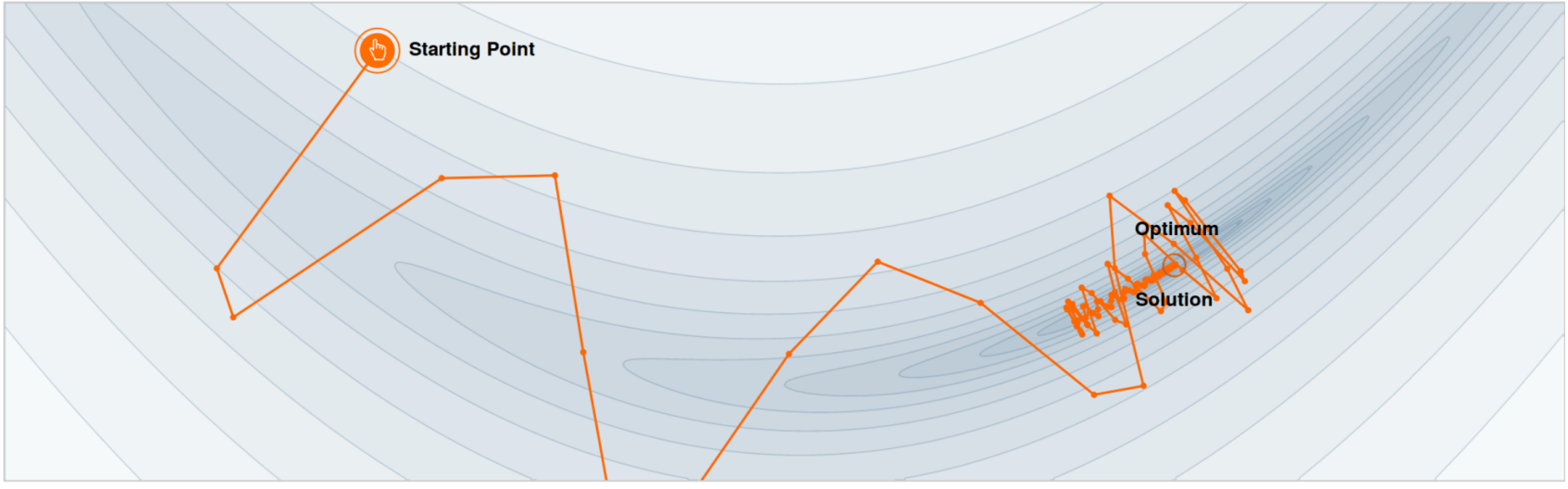
Momentum β = 0.80

0.00 0.500 0.990

We often think of Momentum as a means of dampening oscillations and speeding up the iterations, leading to faster convergence. But it has other interesting behavior. It allows a larger range of step-sizes to be used, and creates its own oscillations. What is going on?

Università
Ca'Foscari
Venezia

Why Momentum Works



Step-size α = 0.0030

0 0.003 0.006

Momentum β = 0.90

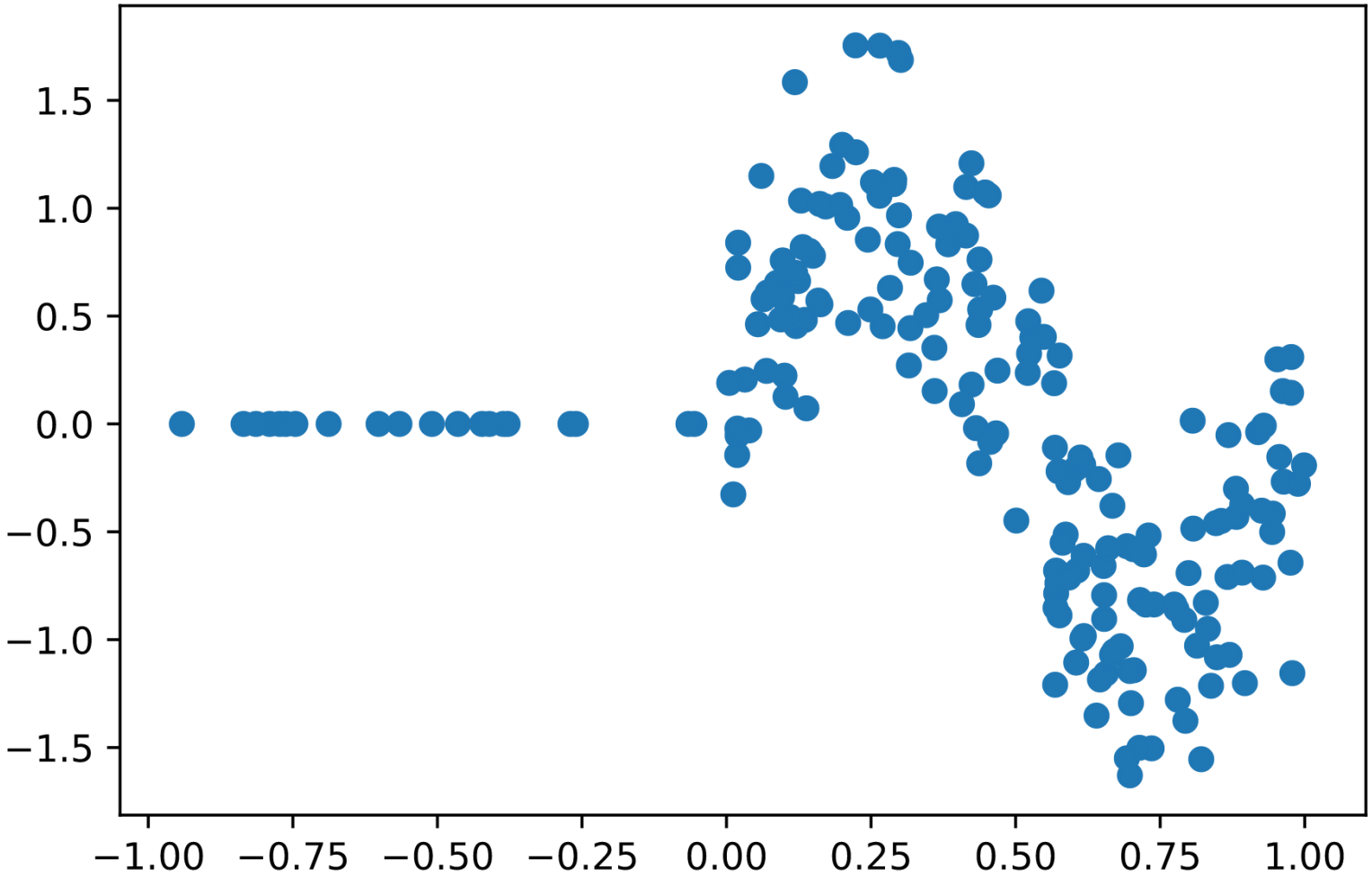
0.00 0.500 0.990

We often think of Momentum as a means of dampening oscillations and speeding up the iterations, leading to faster convergence. But it has other interesting behavior. It allows a larger range of step-sizes to be used, and creates its own oscillations. What is going on?

Università
Ca'Foscari
Venezia

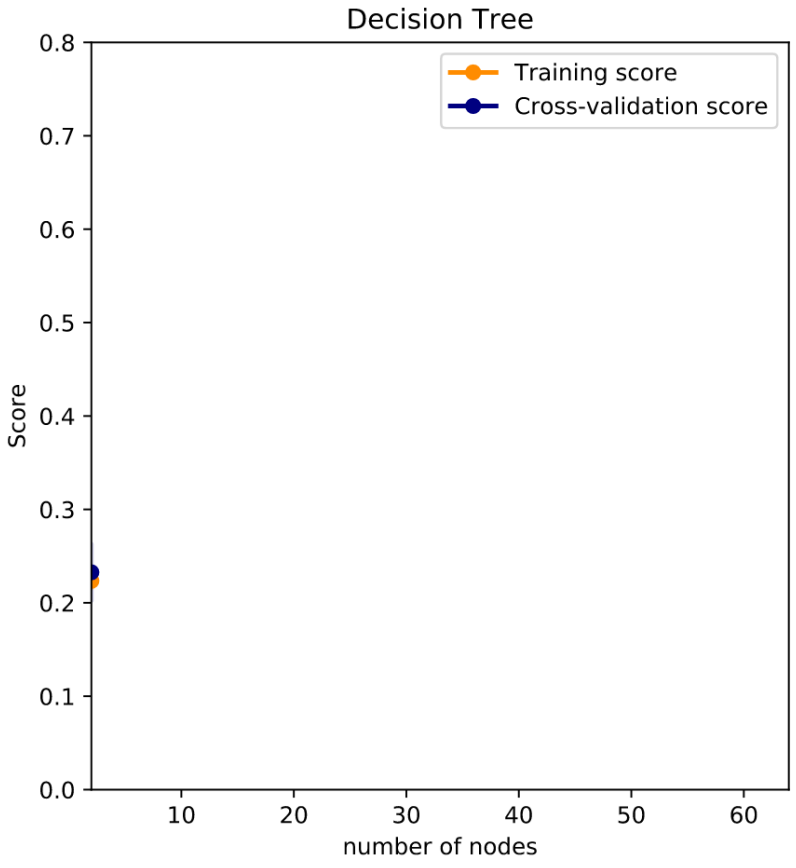
Generalization

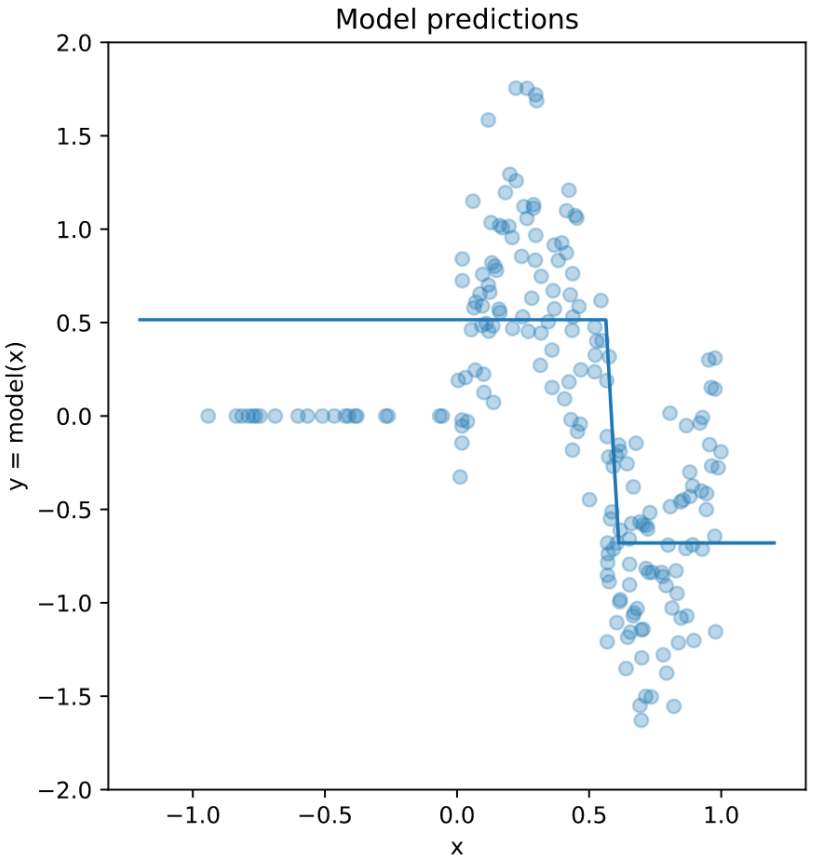
Experiment: regression on 1D data with heteroschedastic noise



Università
Ca'Foscari
Venezia

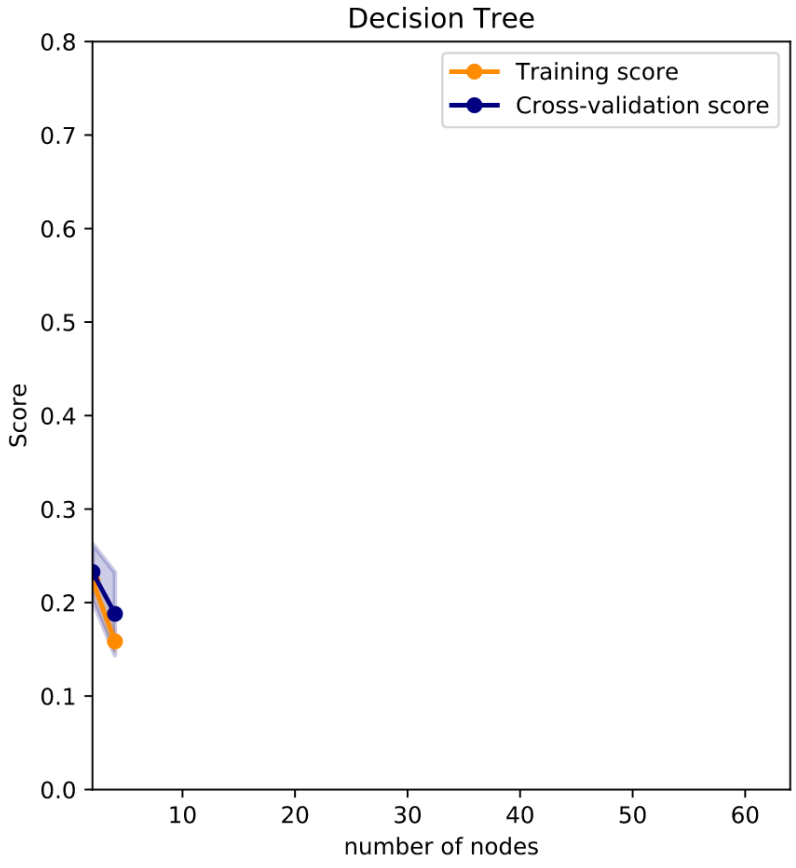
Single Decision Tree

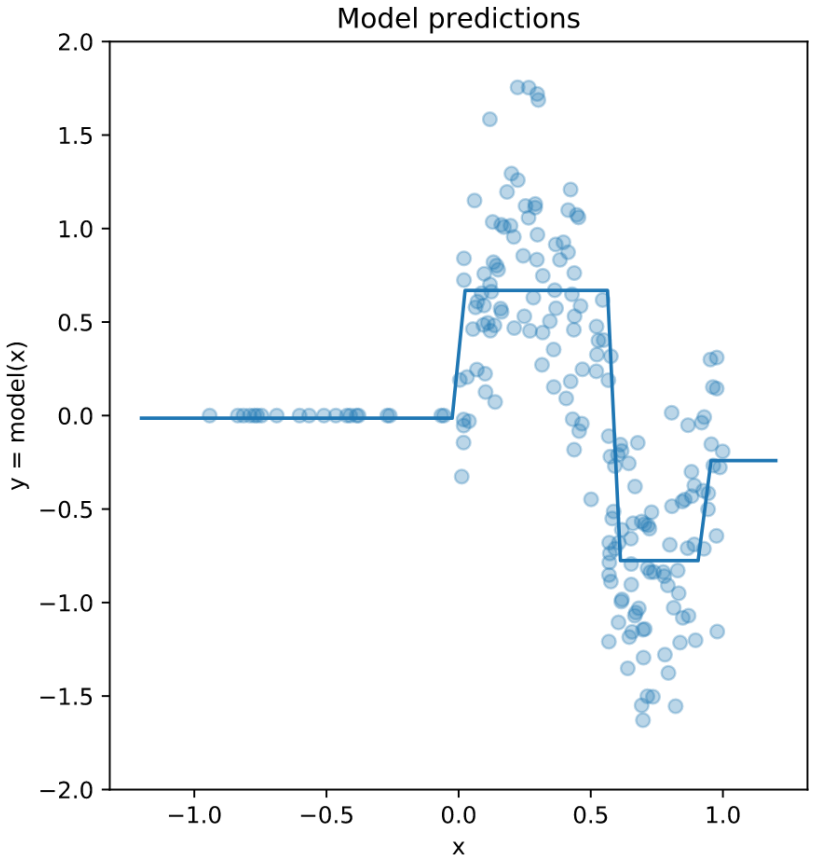




Università
Ca'Foscari
Venezia

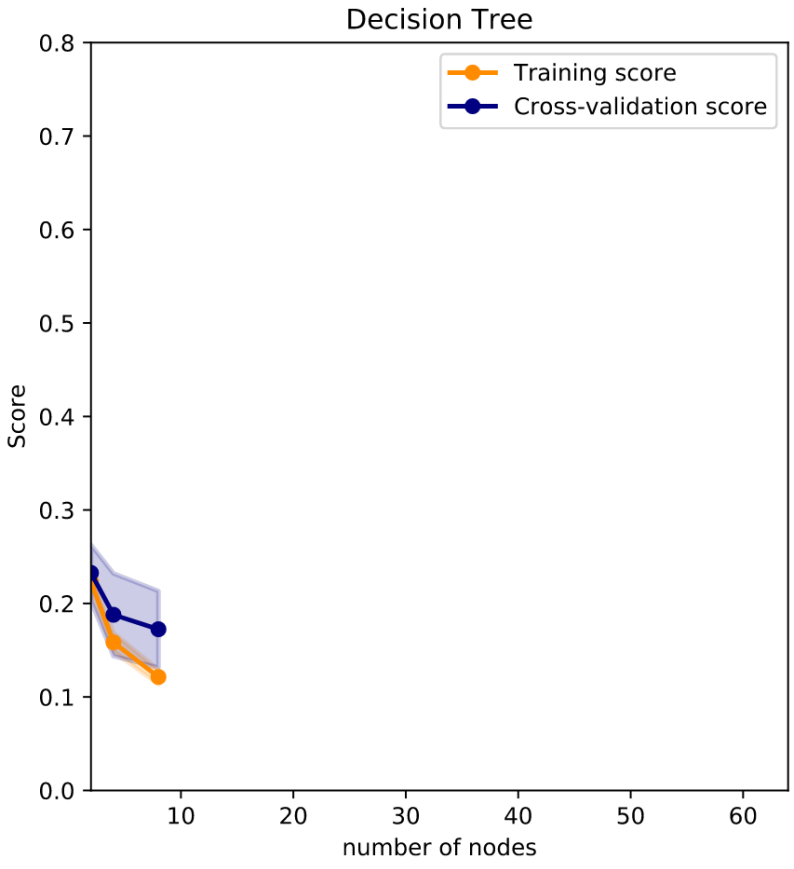
Single Decision Tree

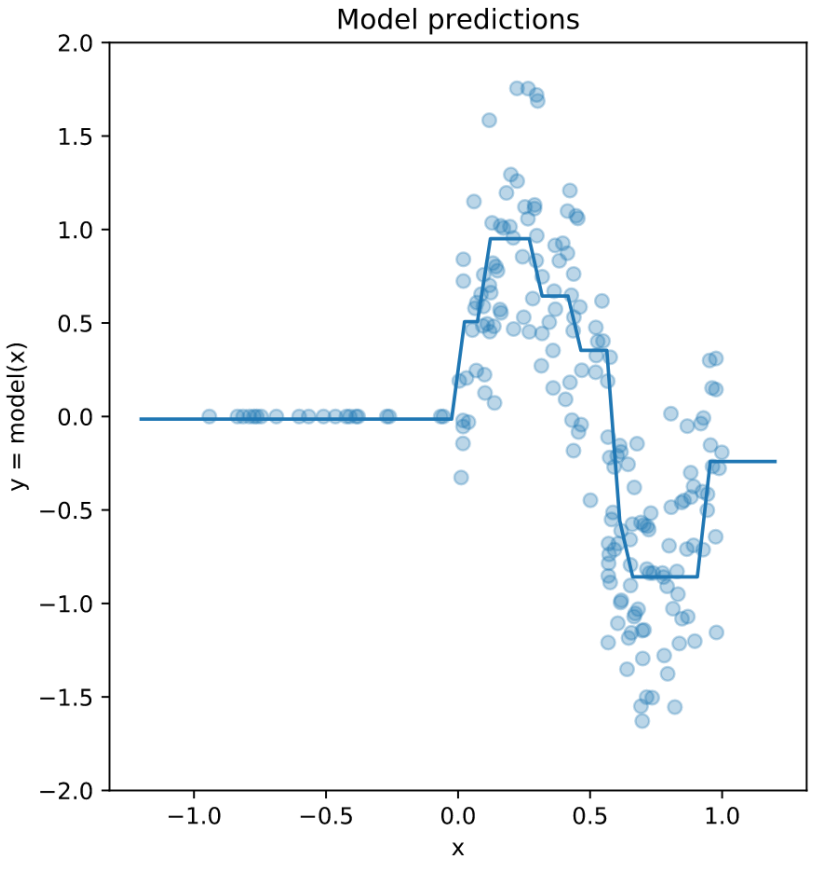




Università
Ca'Foscari
Venezia

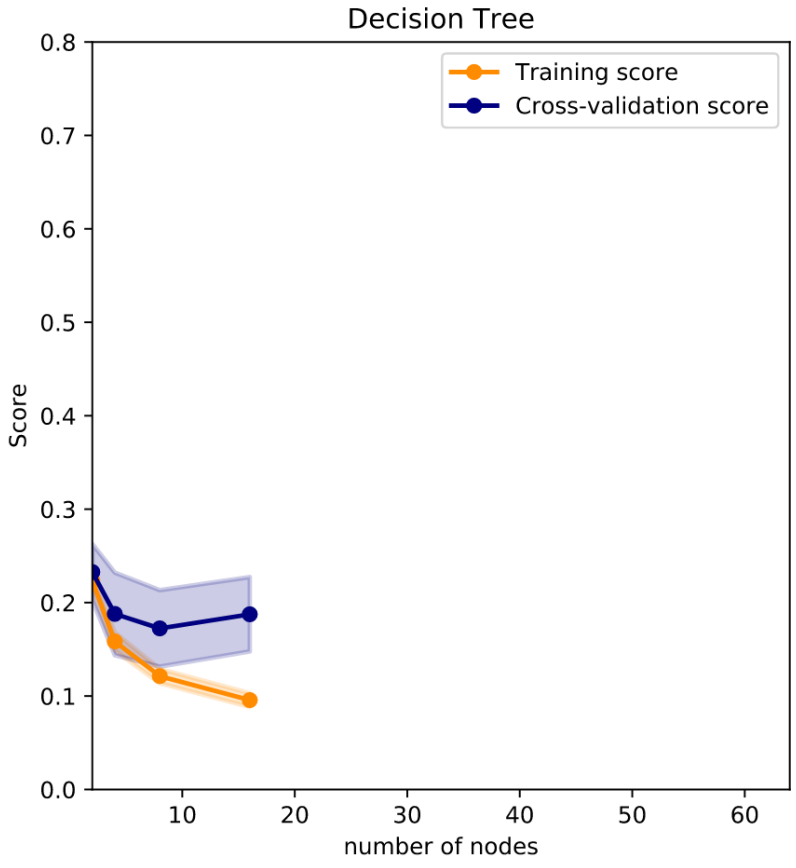
Single Decision Tree

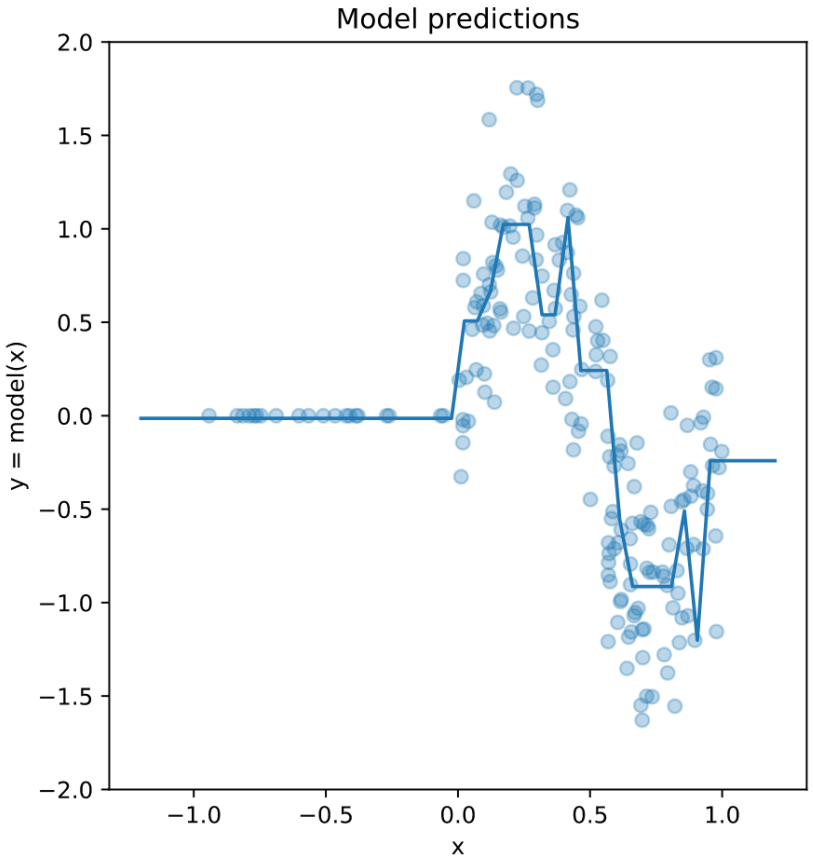




Università
Ca'Foscari
Venezia

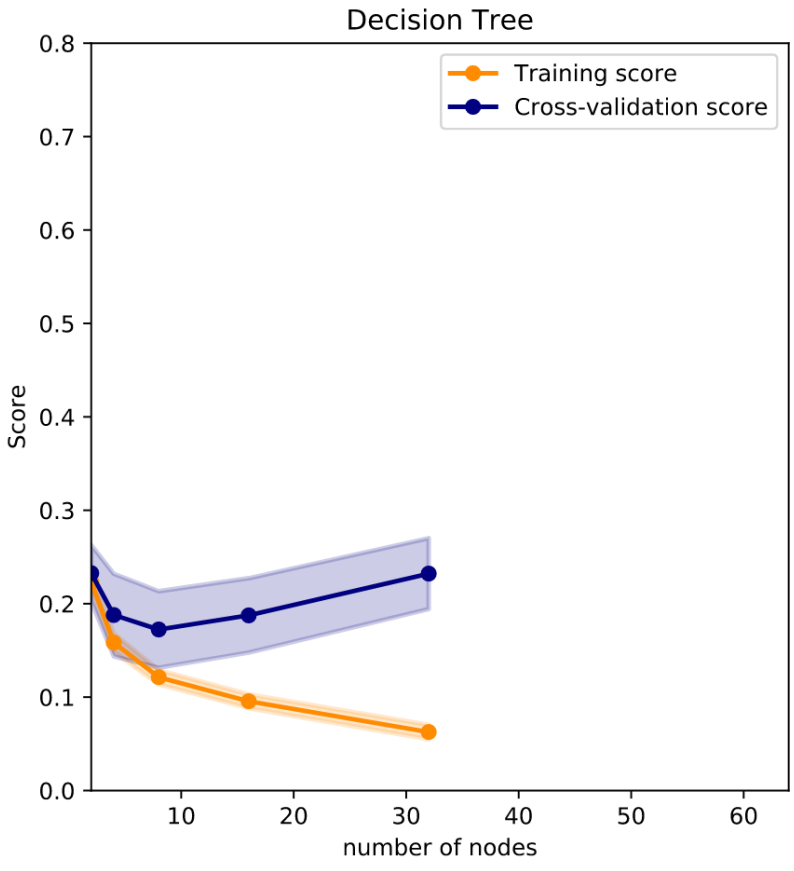
Single Decision Tree

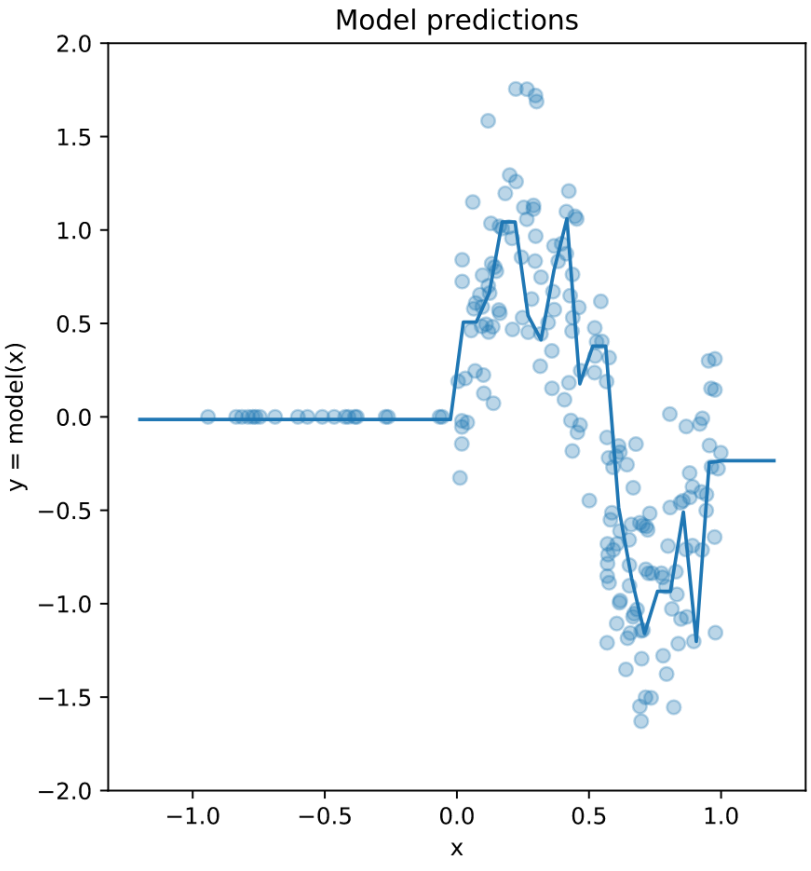




Università
Ca'Foscari
Venezia

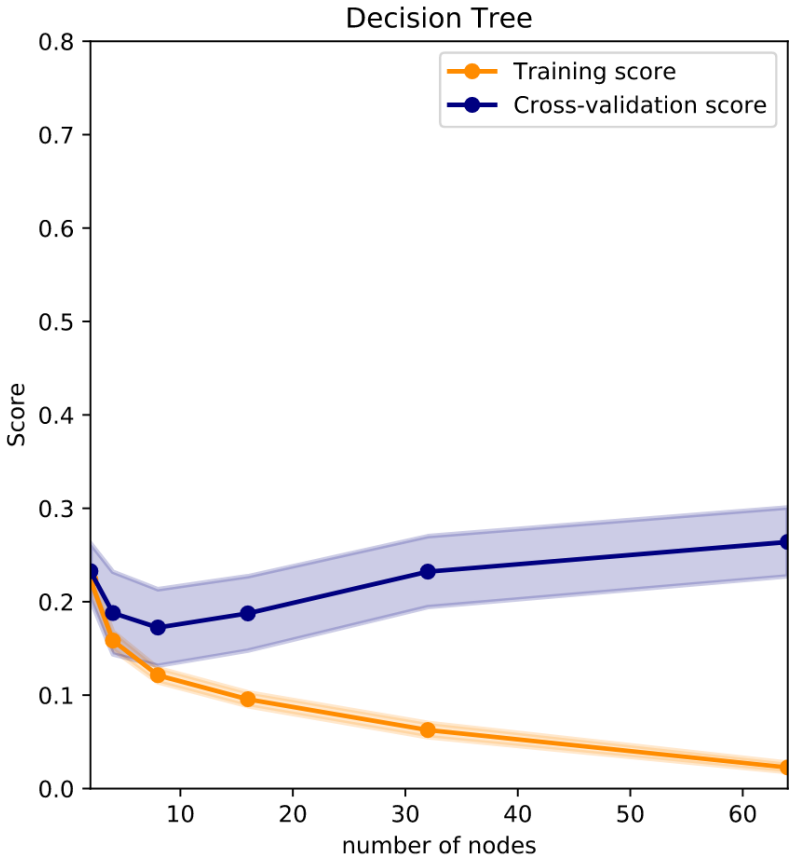
Single Decision Tree

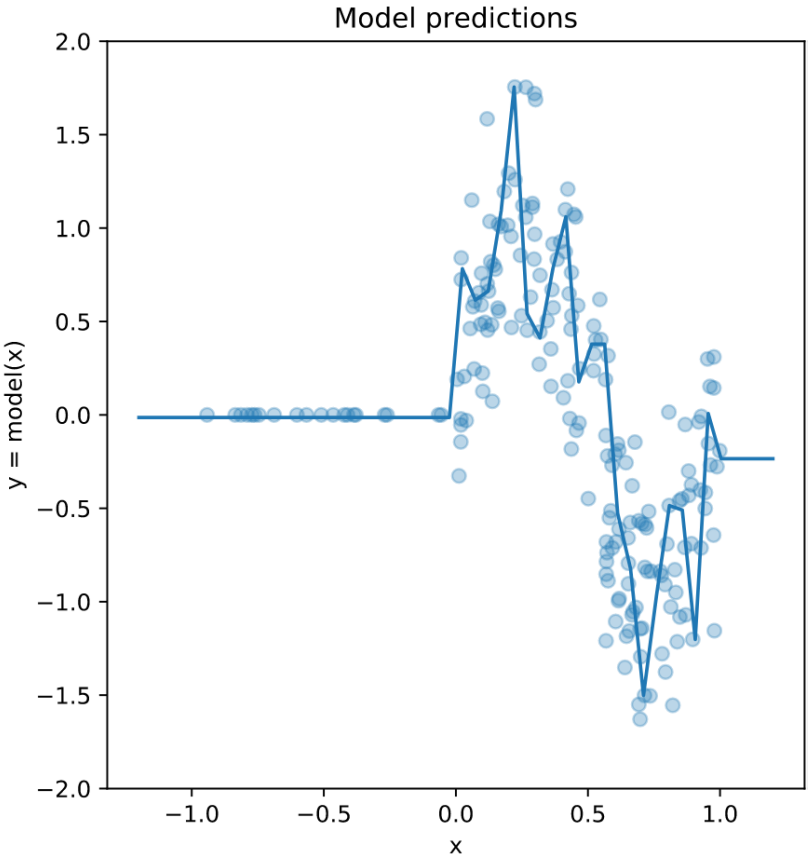




Università
Ca'Foscari
Venezia

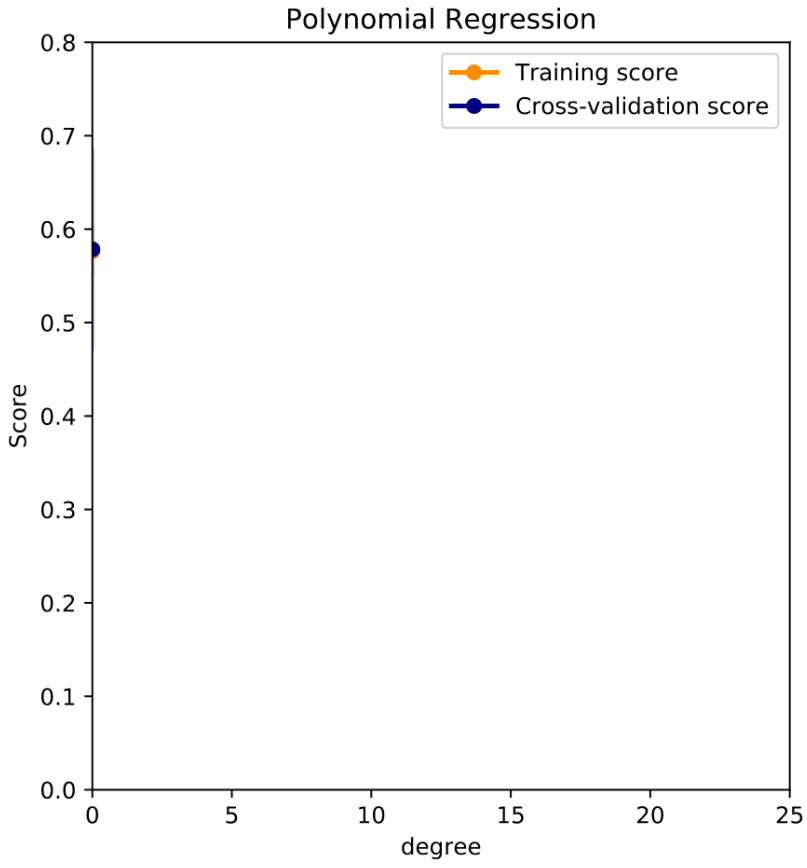
Single Decision Tree

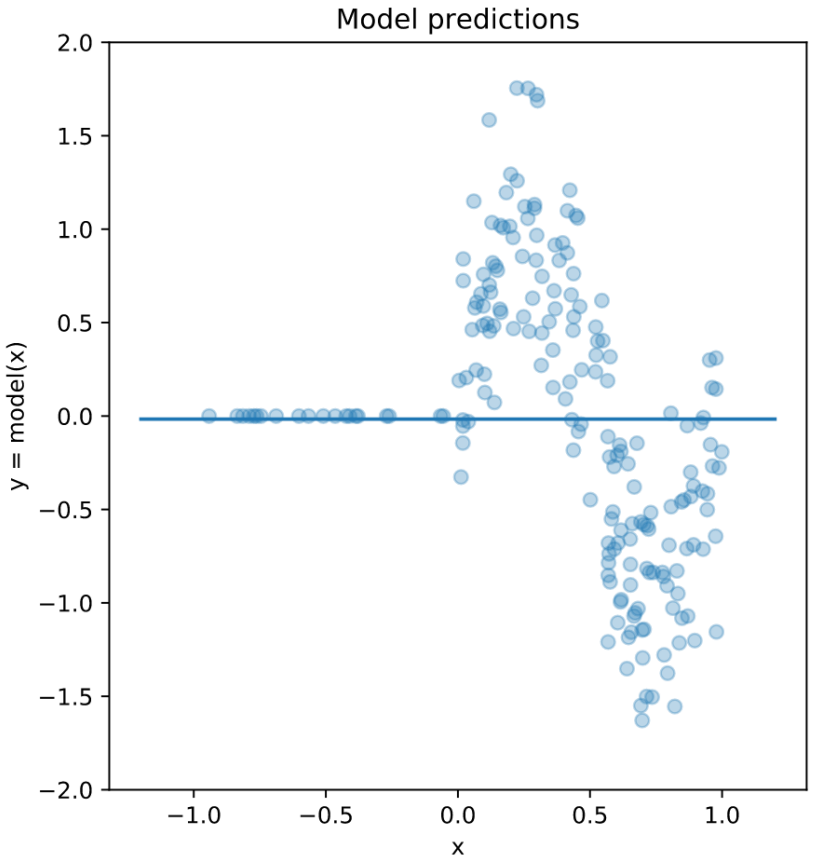




Università
Ca'Foscari
Venezia

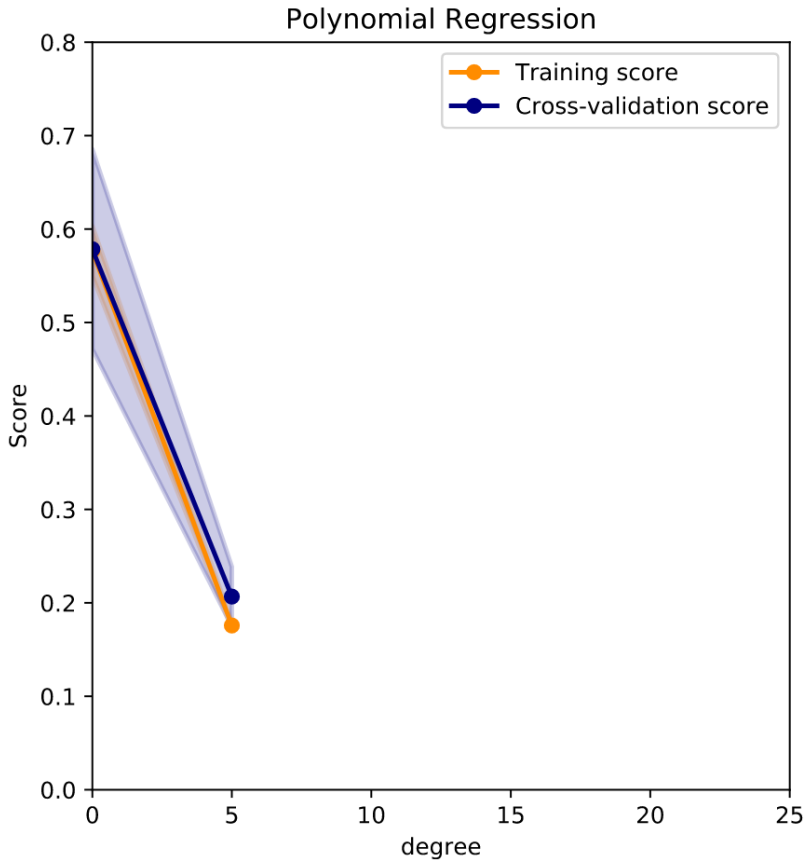
Polynomial Regression

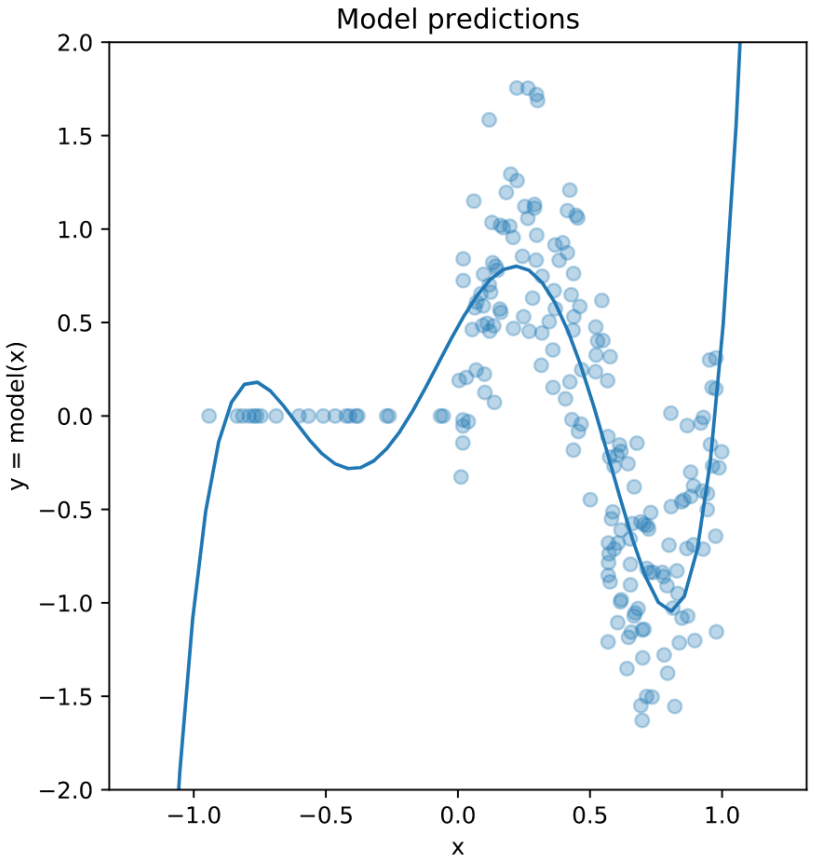




Università
Ca'Foscari
Venezia

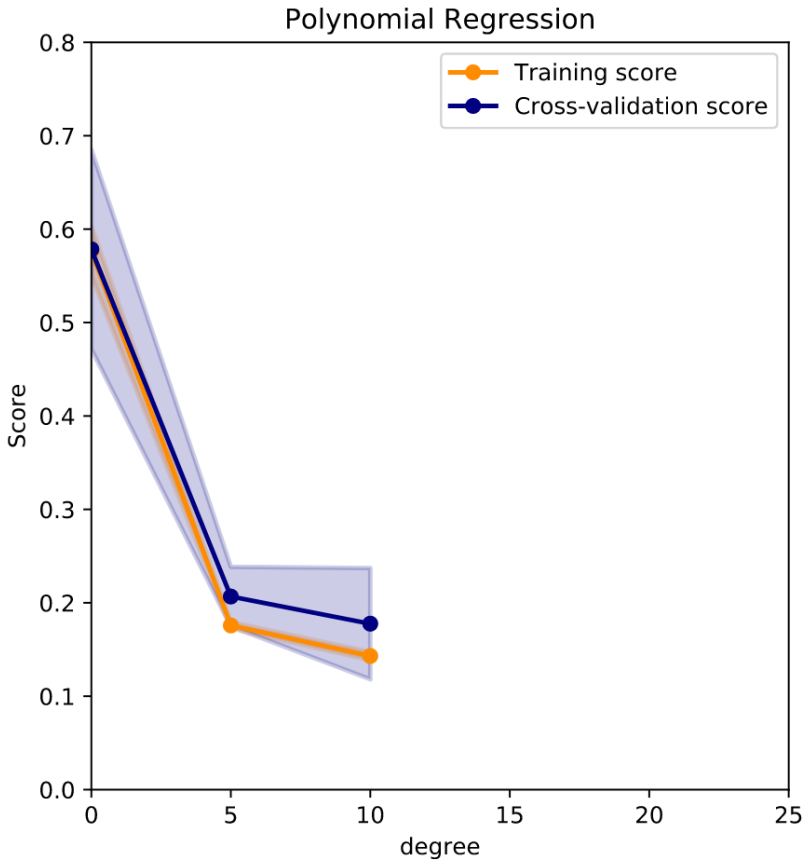
Polynomial Regression

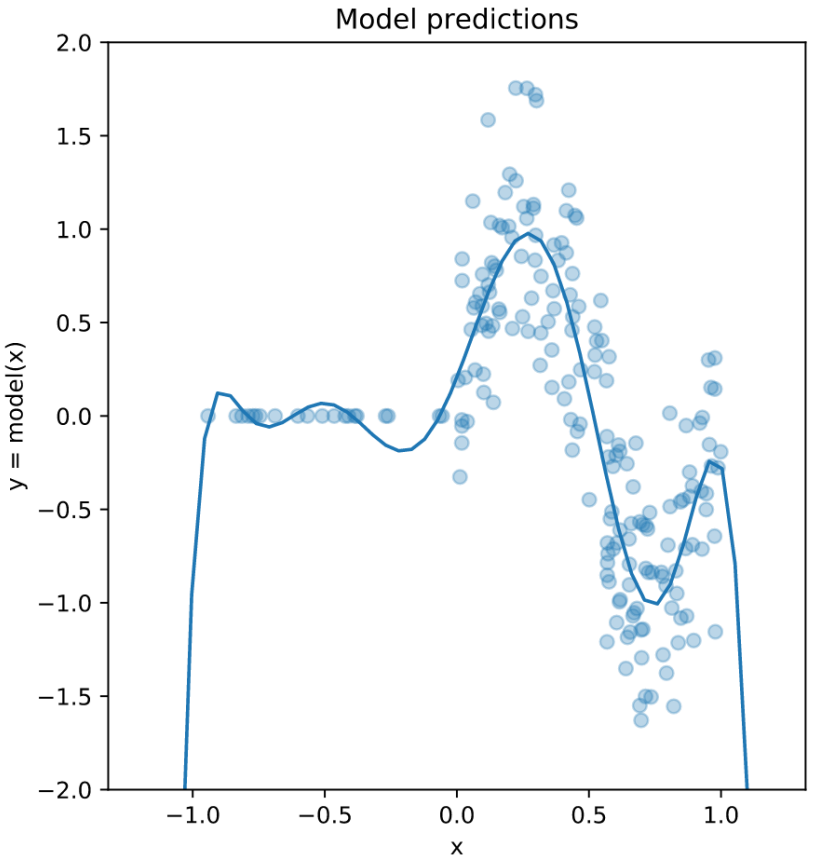




Università
Ca'Foscari
Venezia

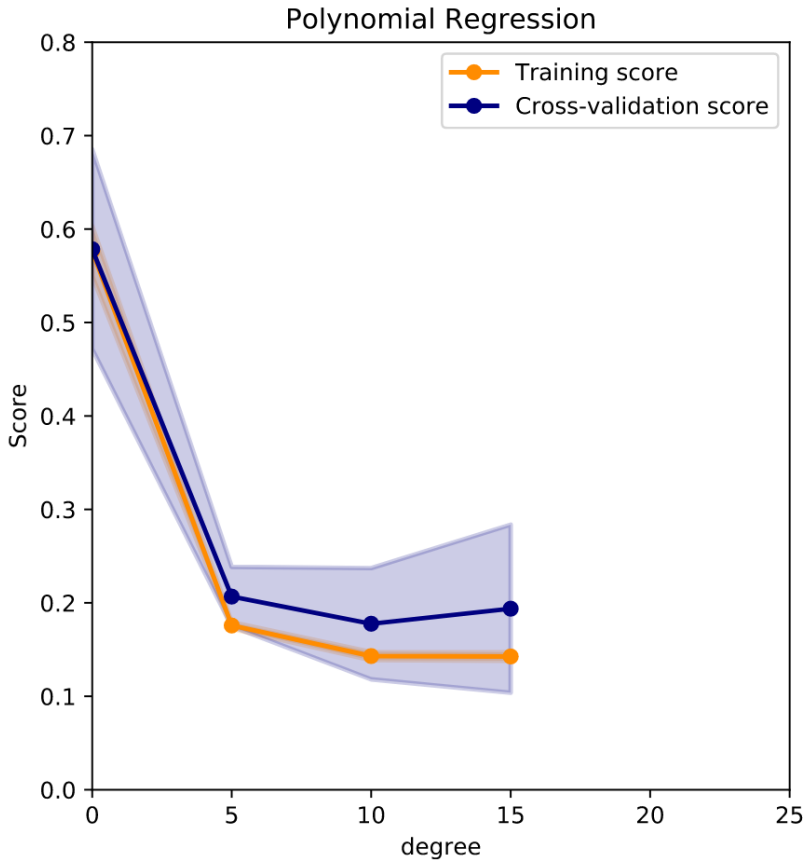
Polynomial Regression

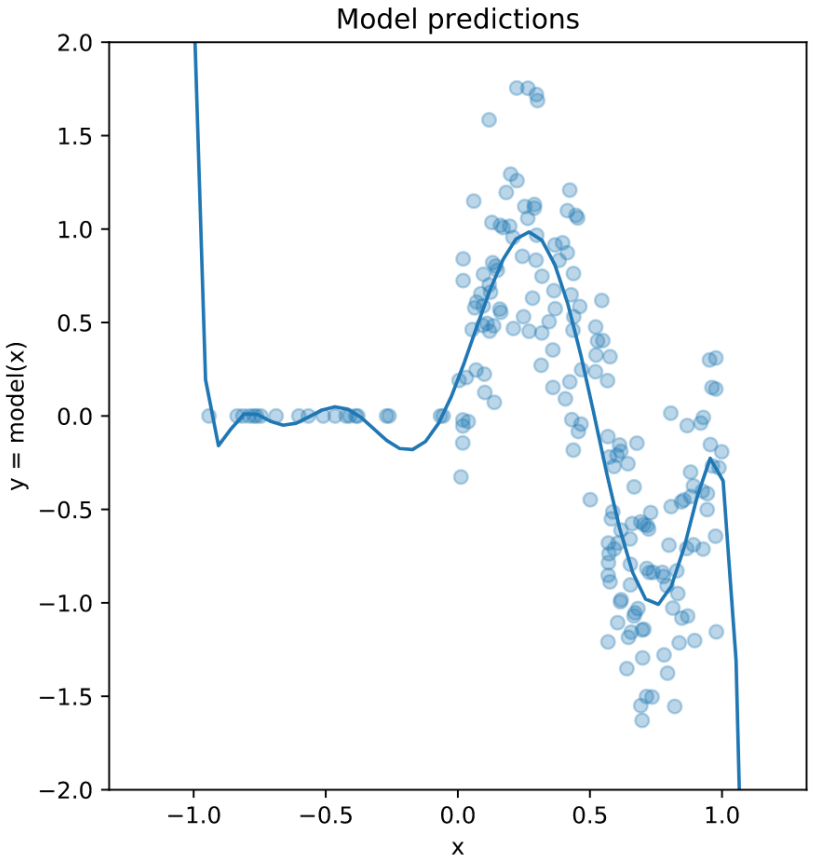




Università
Ca'Foscari
Venezia

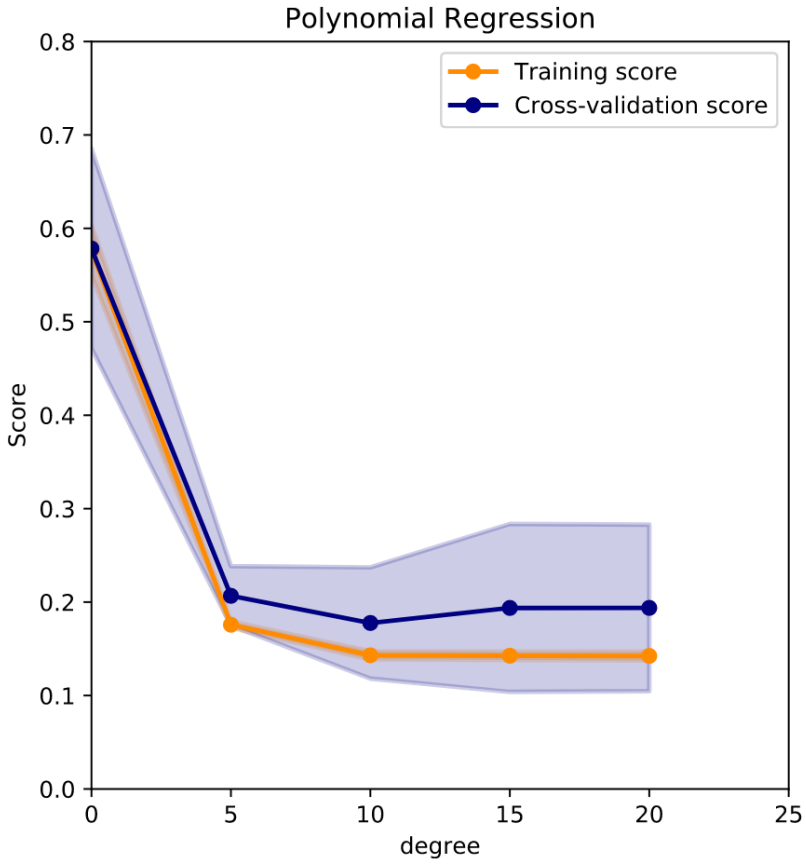
Polynomial Regression

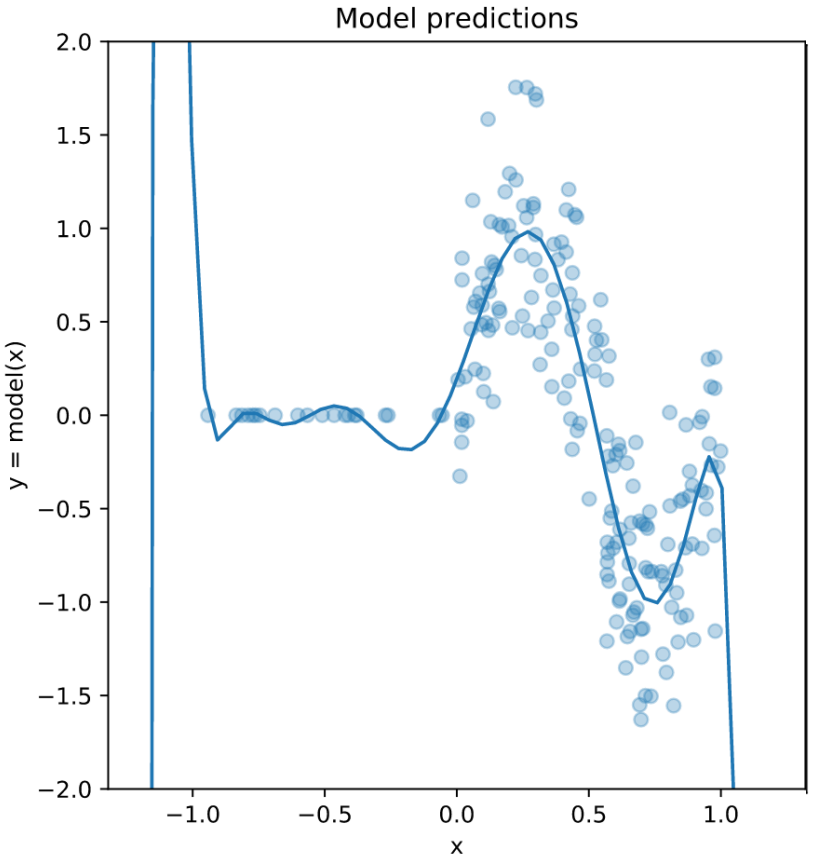




Università
Ca'Foscari
Venezia

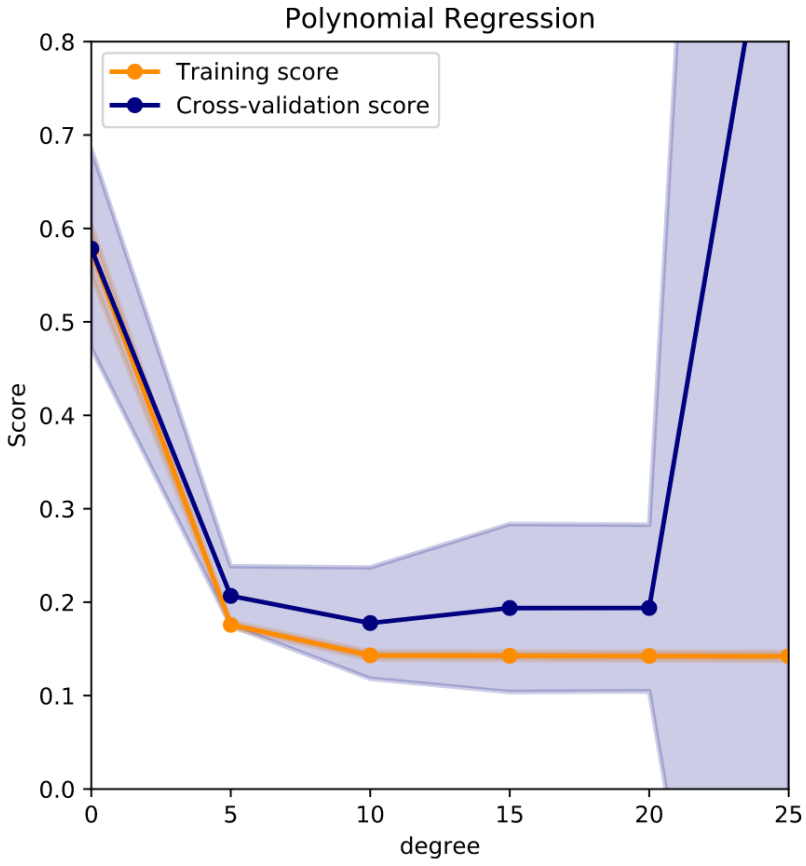
Polynomial Regression

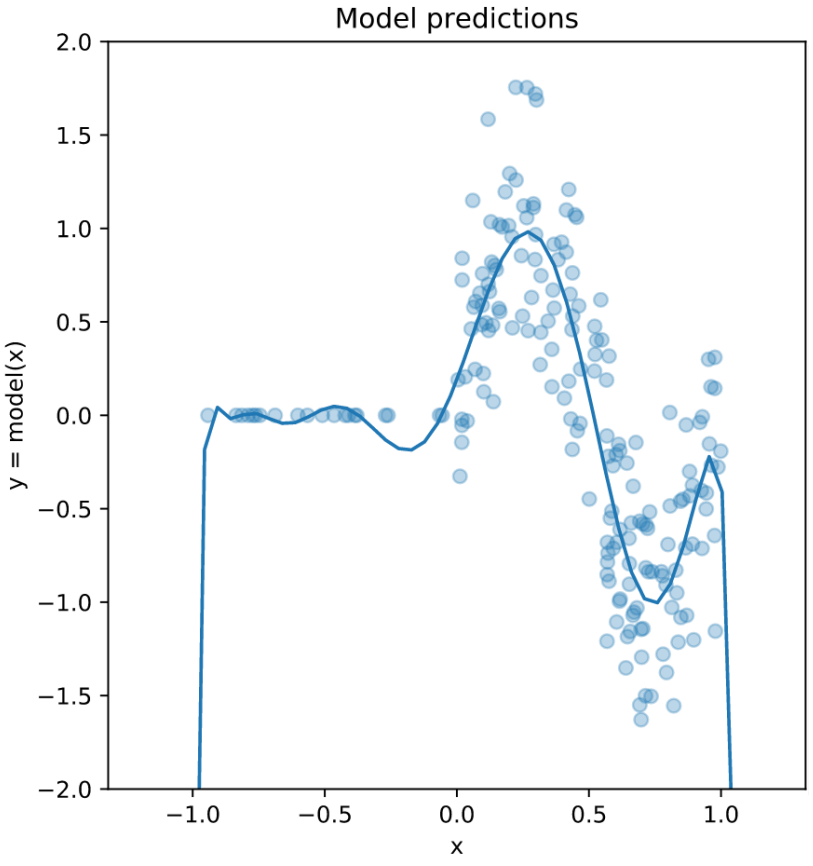




Università
Ca'Foscari
Venezia

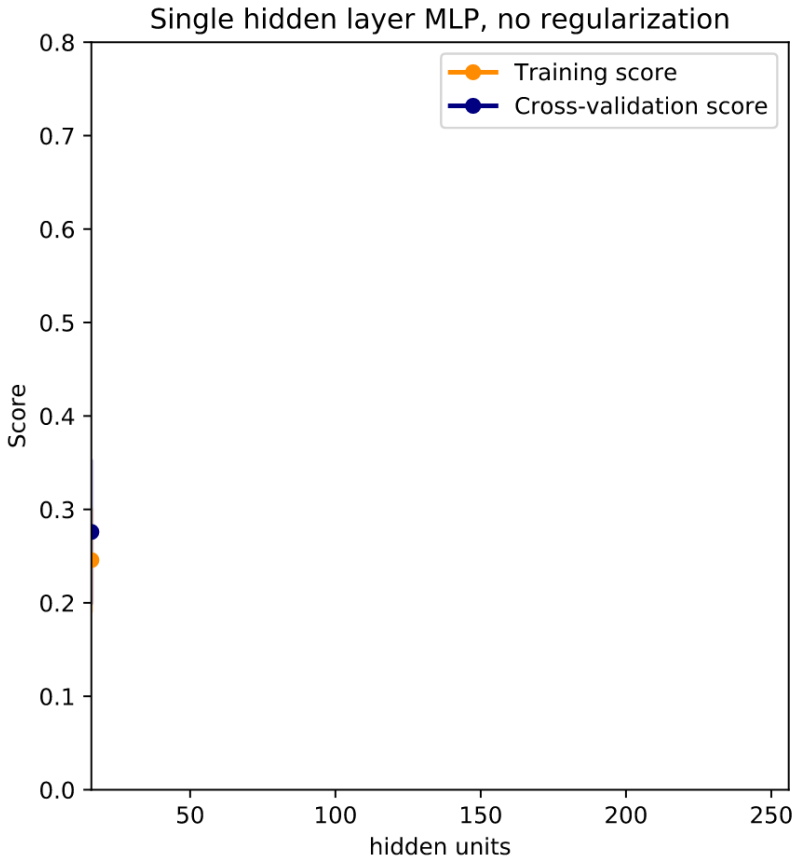
Polynomial Regression

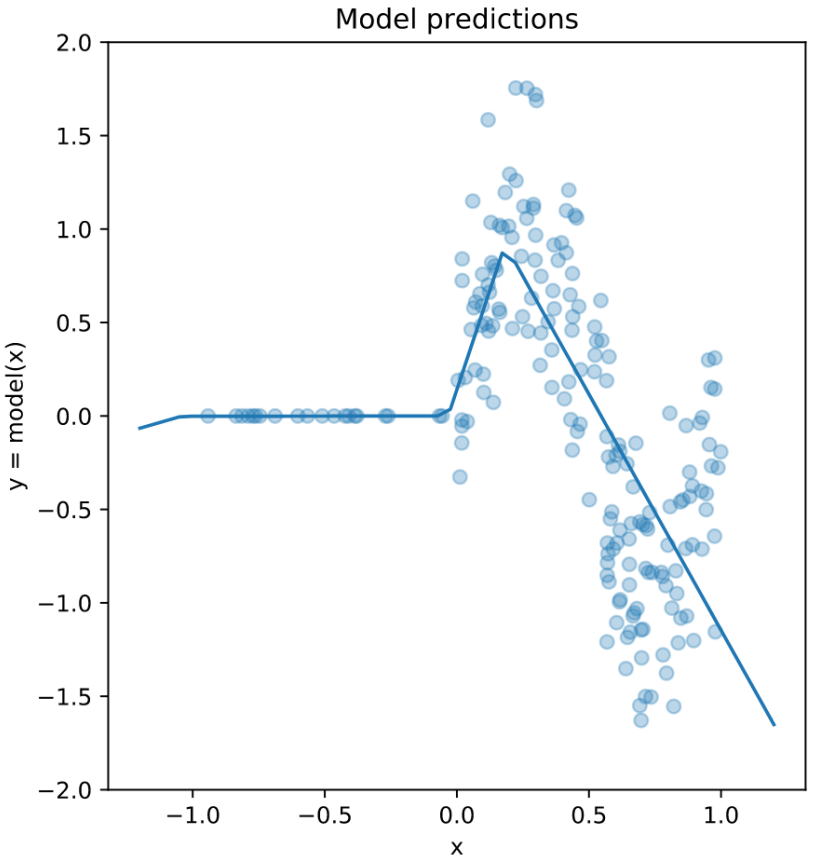




Università
Ca'Foscari
Venezia

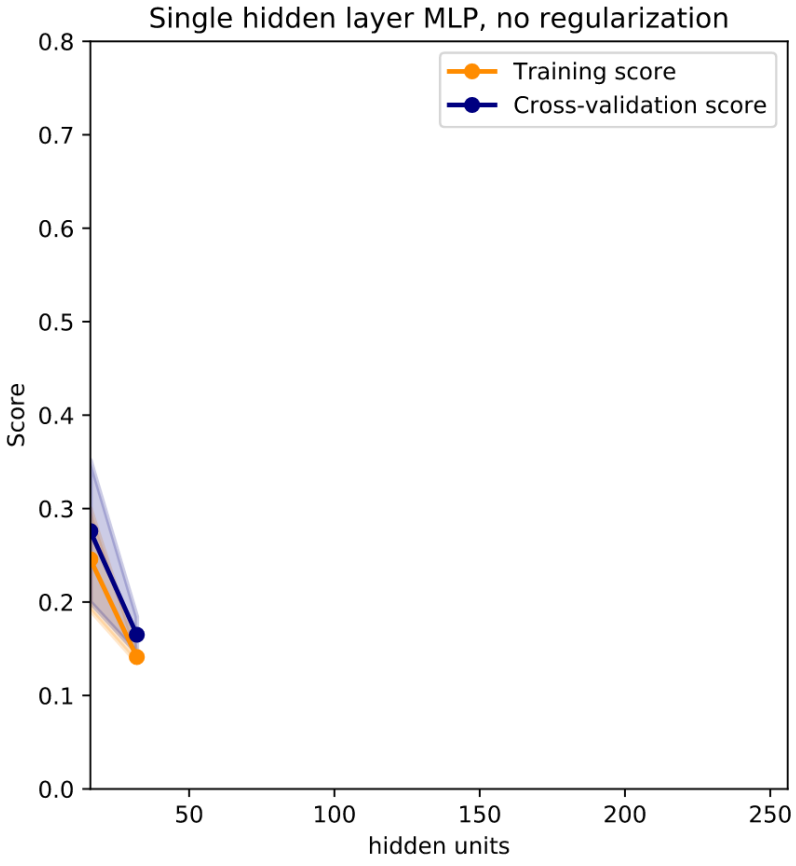
MLP (1 hidden layer)

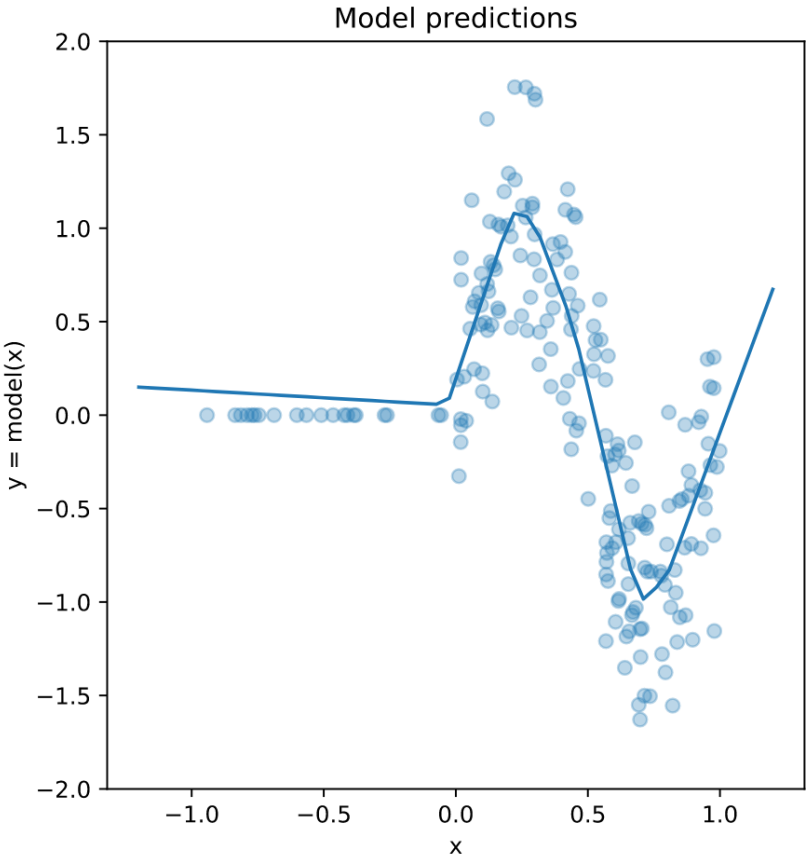




Università
Ca'Foscari
Venezia

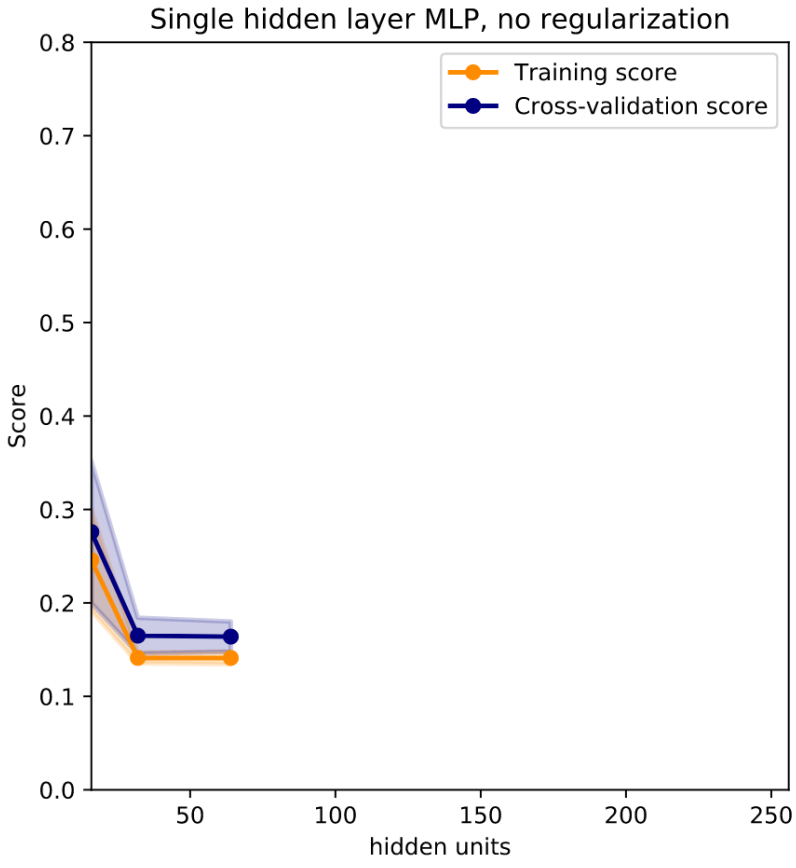
MLP (1 hidden layer)

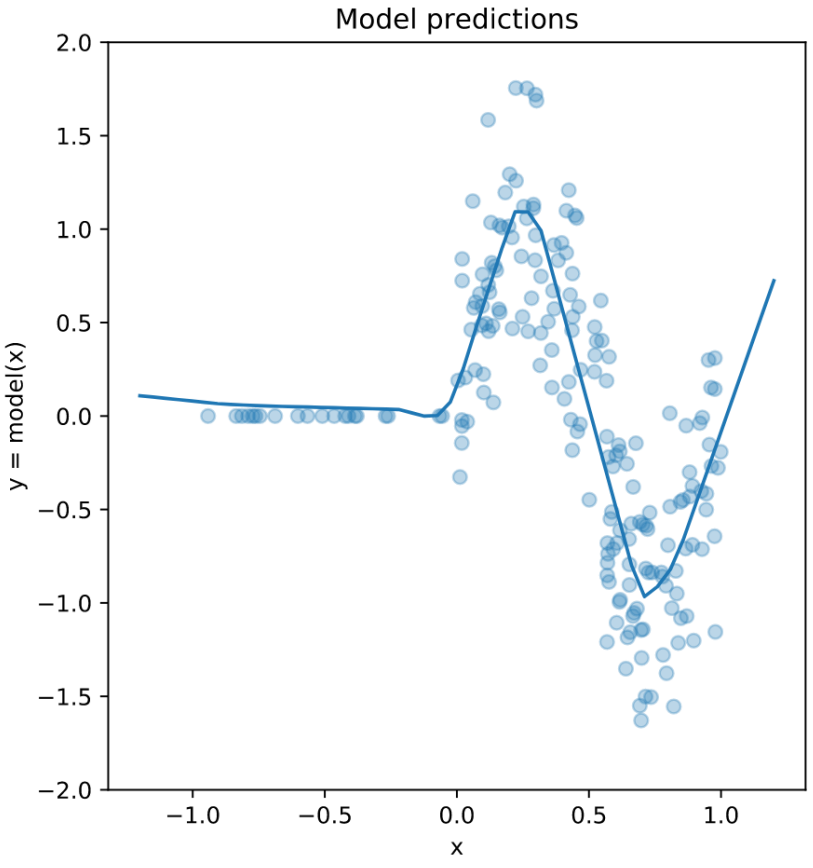




Università
Ca'Foscari
Venezia

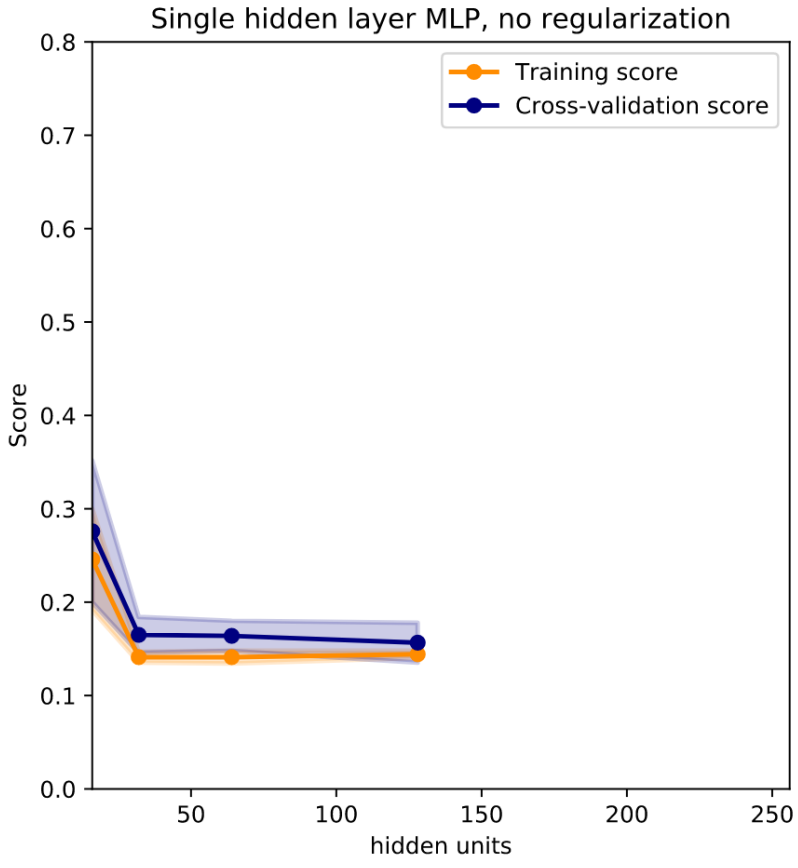
MLP (1 hidden layer)

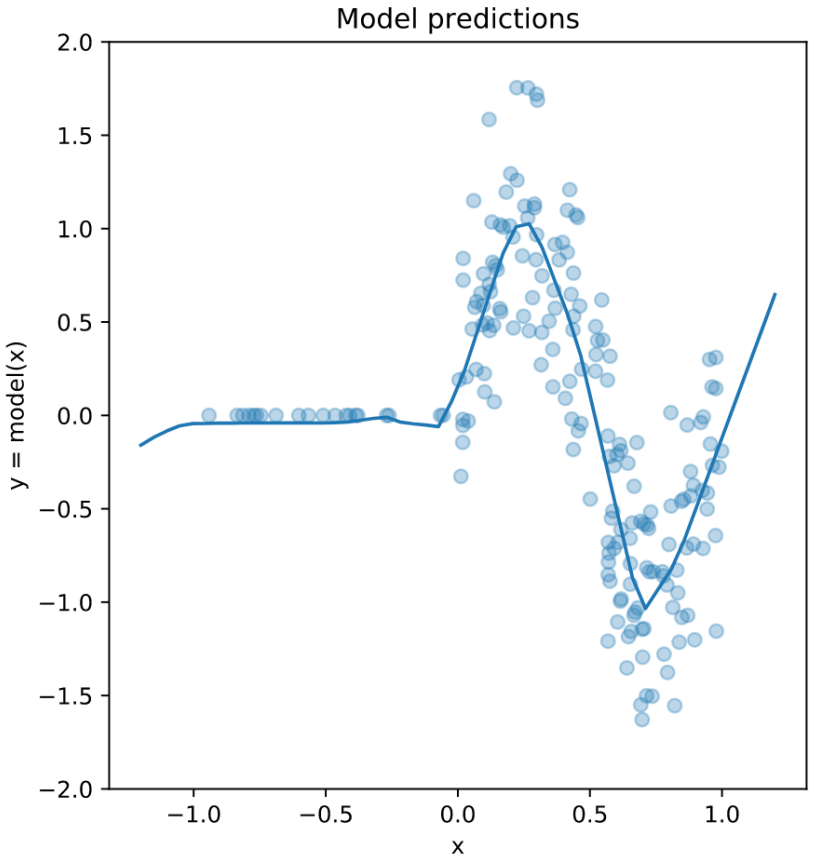




Università
Ca'Foscari
Venezia

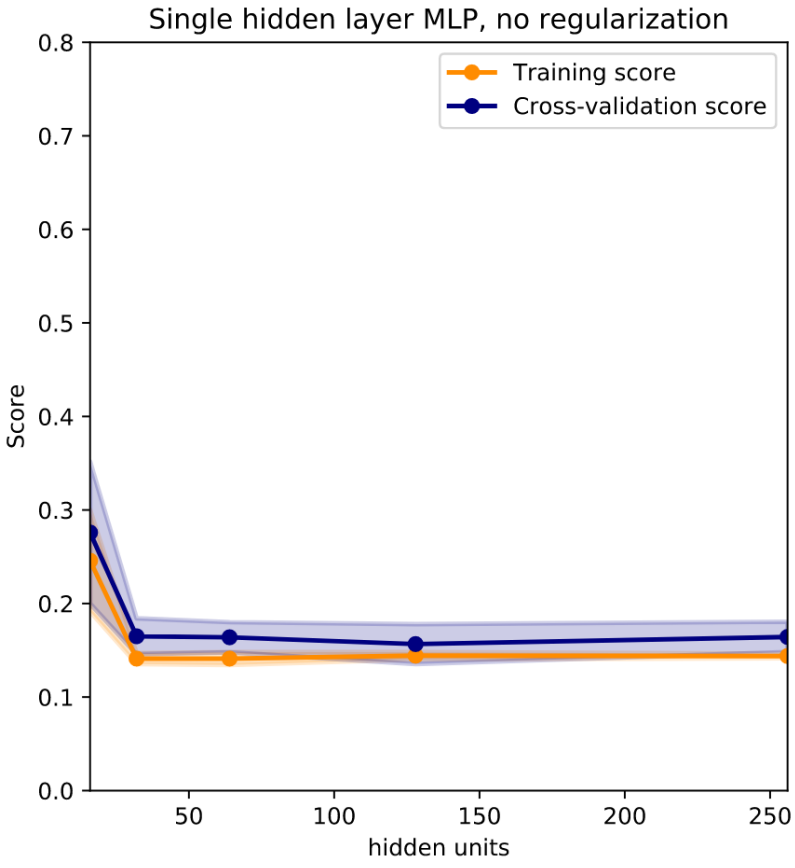
MLP (1 hidden layer)

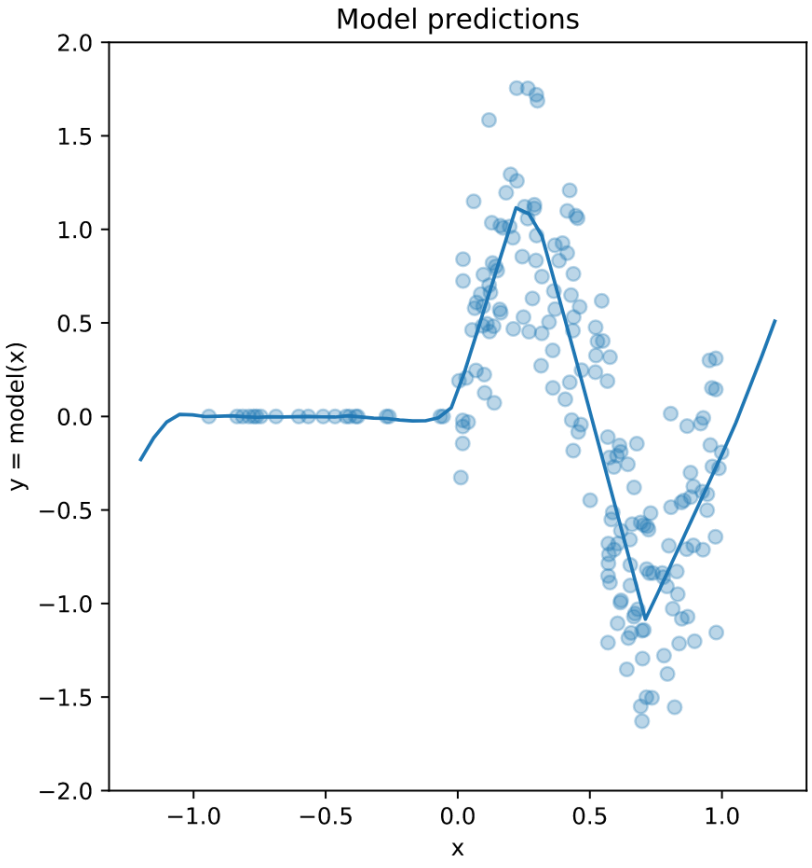




Università
Ca'Foscari
Venezia

MLP (1 hidden layer)





Non-intuitive behavior of MLP

Adding hidden units (larger width):

- rarely lead to (more) overfitting
- make optimization easier (less steps)
- make computation slower (more FLOPS per pass)

Adding layers (larger depth):

- does not always cause much more overfitting either
- can cause optimization issues (underfitting)

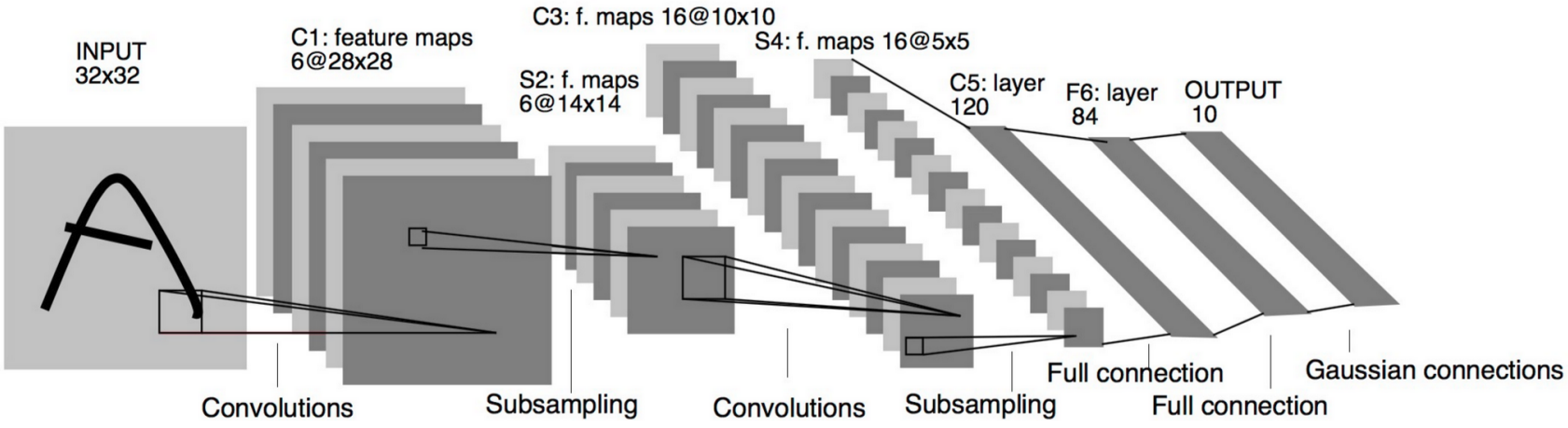
The Mistery of Generalization

What makes deep network generalize so well?

- Intrinsic to the piecewise linear nature of the ReLU NN?
- Deep compositional architectures = strong inductive prior?
- SGD with large step sizes, small minibatches and early stopping?
- Gradient Descent on over-parametrized net from small random init?
- A combination of all of the above?

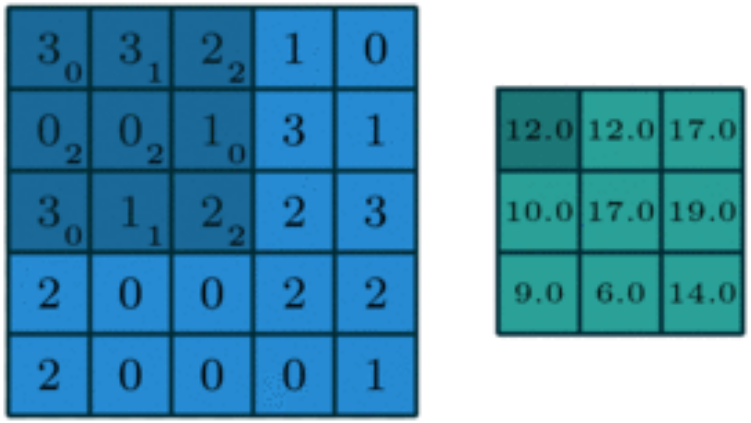
Università
Ca'Foscari
Venezia

Convolutional Neural Networks



LeCun, Y., Bottou, L., Bengio, Y., and Haffner, P. (1998). Gradient-based learning applied to document recognition.

Convolution



x is a 3×3 chunk (dark area) of the image (blue array)

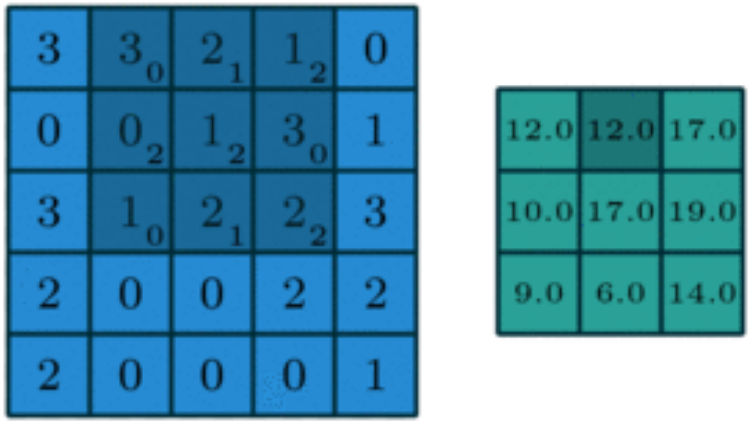
Each output neuron is parametrized with the 3×3 weight matrix w (small numbers)

The activation obtained by sliding the 3×3 window and computing:

$$z(x) = \text{relu}(w^T x + b)$$

Università
Ca'Foscari
Venezia

Convolution



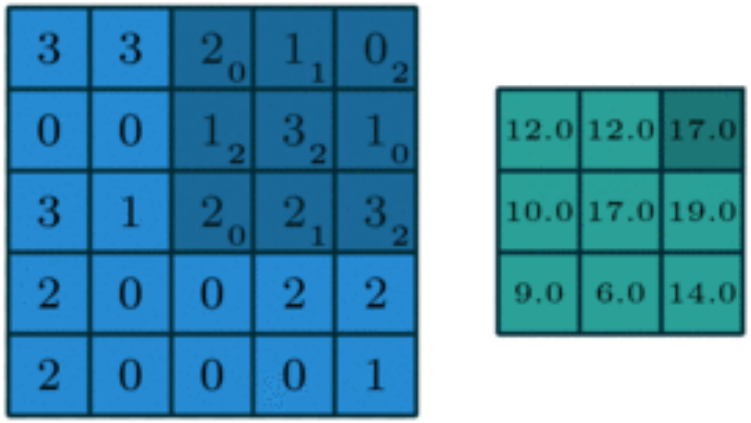
x is a 3×3 chunk (dark area) of the image (blue array)

Each output neuron is parametrized with the 3×3 weight matrix w (small numbers)

The activation obtained by sliding the 3×3 window and computing:

$$z(x) = \text{relu}(w^T x + b)$$

Convolution



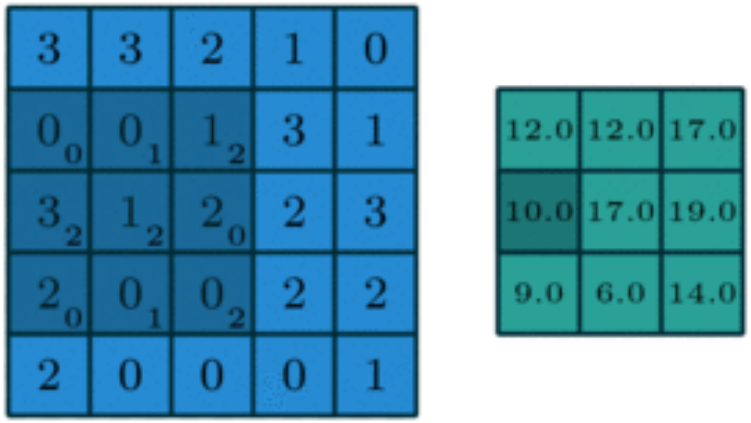
x is a 3×3 chunk (dark area) of the image (blue array)

Each output neuron is parametrized with the 3×3 weight matrix w (small numbers)

The activation obtained by sliding the 3×3 window and computing:

$$z(x) = \text{relu}(w^T x + b)$$

Convolution



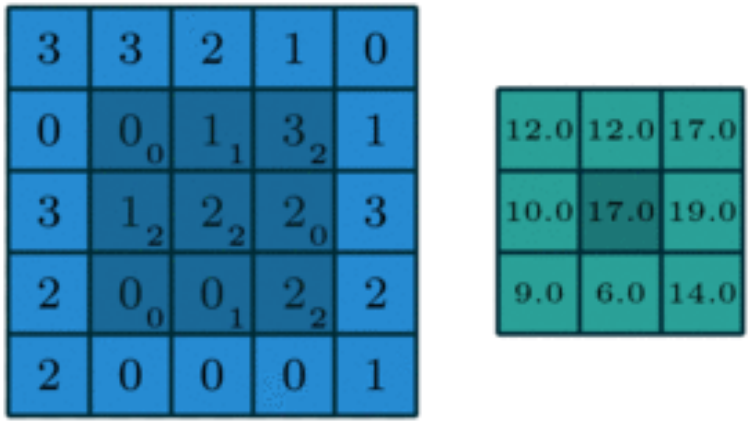
x is a 3×3 chunk (dark area) of the image (blue array)

Each output neuron is parametrized with the 3×3 weight matrix w (small numbers)

The activation obtained by sliding the 3×3 window and computing:

$$z(x) = \text{relu}(w^T x + b)$$

Convolution



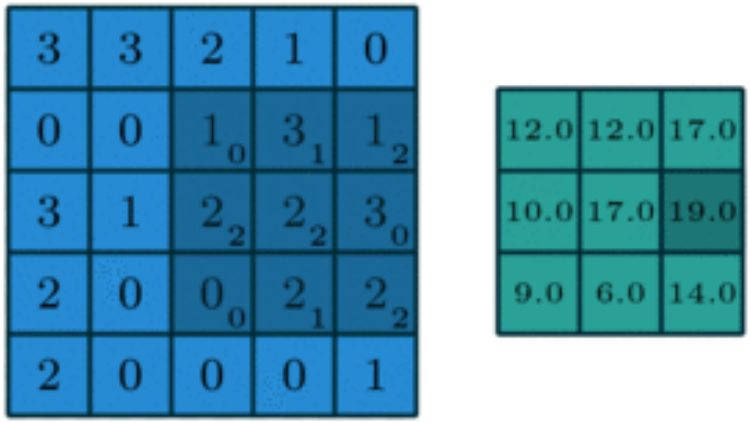
x is a 3×3 chunk (dark area) of the image (blue array)

Each output neuron is parametrized with the 3×3 weight matrix w (small numbers)

The activation obtained by sliding the 3×3 window and computing:

$$z(x) = \text{relu}(w^T x + b)$$

Convolution



x is a 3×3 chunk (dark area) of the image (blue array)

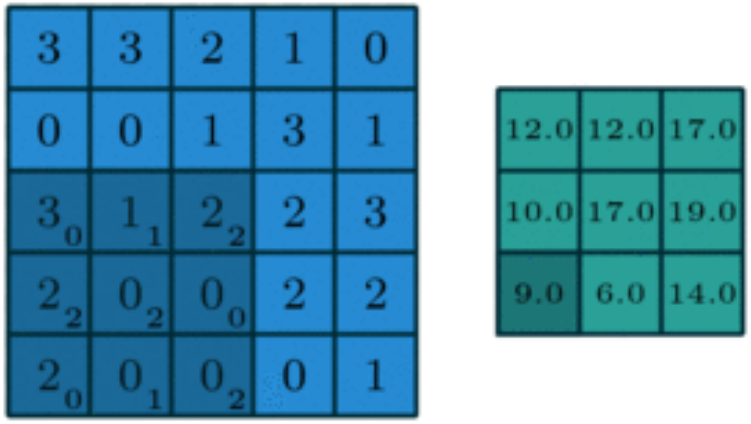
Each output neuron is parametrized with the 3×3 weight matrix w (small numbers)

The activation obtained by sliding the 3×3 window and computing:

$$z(x) = \text{relu}(w^T x + b)$$

Università
Ca'Foscari
Venezia

Convolution



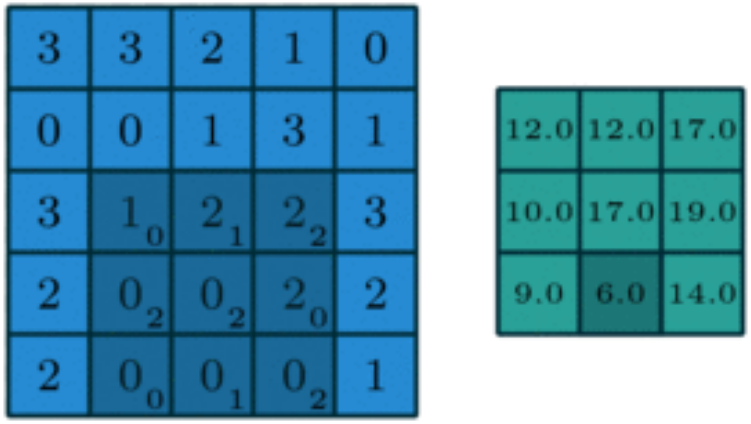
x is a 3×3 chunk (dark area) of the image (blue array)

Each output neuron is parametrized with the 3×3 weight matrix w (small numbers)

The activation obtained by sliding the 3×3 window and computing:

$$z(x) = \text{relu}(w^T x + b)$$

Convolution



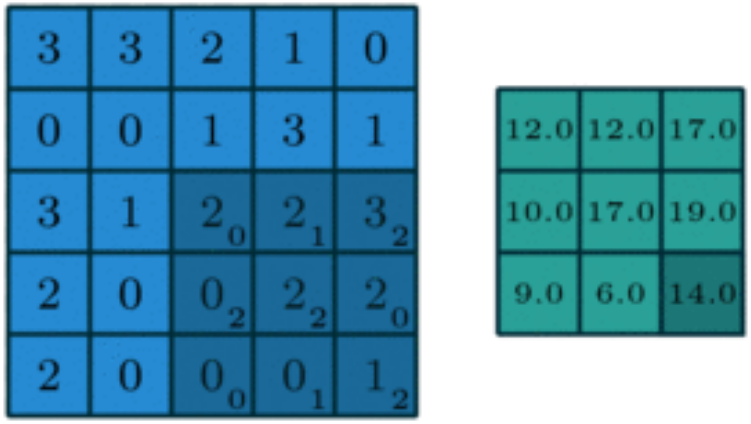
x is a 3×3 chunk (dark area) of the image (blue array)

Each output neuron is parametrized with the 3×3 weight matrix w (small numbers)

The activation obtained by sliding the 3×3 window and computing:

$$z(x) = \text{relu}(w^T x + b)$$

Convolution



x is a 3×3 chunk (dark area) of the image (blue array)

Each output neuron is parametrized with the 3×3 weight matrix w (small numbers)

The activation obtained by sliding the 3×3 window and computing:

$$z(x) = \text{relu}(w^T x + b)$$

Motivations

Local connectivity

- A neuron depends only on a few local input neurons
- Translation invariance

Comparison to Fully connected

- Parameter sharing: reduce overfitting
- Make use of spatial structure: strong prior for vision!

Why Convolution

Discrete convolution (actually cross-correlation) between two functions f and g:

$$(f \star g)(x) = \sum_{a+b=x} f(a)g(b) = \sum_{a} f(a)g(x+a)$$

2D-convolutions (actually 2D cross-correlation):

$$(f \star g)(x,y) = \sum_{n}\sum_{m} f(n,m)g(x+n,y+m)$$

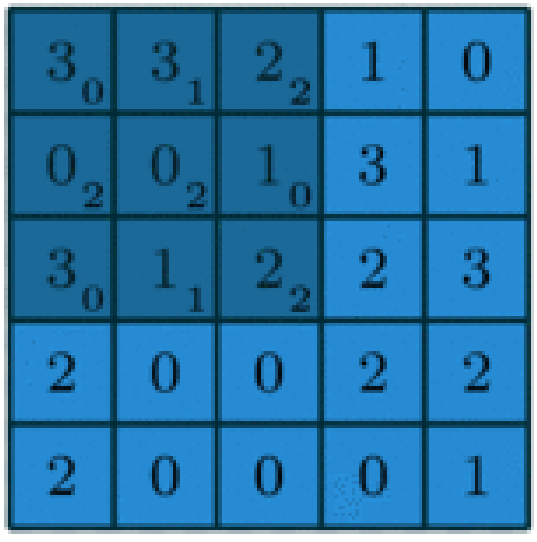
f is a convolution kernel or filter applied to the 2-d map g (our image)

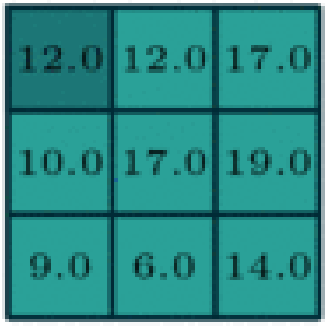
Example

Image: im of dimensions 5×5

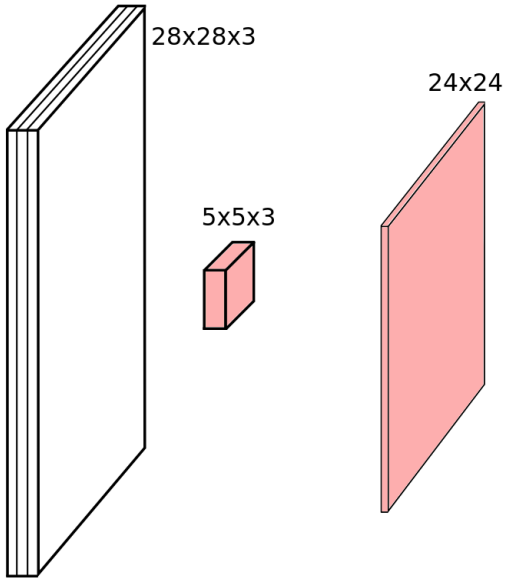
Kernel: k of dimensions 3×3

$$(k \star im)(x, y) = \sum_{n=0}^{2} \sum_{m=0}^{2} k(n, m) im(x + n - 1, y + m - 1)$$





Channels

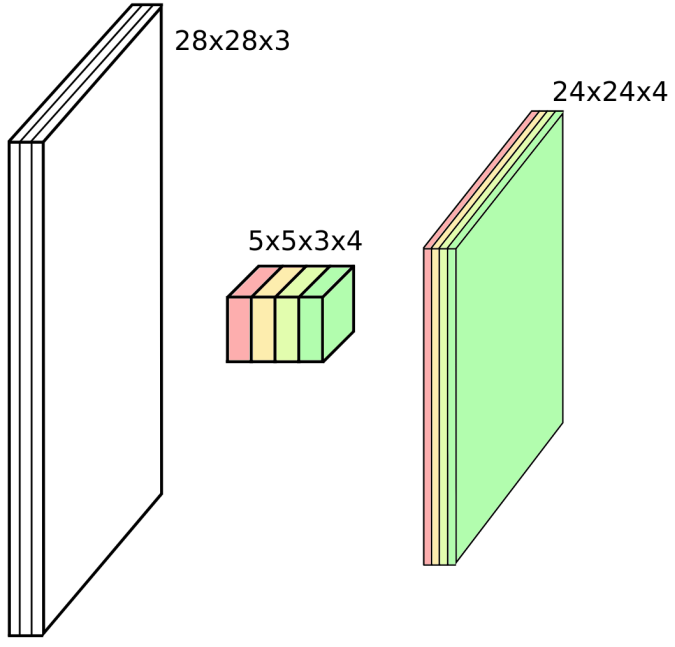


Colored image = tensor of shape (height, width, channels)
Convolutions are usually computed for each channel and summed:

$$(k \star im^{color}) = \sum_{c=0}^{2} k^c \, im^c$$

Università
Ca'Foscari
Venezia

Multiple convolutions



Kernel size aka receptive field (usually 1, 3, 5, 7, 11)

Output dimension: length - kernel_size + 1

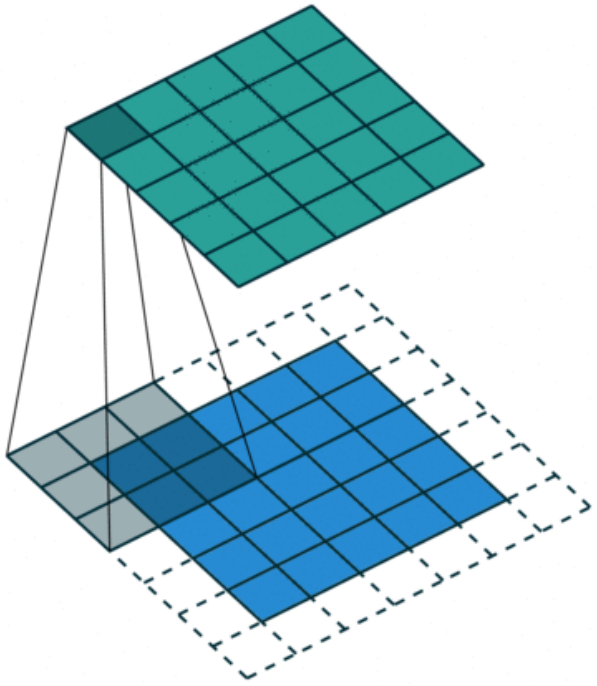
Stride and Padding

Strides: increment step size for the convolution operator

- Reduces the size of the output map

Padding: artificially fill borders of image

- Useful to keep spatial dimension constant across filters
- Useful with strides and large receptive fields



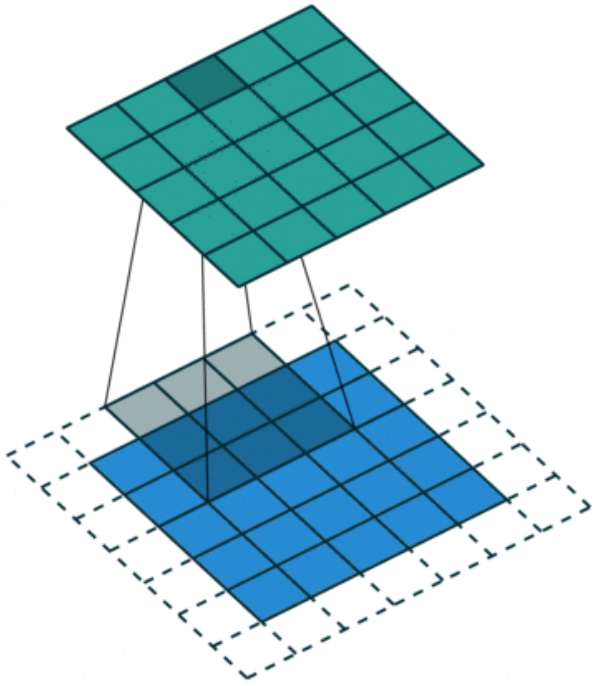
Stride and Padding

Strides: increment step size for the convolution operator

- Reduces the size of the output map

Padding: artificially fill borders of image

- Useful to keep spatial dimension constant across filters
- Useful with strides and large receptive fields



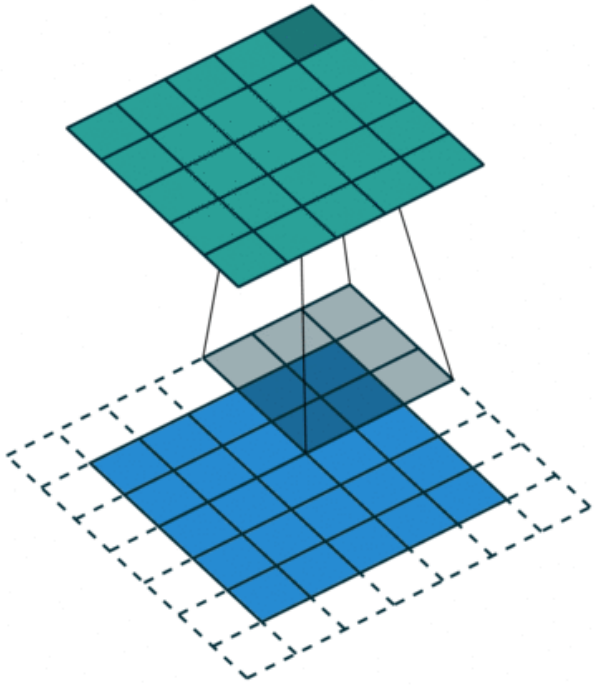
Stride and Padding

Strides: increment step size for the convolution operator

- Reduces the size of the output map

Padding: artificially fill borders of image

- Useful to keep spatial dimension constant across filters
- Useful with strides and large receptive fields



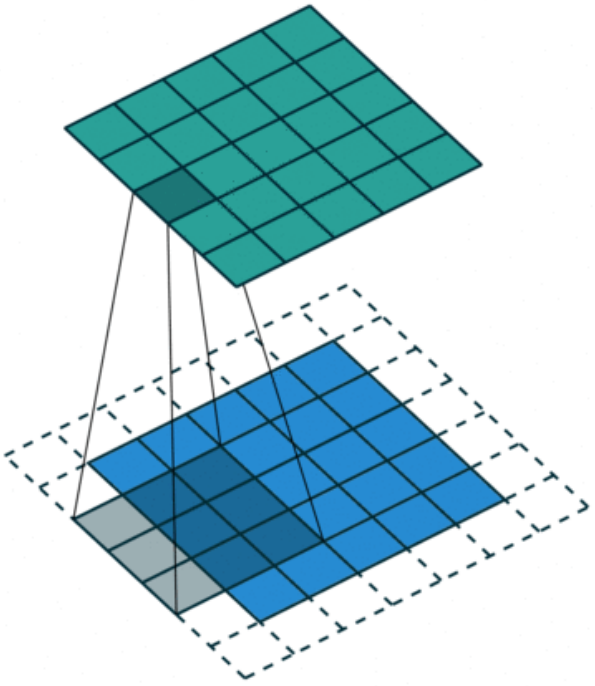
Stride and Padding

Strides: increment step size for the convolution operator

- Reduces the size of the output map

Padding: artificially fill borders of image

- Useful to keep spatial dimension constant across filters
- Useful with strides and large receptive fields



Università
Ca'Foscari
Venezia

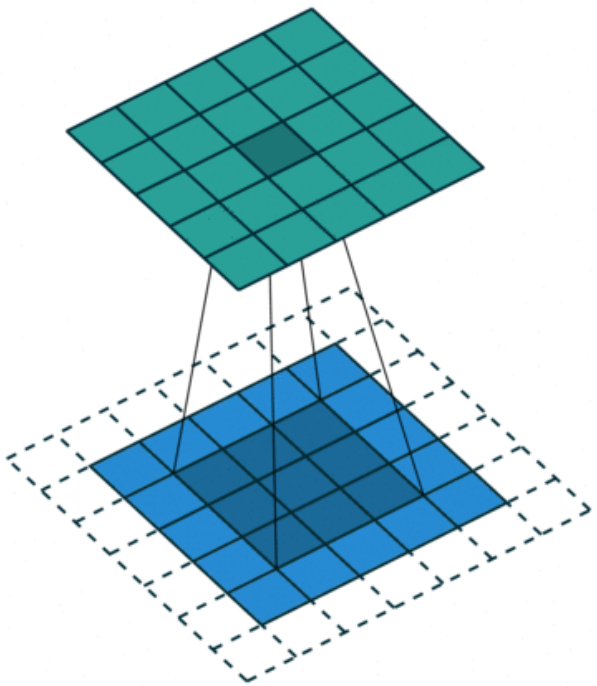
Stride and Padding

Strides: increment step size for the convolution operator

- Reduces the size of the output map

Padding: artificially fill borders of image

- Useful to keep spatial dimension constant across filters
- Useful with strides and large receptive fields



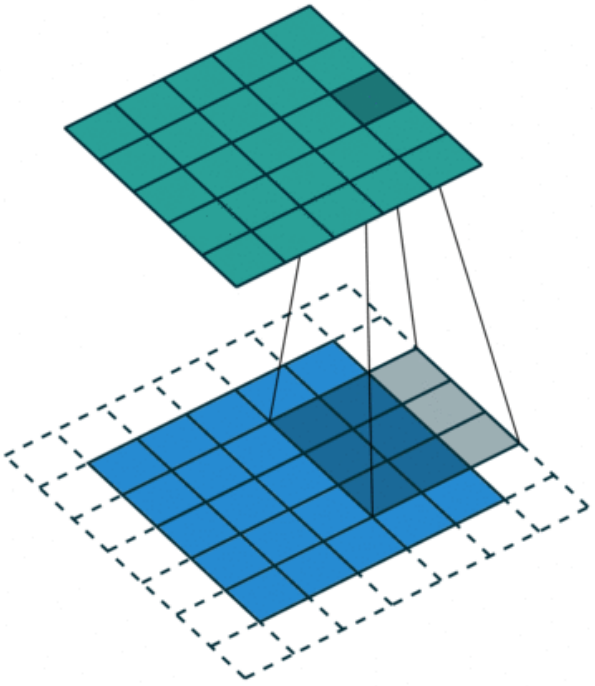
Stride and Padding

Strides: increment step size for the convolution operator

- Reduces the size of the output map

Padding: artificially fill borders of image

- Useful to keep spatial dimension constant across filters
- Useful with strides and large receptive fields



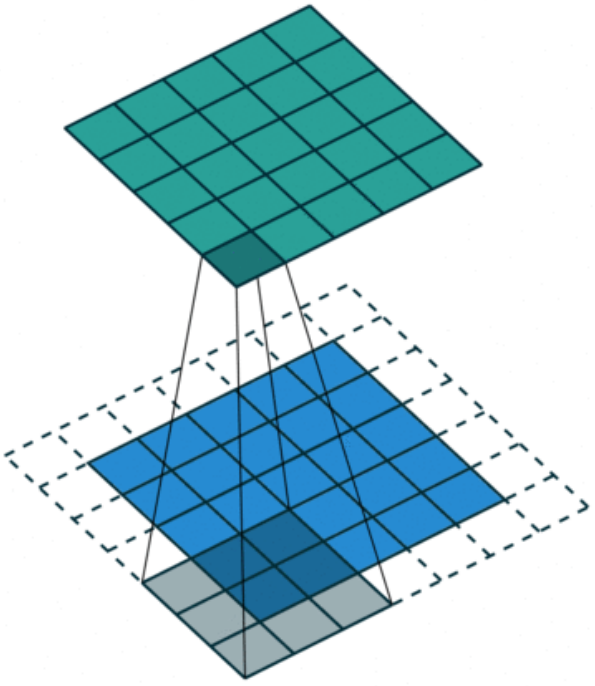
Stride and Padding

Strides: increment step size for the convolution operator

- Reduces the size of the output map

Padding: artificially fill borders of image

- Useful to keep spatial dimension constant across filters
- Useful with strides and large receptive fields



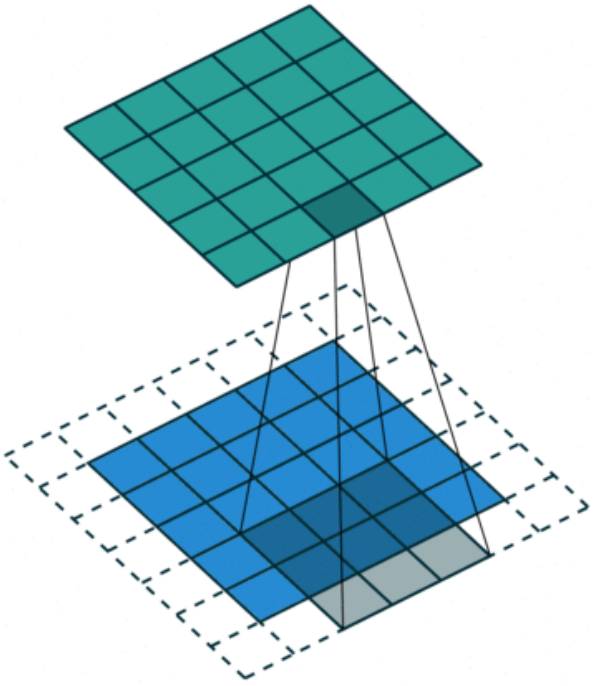
Stride and Padding

Strides: increment step size for the convolution operator

- Reduces the size of the output map

Padding: artificially fill borders of image

- Useful to keep spatial dimension constant across filters
- Useful with strides and large receptive fields



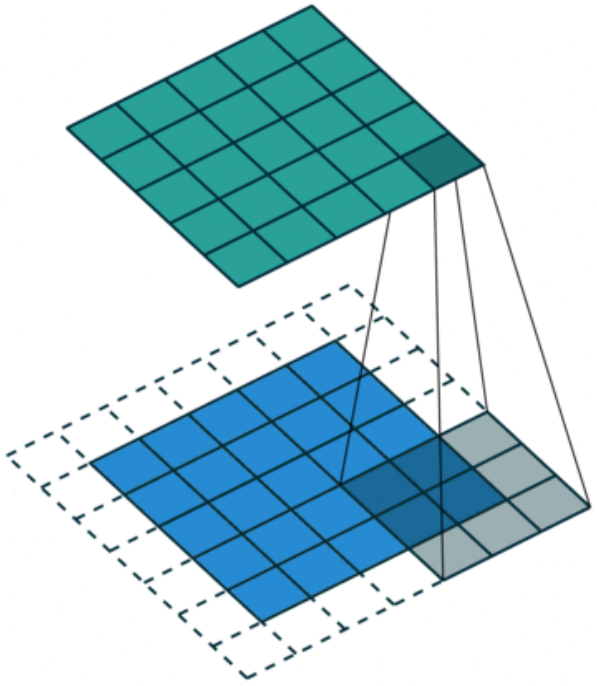
Stride and Padding

Strides: increment step size for the convolution operator

- Reduces the size of the output map

Padding: artificially fill borders of image

- Useful to keep spatial dimension constant across filters
- Useful with strides and large receptive fields

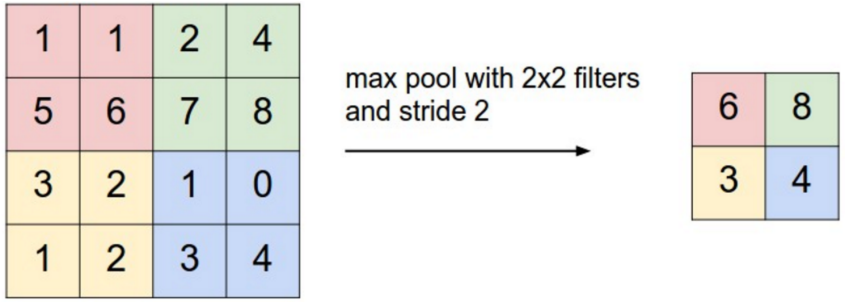


Pooling

Spatial dimension reduction

Local invariance

No parameters: max or average of 2×2 units



Classic CNN Architecture

Input

Conv blocks

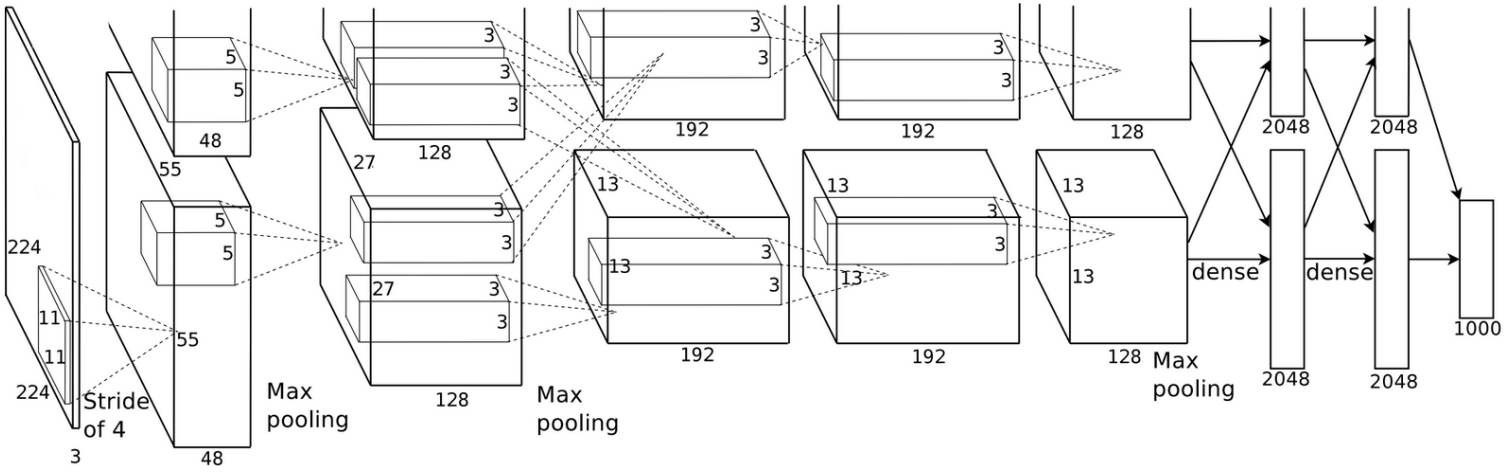
- Convolution + activation (relu)
- Convolution + activation (relu)
- ...
- Maxpooling 2x2

Output

- Fully connected layers
- Softmax

Università
Ca'Foscari
Venezia

AlexNet



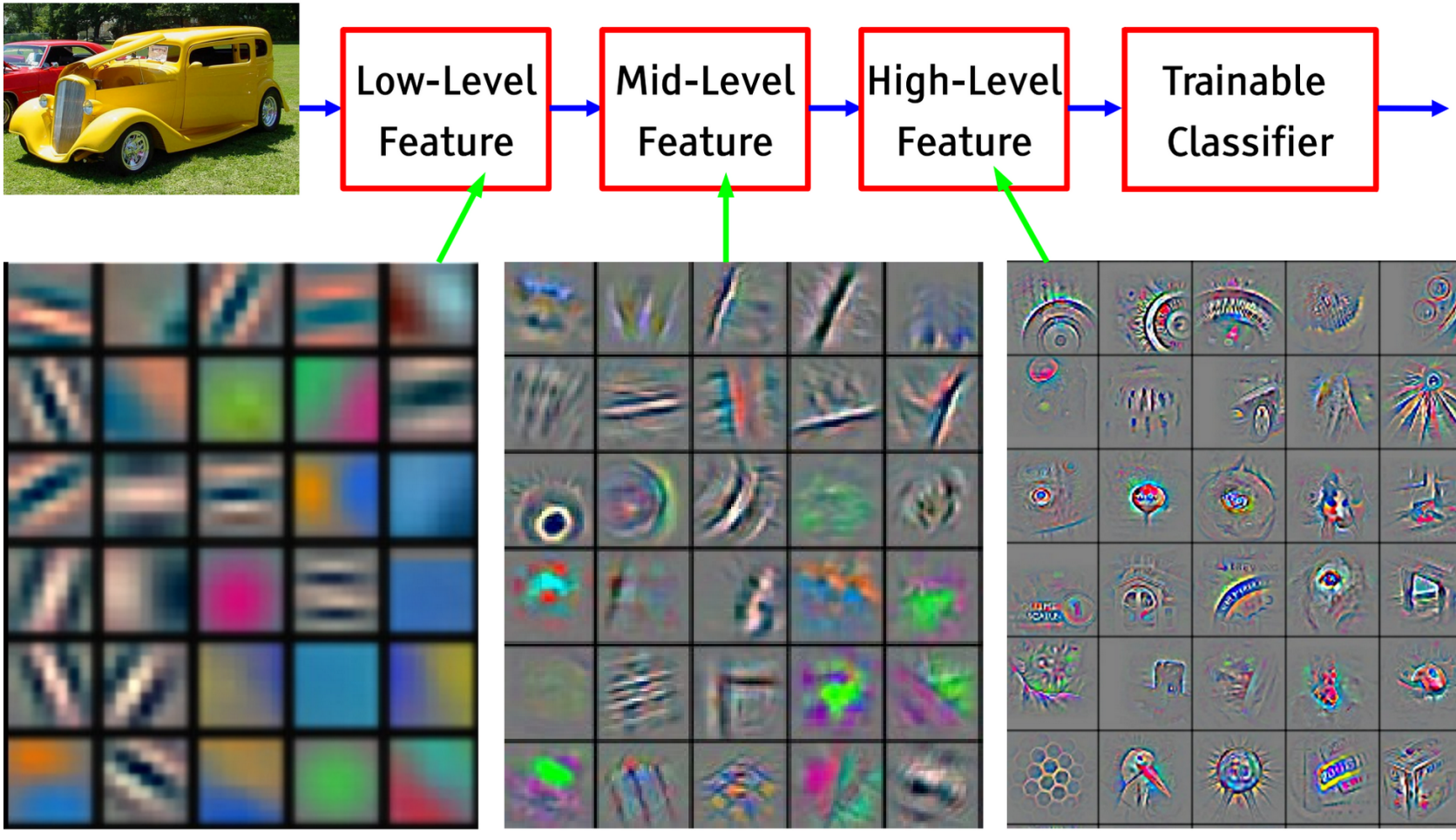
Krizhevsky, Alex, Sutskever, and Hinton. "Imagenet classification with deep convolutional neural networks." NIPS 2012

First conv layer: kernel 11x11x3x96 stride 4

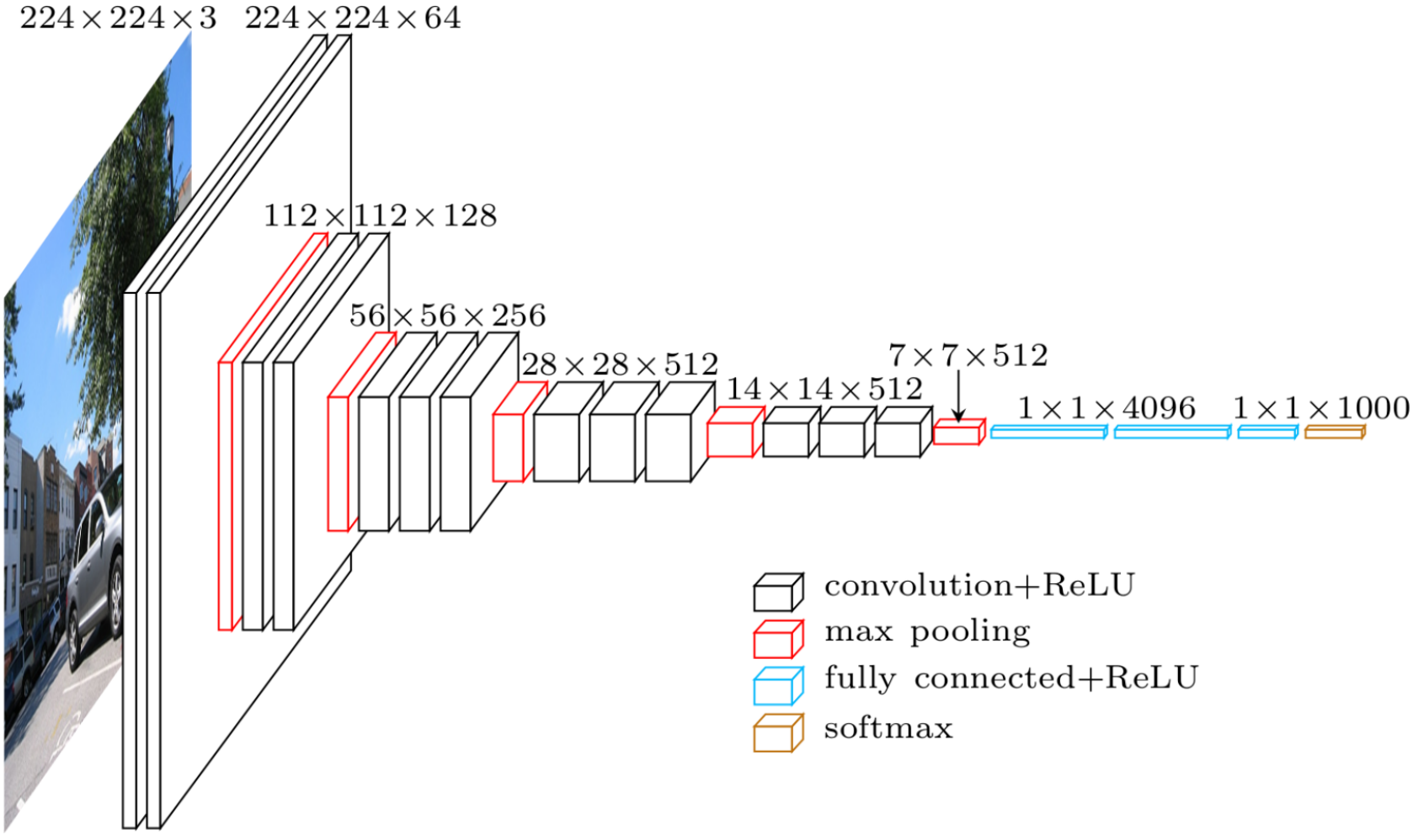
- Kernel shape: (11,11,3,96)
- Output shape: (55,55,96)
- Number of parameters: 34,944
- Equivalent MLP parameters: 43.7×10^9

Università
Ca'Foscari
Venezia

Hierarchical representation



VGG-16



Simonyan, Karen, and Zisserman. "Very deep convolutional networks for large-scale image recognition." (2014)

Università Ca'Foscari Venezia

ResNet

Even deeper models:
34, 50, 101, 152 layers

A block learns the residual w.r.t. identity

- Good optimization properties

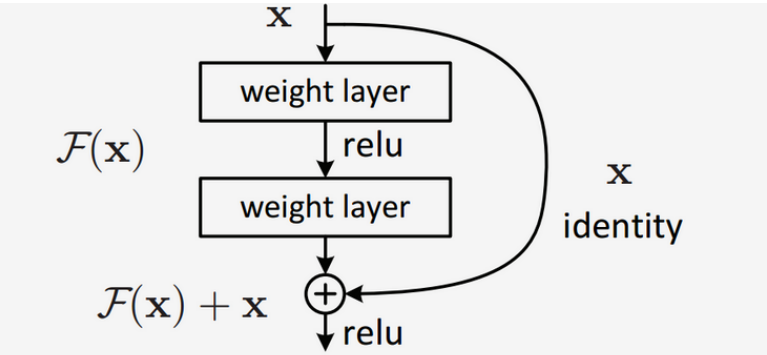
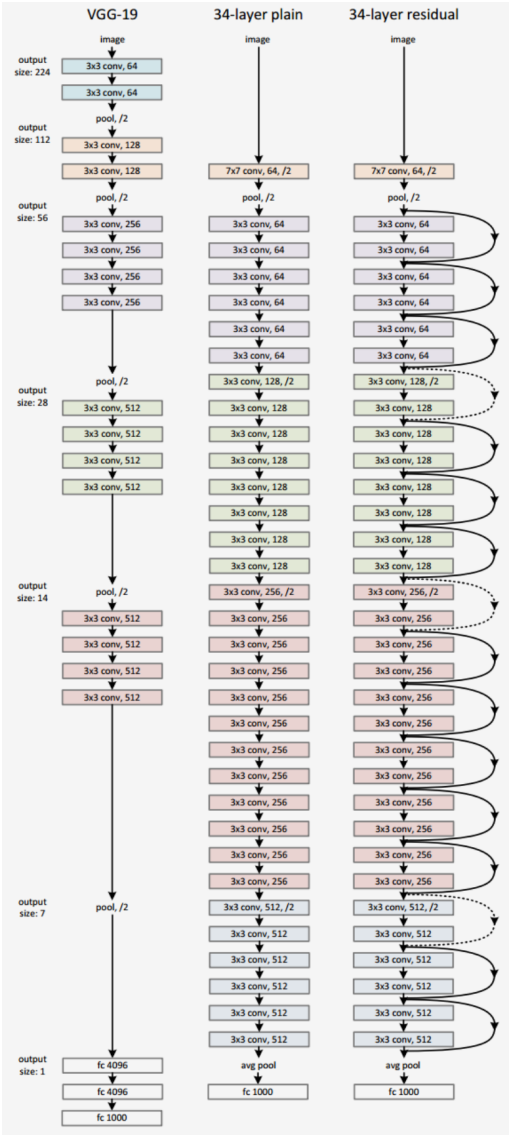


Figure 2. Residual learning: a building block.



He, Kaiming, et al. "Deep residual learning for image recognition." CVPR. 2016.

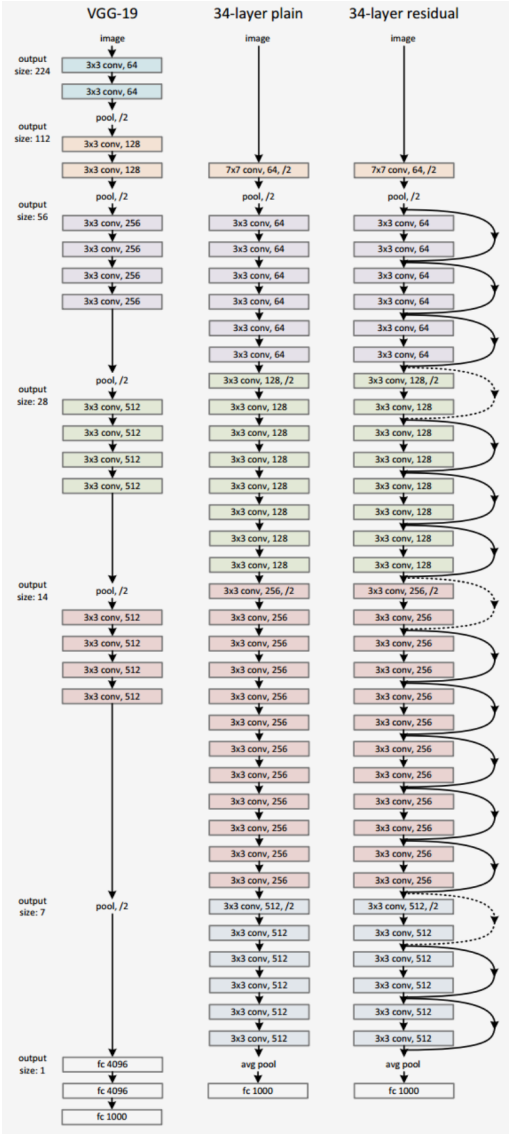
Università
Ca'Foscari
Venezia

ResNet

ResNet50 Compared to VGG:

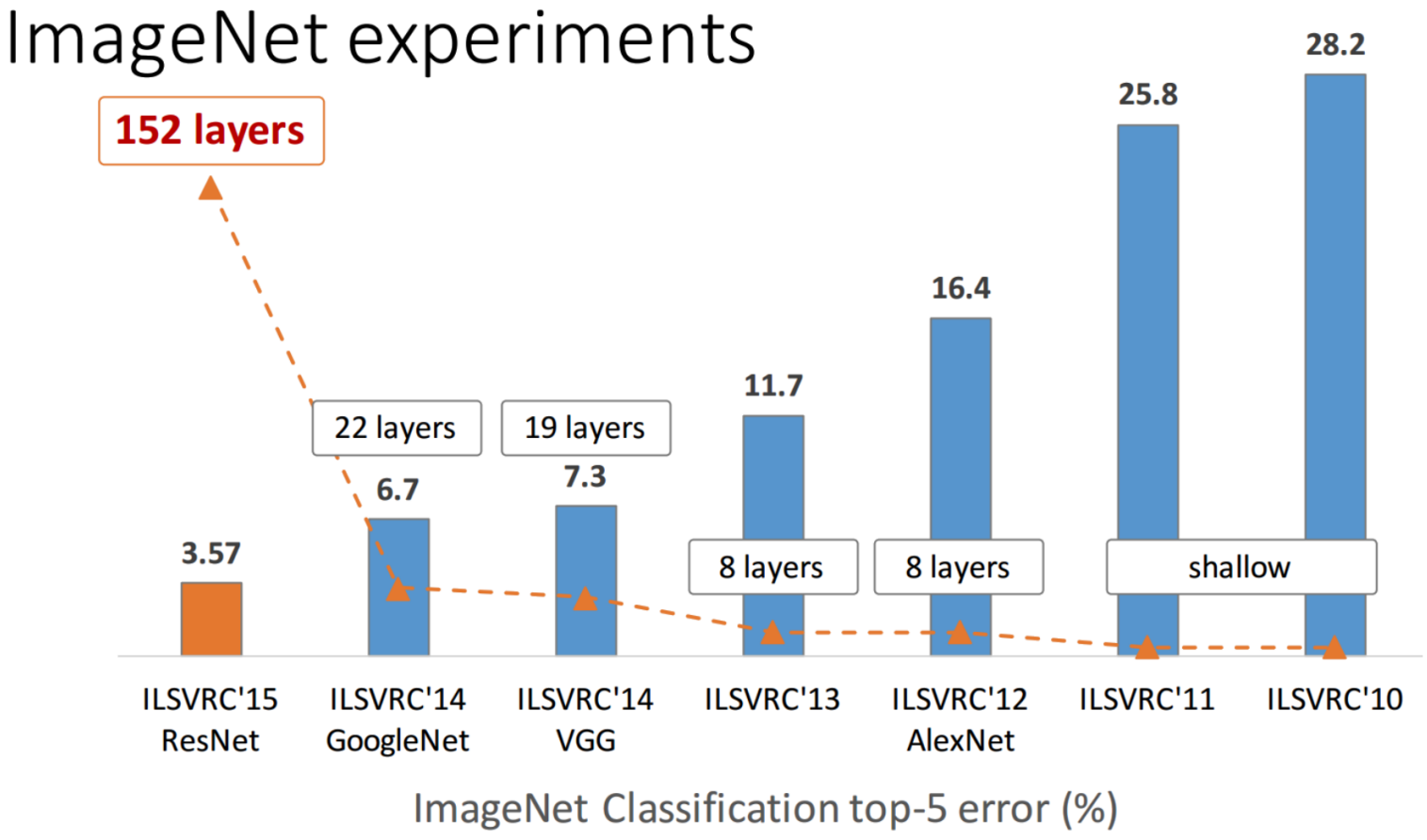
- Superior accuracy in all vision tasks
- 5.25% top-5 error vs 7.1%
- Less parameters
- 25M vs 138M
- Computational complexity
- 3.8B Flops vs 15.3B Flops
- Fully Convolutional until the last layer

He, Kaiming, et al. "Deep residual learning for image recognition." CVPR. 2016.



Università
Ca'Foscari
Venezia

Deeper is better



He, Kaiming. "Deep residual learning for image recognition." ICML. 2016. (slides)

Università
Ca'Foscari
Venezia

Pre-trained models

Training a model on ImageNet from scratch takes days or weeks.

Many models trained on ImageNet and their weights are publicly available!

Università
Ca'Foscari
Venezia

Transfer learning

Use pre-trained weights, remove last layers to compute representations of images

Train a classification model from these features on a new classification task

The network is used as a generic feature extractor

Better than handcrafted feature extraction on natural images

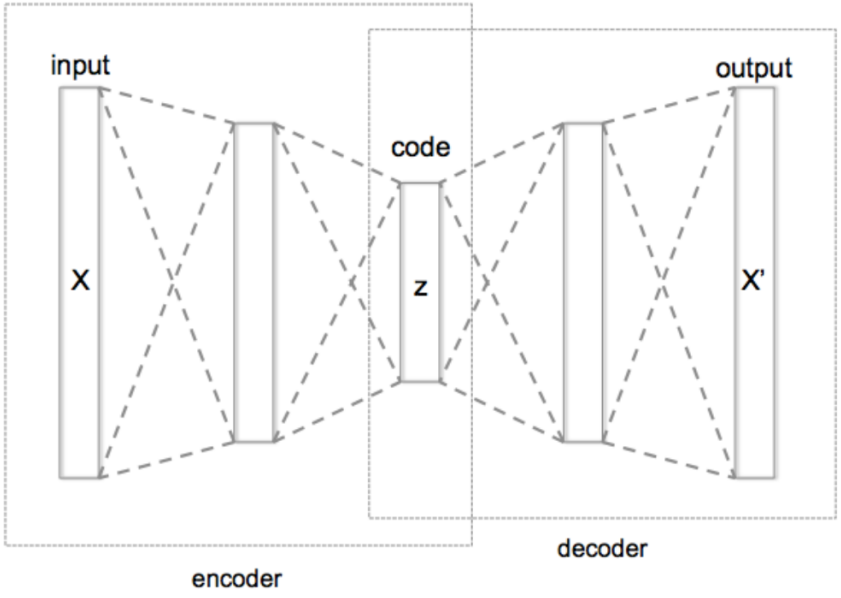
Fine-Tuning

Retraining the (some) parameters of the network (given enough data)

- Truncate the last layer(s) of the pre-trained network
- Freeze the remaining layers weights
- Add a (linear) classifier on top and train it for a few epochs
- Then fine-tune the whole network or the few deepest layers
- Use a smaller learning rate when fine tuning

Università
Ca'Foscari
Venezia

Autoencoder



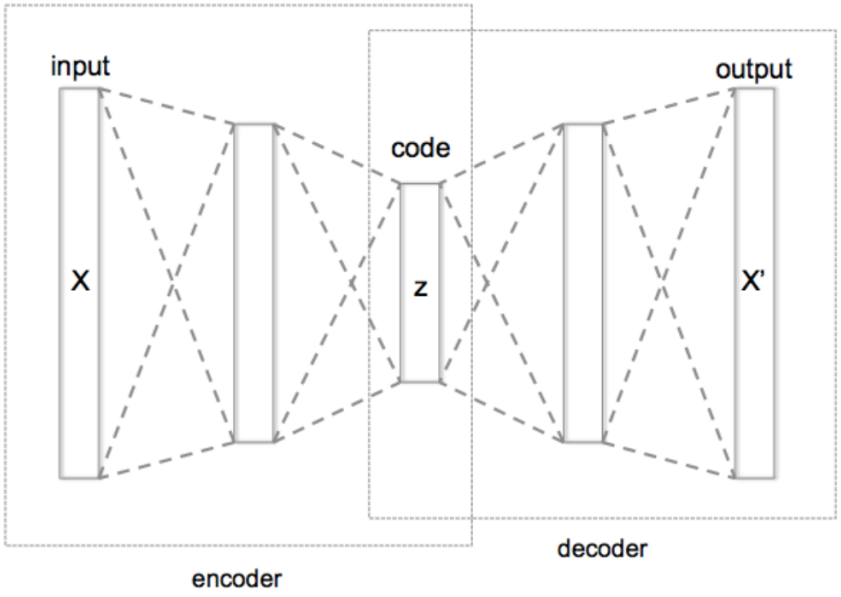
Supervision : reconstruction loss of the input, usually:

$$l(x, f(x)) = \|f(x) - x\|_2^2$$

Binary crossentropy is also used

Università
Ca'Foscari
Venezia

Autoencoder



Keeping the latent code z low-dimensional forces the network to learn a "smart" compression of the data, not just an identity function

Encoder and decoder can have arbritrary architecture (MLPs, CNNs, ...)

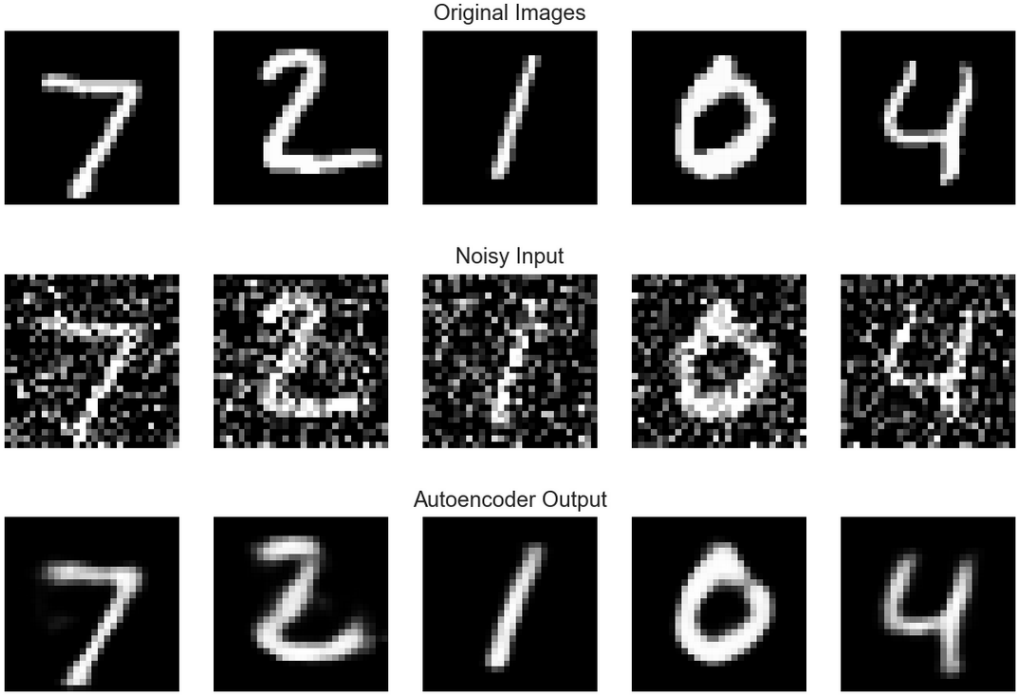
Sparse/Denoising Autoencoder

Adding a sparsity constraint on activations:

$$\|encoder(x)\|_1 \sim \rho,\ \rho = 0.05$$

Learns sparse features, easily interpretable

Denoising Autoencoder: train features for robustness to noise.



Uses and Limitations

Uses:

- After pre-training use the latent code z as input to a classifier instead of x
- Semi-supervised learning simultaneous learning of the latent code (on a large, unlabeled dataset) and the classifier (on a smaller, labeled dataset)
- Other use: Use decoder $D(x)$ as a Generative model: generate samples from random noise

Limitations:

- Direct autoencoder fails to capture good representations for complex data such as images
- The generative model is usually of very poor quality (very blurry for images for instance)

Variational Autoencoders (VAE)

Assume the data samples $x^{(i)}$ are generated by the model:

$$p_{\theta^*}(x, z) = p_{\theta^*}(z)p_{\theta^*}(x|z)$$

- x is an observed r.v. with values in R^n;
- z is a latent r.v. with values in R^d;
- True continuous parameters θ^* are unknown;
- Estimate parameters θ from data $x^{(i)}$ by maximizing the marginal likelihood (MLE):

$$p_\theta(x) = \int p_\theta(z)p_\theta(x|z)\, dz$$

but this high dimensional integral cannot be estimated efficently.

Variational Autoencoders (VAE)

Introduce approximate (variational) posterior on the latent variable: $q_\phi(z|x)$ with continuous parameters ϕ.

We can rewrite the log-likelihood as:

$$\log p_\theta(x^{(i)}) = D_{KL}\left(q_\phi(z|x^{(i)}) \,\|\, p_\theta(z|x^{(i)})\right) + \mathcal{L}(\theta, \phi; x^{(i)})$$

with:

$$\mathcal{L}(\theta, \phi; x^{(i)}) = D_{KL}\left(q_\phi(z|x^{(i)}) \,\|\, p_\theta(z)\right) + \mathbb{E}_{q_\phi(z|x^{(i)})} \log p_\theta(x^{(i)}|z)$$

- $D_{KL}\left(q_\phi(z|x^{(i)}) \,\|\, p_\theta(z|x^{(i)})\right)$ is positive but unknown. It depends on the quality of our approximation.
- $\mathcal{L}(\theta, \phi; x^{(i)})$ is a lower bound to maximize w.r.t. θ and ϕ.

Variational Autoencoders (VAE)

Assume prior distribution for latent variable z:

$$p_\theta(z) = \mathcal{N}(0, 1)$$

Parametrize $p_\theta(x|z)$ by a neural network f_θ (decoder):

$$p_\theta(x^{(i)}|z) = \mathcal{N}(f_\theta(z), 1)$$

Parametrize $q_\phi(z|x^{(i)})$ by a neural network with two heads μ_ϕ and σ_ϕ (encoder):

$$q_\phi(z|x^{(i)}) = \mathcal{N}(\mu_\phi(x^{(i)}), \sigma_\phi(x^{(i)}))$$

Reparametrization trick:

$$z = \mu_\phi(x^{(i)}) + \sigma_\phi(x^{(i)}) \cdot \epsilon \text{ with } \epsilon \sim \mathcal{N}(0, 1)$$

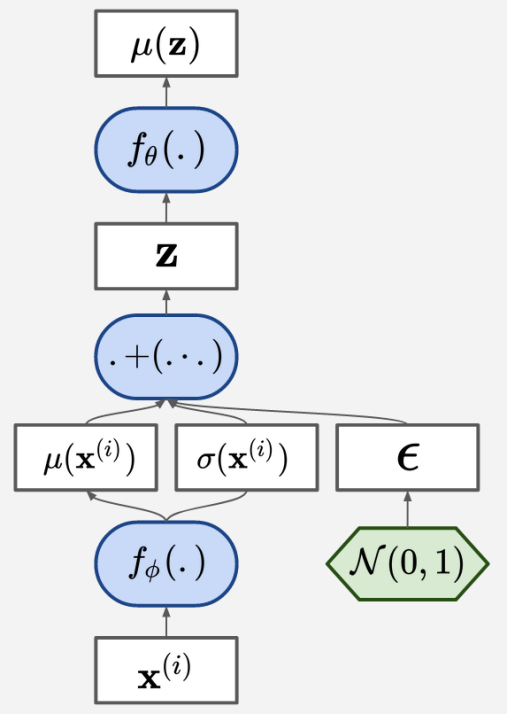
Variational Autoencoders (VAE)

The decoder f_θ defines the likelihood of data.

The latent variable z is stochastic.

The encoder f_ϕ defines an approximate posterior on z.

The reparametrization makes the objective differentiable w.r.t. θ and ϕ.



Variational Autoencoders (VAE)

Remarks

- Similar to Denoising AE but noise added to hidden layer;
- Motivated by a well-defined probablistic model of the generative process;
- Quite easy to train in practice.

Limitations

- Is the continuous parametrization of posterior latent distribution too restrictive?
- Would a discrete latent variable make more sense?
- Gaussian parametrization of the decoder output results in blurry images.

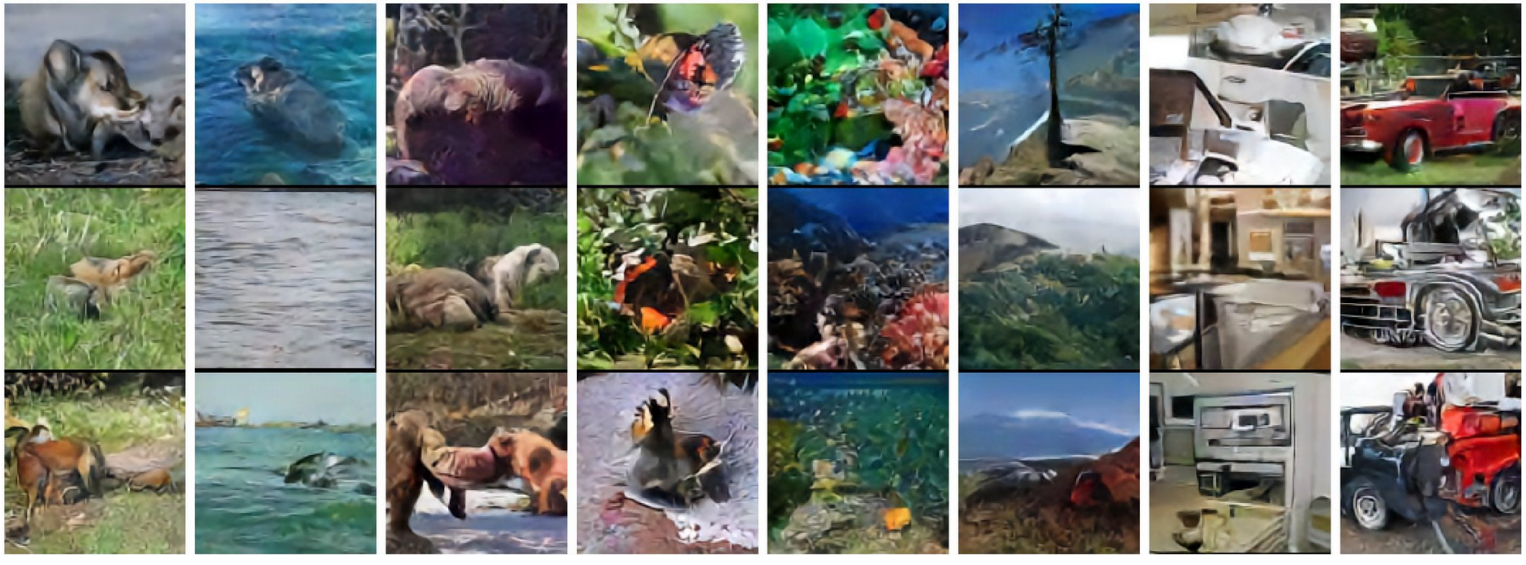
VQ-VAE

z is a vector indexed in a trainable embedding matrix.

Select z as embedding vector closest to encoder output.

Approximate backprop via "gradient-copy" trick.

Very expressive model, especially when combined with strong decoders and priors.



van den Oord et al. Neural Discrete Representation Learning.

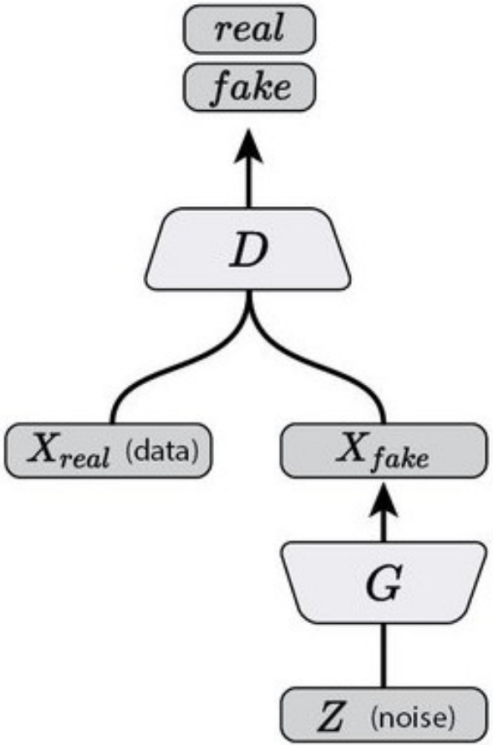
Generative Adversarial Networks

Alternate training of a generative network G and a discrimininative network D

D tries to find out which example are generated or real

G tries to fool D into thinking its generated examples are real

Goodfellow, Ian, et al. Generative adversarial nets. NIPS 2014.



Generative Adversarial Networks

D tries to find out which example are generated or real

G tries to fool D into thinking its generated examples are real

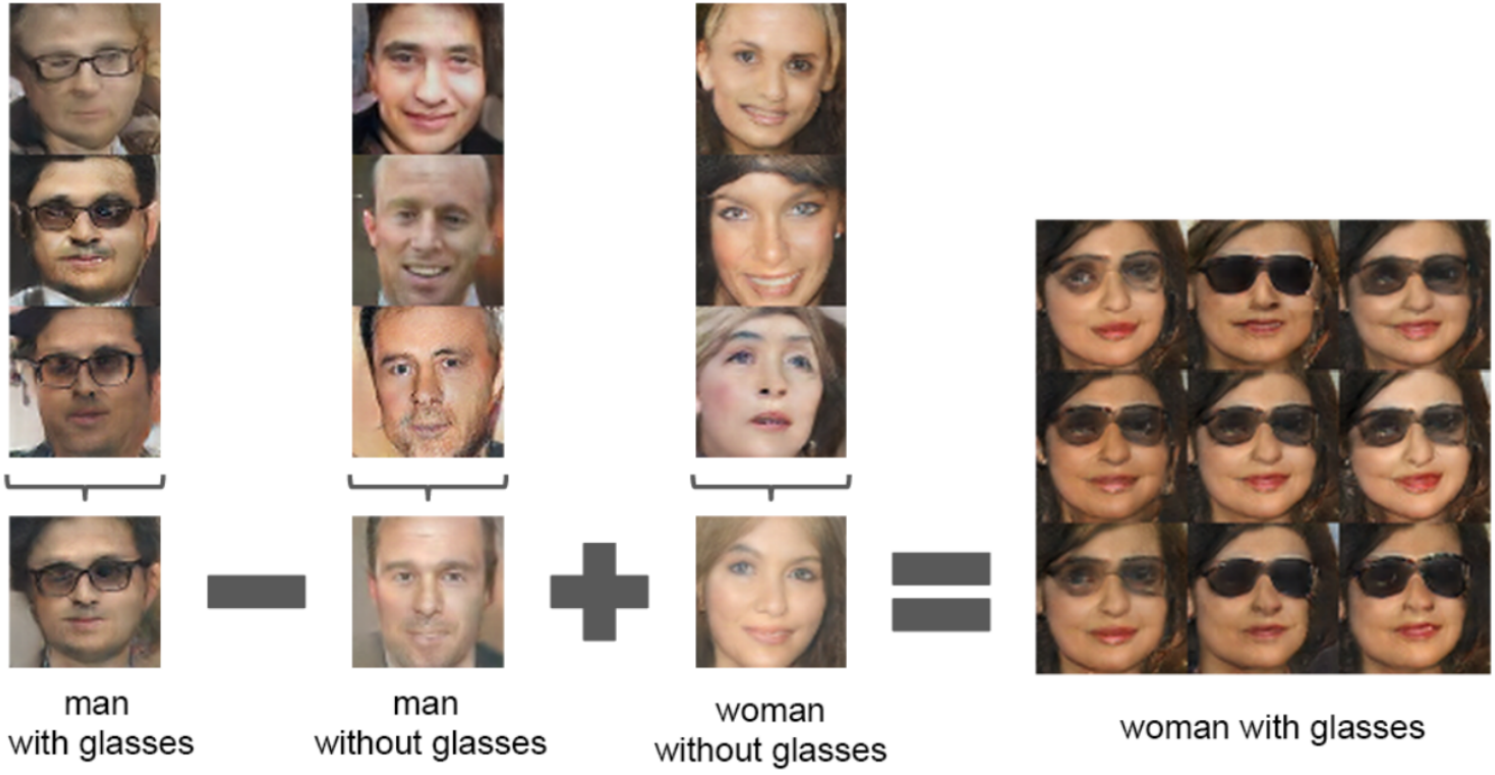
- Sample real data $x \sim p_{data}$
- Sample z and generate fake data $G(z)$

$$\min_G \max_D V(D,G) = \mathbb{E}_{x \sim p_{data}(x)}[\log D(x)] + \mathbb{E}_{x \sim p_z(z)}[\log\left(1 - D(G(z))\right)]$$

G never "sees" training data, it is solely updated from gradients coming from D

Università
Ca'Foscari
Venezia

DC-GAN

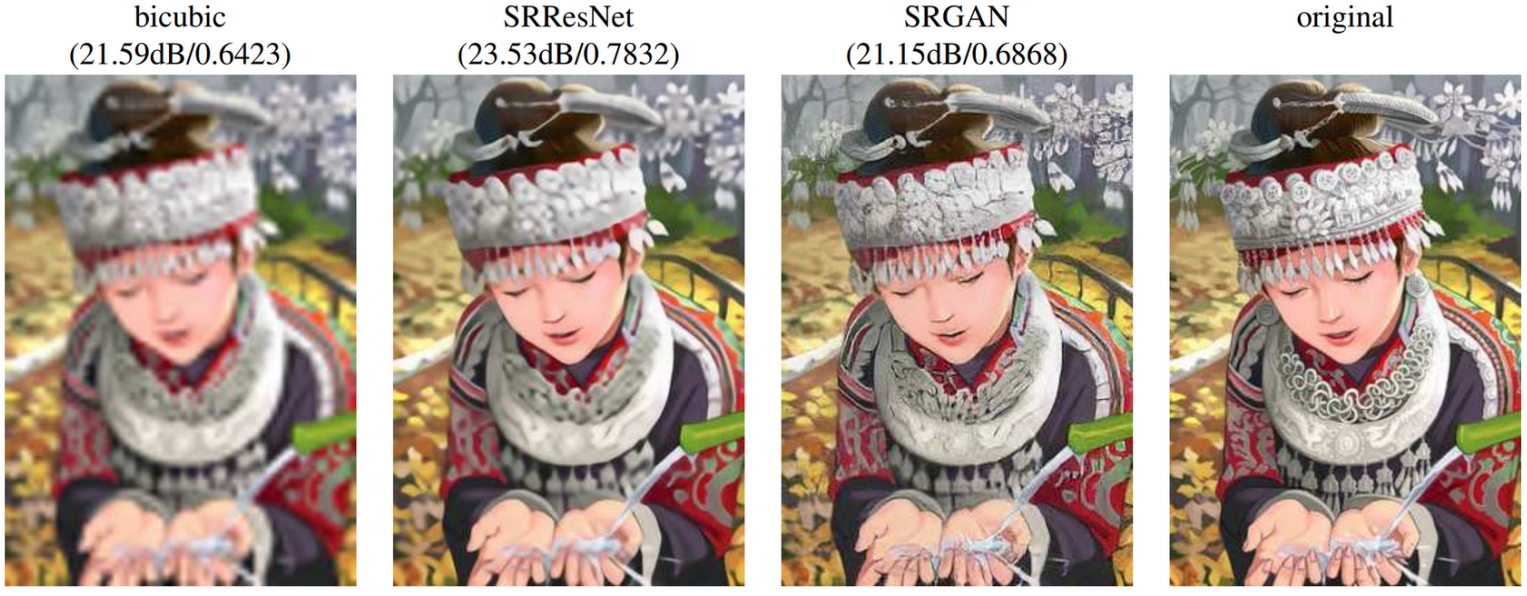


Radford, et al. Unsupervised representation learning with deep convolutional generative adversarial networks. 2015.

Generator generates less-blurry images than VAEs

Latent space has some local linar properties (vector arithmetic)

Super Resolution



Ledig et al. Photo-realistic single image super-resolution using a generative adversarial network. CVPR 2016.

"Perceptual" loss = combining pixel-wise loss mse-like loss with GAN loss

Takeaways

(Reconstruction) Autoencoders

- have no direct probabilistic interpretation;
- are not designed to generate useful samples;
- encoder defines a useful latent representation.

VAEs

- model explicitly (a lower bound of) the likelihood;
- high quality samples from high dimensional distributions;
- encoder defines a useful latent representation;
- optimization problem is often well-behaved.

Takeaways

GANs

- likelihood-free generative models;
- high quality samples from high dimensional distributions;
- discriminator not meant be used as encoder;
- optimization problem is trickier than for VAEs (open research).